

www.devexpress.com

ExpressPrintingSystem v2.0

by Developer Express



General Reference

Copyright © 2001 Developer Express Inc.

Table of Contents

Table of Contents	2
The ExpressPrinting System - Introduction	5
Getting Started with the Express Printing System	6
What Is New In Version 2	19
TdxBackground Object	20
TdxBackground Properties	21
TdxBackground Methods	25
TdxBackground Events	30
TdxPrinterPage Object	33
TdxPrinterPage Properties	34
TdxPrinterPage Methods	44
TdxPrinterPage Events	51
TdxPointWrapper	52
TdxPointWrapper Properties	53
TdxPointWrapper Methods	54
TdxPointWrapper Events	57
TdxRectWrapper	58
TdxRectWrapper Properties	59
TdxRectWrapper Methods	63
TdxRectWrapper Events	65
TCustomdxComponentPrinter Object	67
TCustomdxComponentPrinter Properties	68
TCustomdxComponentPrinter Methods	76
TCustomdxComponentPrinter Events	97
TdxComponentPrinter Object	107

TdxComponentPrinter Properties	108
TdxComponentPrinter Methods	109
TdxComponentPrinter Events	112
TBasedxPrintStyle Object	114
TBasedxPrintStyle Properties	115
TBasedxPrintStyle Methods	120
TBasedxPrintStyle Events	123
TdxPSPrintStyle	124
TdxPSPrintStyle Properties	125
TdxPSPrintStyle Methods	125
TdxPSPrintStyle Events	126
TdxPrintStyleManager Object	128
TdxPrintStyleManager Properties	129
TdxPrintStyleManager Methods	134
TdxPrintStyleManager Events	140
TdxPageSetupDialog Object	141
TdxPageSetupDialog Properties	142
TdxPageSetupDialog Methods	147
TdxPageSetupDialog Events	148
TAbstractdxStyleManagerDesigner	149
TdxPrintDialog Object	150
TdxPrintDialog Properties	151
TdxPrintDialog Methods	162
TdxPrintDialog Events	163
TdxPSEngine Object	165
TdxPSEngine Properties	166
TdxPSEngineController Object	169

TdxPSEngineController Properties	170
TdxPSEngineController Methods	172
TdxBaseObject Object	173
TdxBaseObject Methods	174
TdxPrinterPageObject	177
TdxPrinterPageObject Properties	178
TdxPrinterPageObject Methods	179
TCustommdxPageObject	180
TCustommdxPageObject Properties	181
TCustommdxPageObject Methods	188
TdxPageHeader	189
TdxPageHeader Properties	190
TdxPageHeader Methods	190
TdxPageFooter	191
TdxPageFooter Properties	192
TdxPageFooter Methods	192
TdxPreviewOptions Object	193
TdxPreviewOptions Properties	194
TdxPreviewOptions Methods	201
TdxReportCells Object	202
TBasedxPreviewWindow Object	203
TAbstractdxPreviewWindowDesigner Object	204
TdxReportTitle Object	205
TdxReportTitle Properties	206
TdxReportTitle.Methods	208
Global Routines and Variables	209
Index	212

The ExpressPrinting System - Introduction

The ExpressPrinting System allows you to bring the advanced presentation features of Developer Express products such as the ExpressQuantumGrid and QuantumTreeList, and other TWinControls, to the printed page. It enables you to easily design and print elegant reports based on standard and third-party VCL components.

Like all Developer Express technologies, the ExpressPrinting System insures your ability to put control back in the hands of your end-users via a runtime interface that is simple to use, with an astounding level of flexibility. It gives you the power to quickly add superior reporting interfaces to your programs without designing a single report. Additionally, the ExpressPrinting System includes an open architecture so that you can add extensions to print components from Borland and other third-party vendors. Simply said, the ExpressPrinting System provides you with unlimited options for data presentation on the printed page.

To create a report with the ExpressPrinting System, just drop the TdxComponentPrinter component onto a form and specify components to be printed. With the help of the *Preview* method, call the preview window that provides a visual representation of a report. You can print it directly from the preview window or via specific methods (*PrintEx*, *PrintPages*, *PrintPagesEx*) to maintain closer control over the printing process. The ExpressPrinting System provides full support for design and runtime preview windows. You can obtain any preview window you need – the only thing you have to do is to define some of its properties at design-time.

The ExpressPrinting System includes events that help you display progress indicators when building or printing a report, which consumes an excessive amount of time.

The ExpressPrinting System incorporates *report links* to print components. *Report links* offer a very wide range of methods and properties to customize a report's look and page attributes and includes specific *designers* that provide visual customization of a report.

IMPORTANT: The *printing component* only prints VCL (or third-party) component classes provided with *report links*.

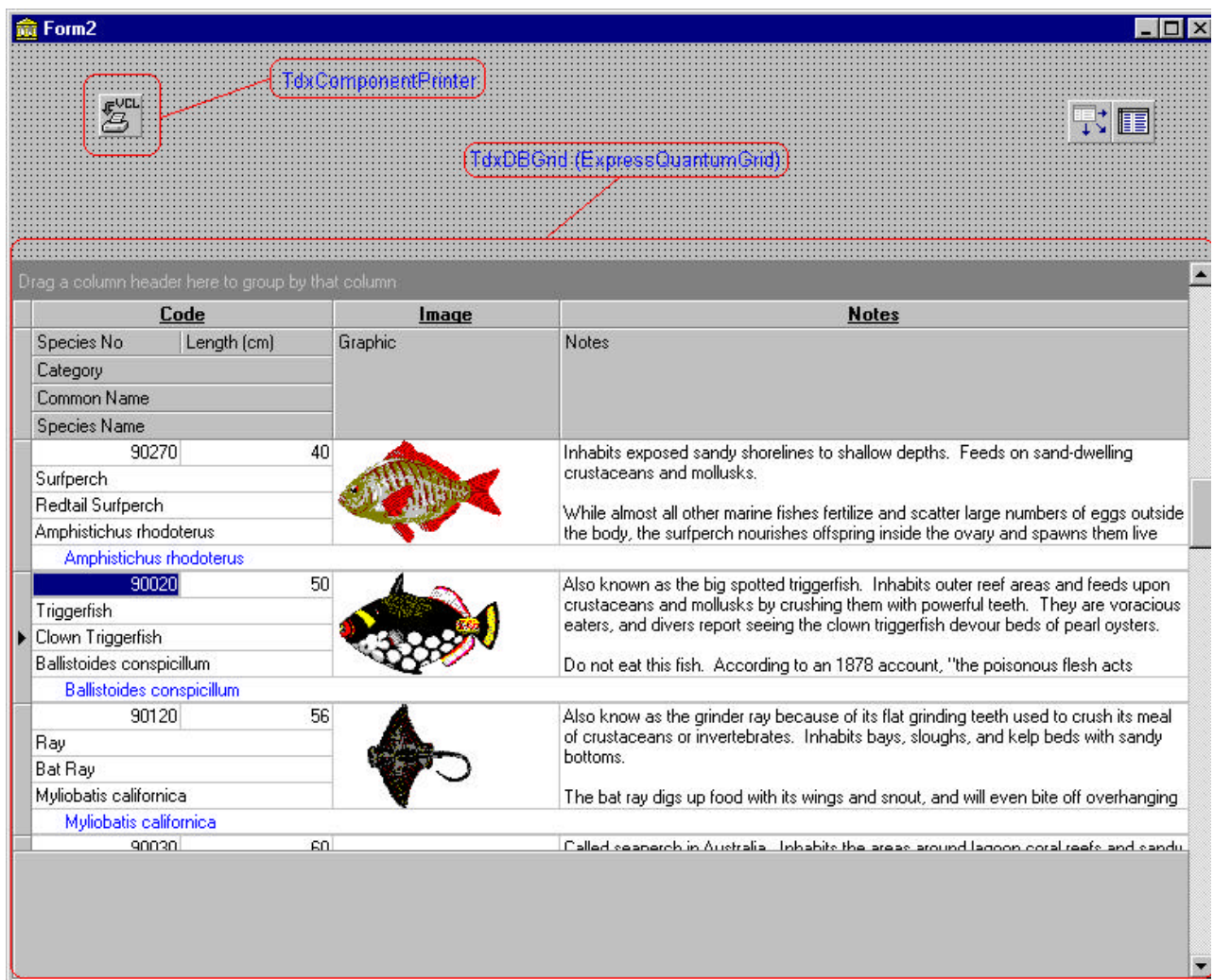
Here is the list available *report links*:

Report links	Components
TBasedxReportLink	-
TdxStringGridReportLink	TStringGrid
TdxDrawGridReportLink	TDrawGrid
TdxListBoxReportLink	TListBox
TdxCheckListBoxReportLink	TCheckListBox
TdxDBTreeListReportLink	TdxDB TreeList ExpressQuantumDBTreeList
TdxTreeListReportLink	TdxTreeList ExpressQuantumTreeList
TdxDBGridReportLink	TdxDBGrid ExpressQuantumGrid
TdxFlowChartReportLink	TdxFlowChart ExpressFlowChart
TdxOrgChartReportLink	TdxOrgChart ExpressOrgChart
TdxDBOrgChartReportLink	TdxDBOrgChart ExpressDBOrgChart
TdxTeeChartReportLink	TChart
TdxDBTeeChartReportLink	TDBChart

Getting Started with the Express Printing System

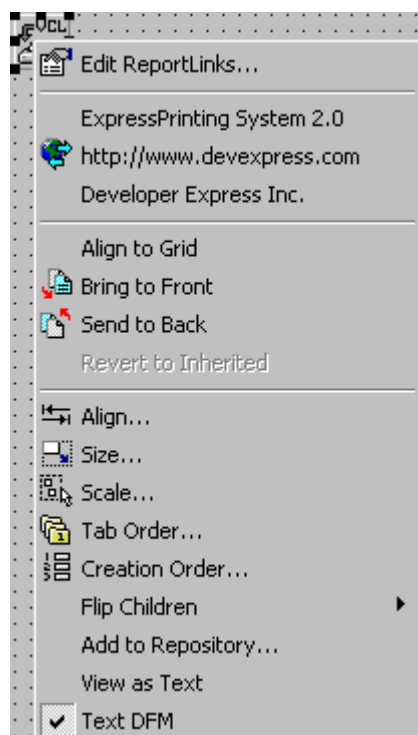
In this section you will learn how easy it is to use the Express Printing System at design-time.

First, make certain that you have placed one of the following components on a **Form**: TStringGrid, TDrawGrid, TListBox, TCheckListBox, TdxDBTreeList (ExpressQuantumDBTreeList), TdxTreeList (ExpressQuantumTreeList), TdxDBGrid (ExpressQuantumGrid), TdxFlowChart (ExpressFlowChart), TdxOrgChart (ExpressOrgChart), TdxDBOrgChart (ExpressDBOrgChart), TChart or TDBChart. Locate the TdxComponentPrinter component on the component palette and place it on the same **Form**, as shown below. In this example we use the TdxComponentPrinter component with a TdxDBGrid (ExpressQuantumGrid) component.



Next, you should add a link to the component you wish to print. Right-click on the component and activate its popup menu and select the "Report Links ..." option.



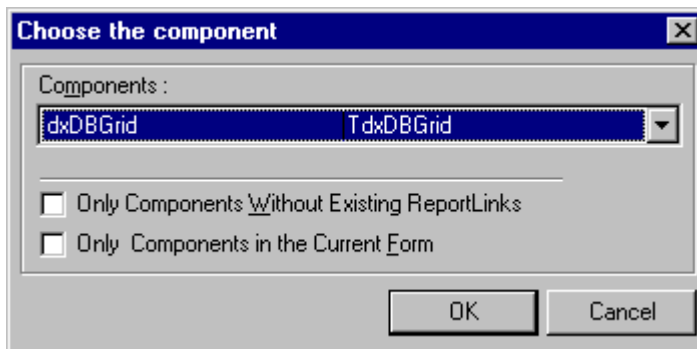


Now you will see the report links editing window. Only one button ("**Add...**") is enabled and it means that there are no links attached to our printing component.

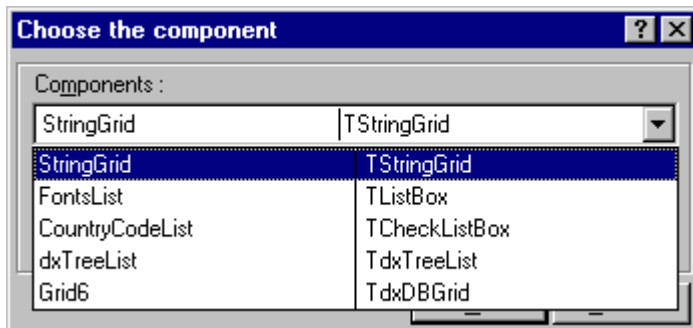


Pressing the "**Add...**" button brings up a window wherein you can choose a new component for printing. Two check-boxes determine whether to display all components or only those which don't

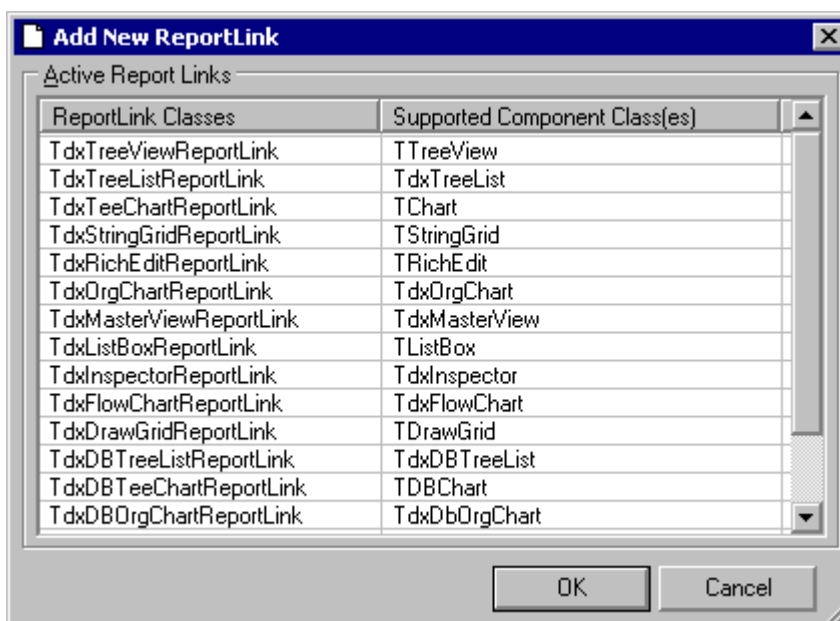
have links and whether to display components from the current form only or from all modules specified in the 'uses' section of the current unit.



You can choose the desired components to print via the dropdown list.



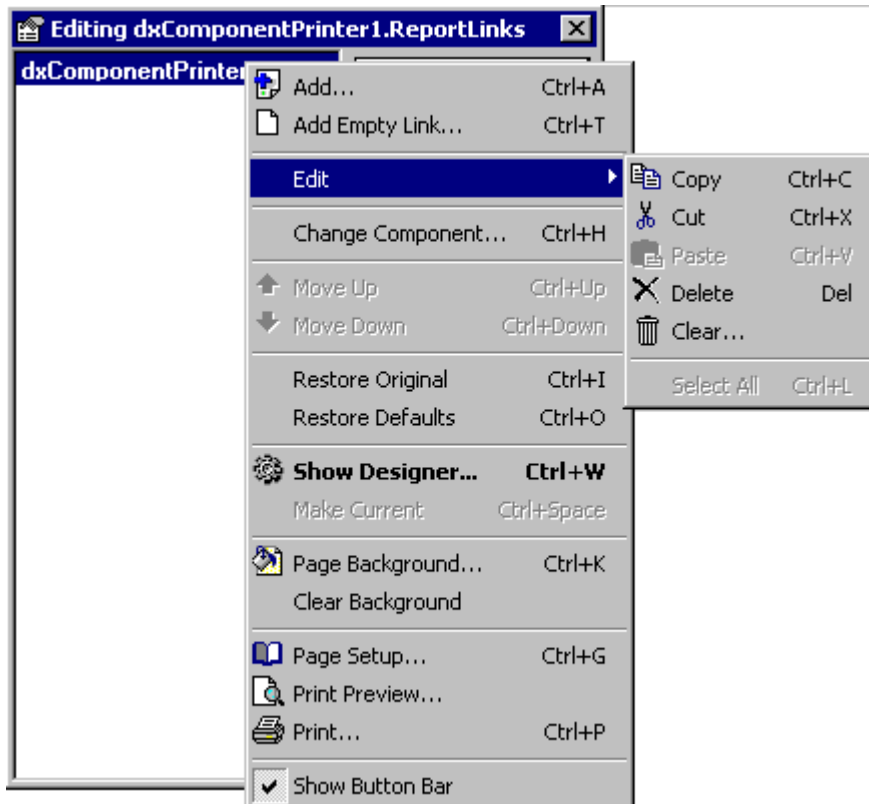
Pressing the "Add Empty..." button activates a window from which you can choose to link any supported component class. This link will not be connected to any component at design-time and will allow you to specify the component to print at runtime.



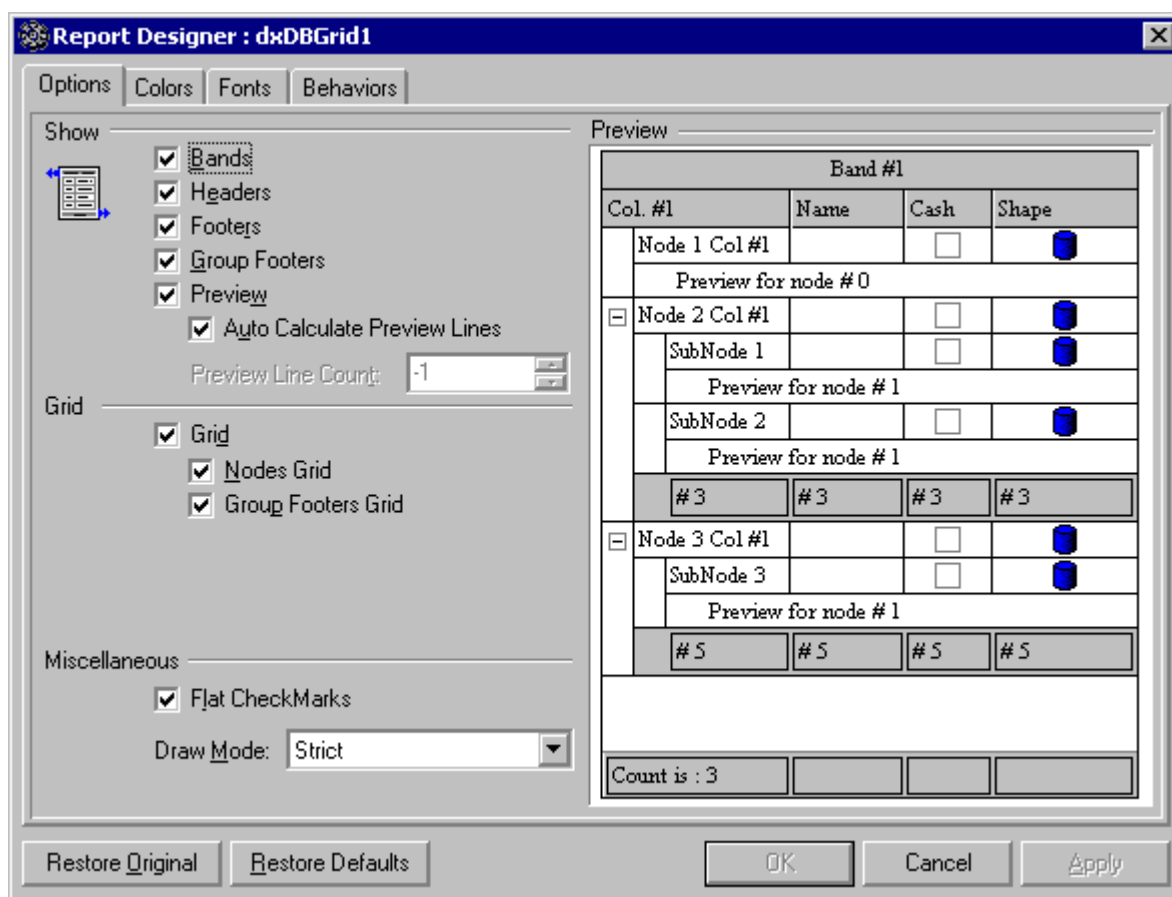
The report links editing window now reflects the changes you have made. A series of buttons are now enabled, and these buttons provide design-time functionality to help you work with links and even to preview and print them.



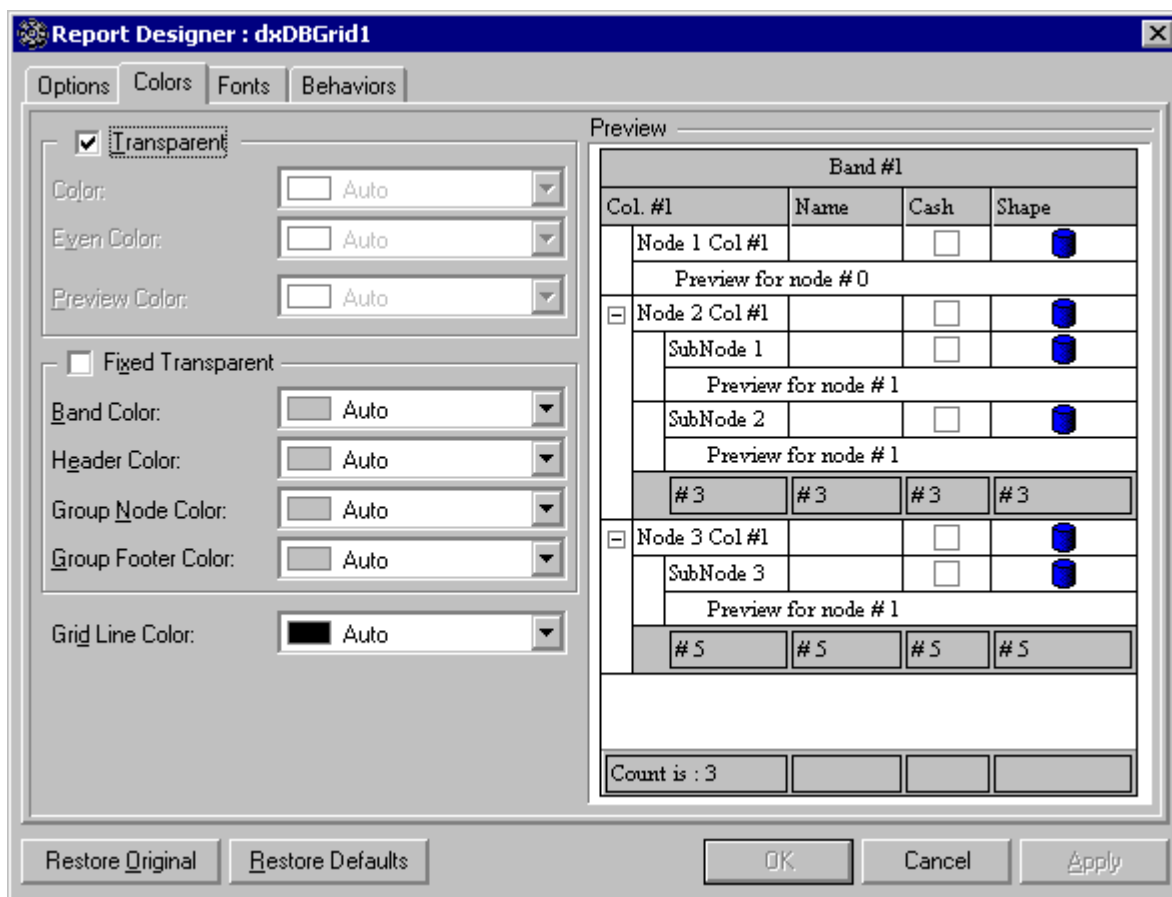
You can use the popup menu associated with the designer. This menu provides the same functionality as the command buttons and allows to operate with report links, as objects, via the following commands: **Copy**, **Cut**, **Paste**, **Delete**, **Clear**, **Select All**.



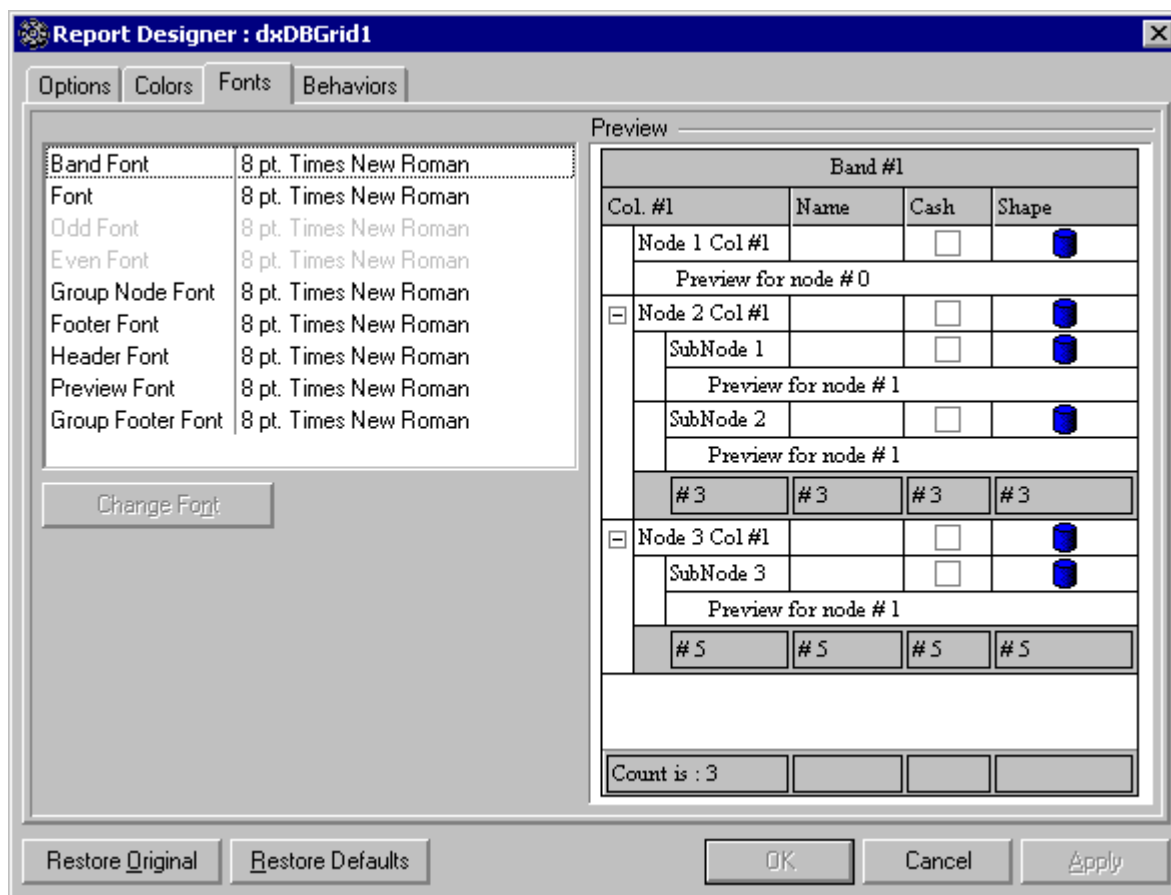
There are a series of methods to transform the appearance of your component image. You can access them through the designer window. Press the **"Show Designer"** button and the designer window will be displayed. Each component has its own designer. This designer is for `TdxDBGrid` (`ExpressQuantumGrid`) only. Every change you make is reflected in the window, so you can easily create a view of your component. As you can see there are 4 pages in this window: the **"Options"** page presented above is responsible for the options that refer to the printed component.



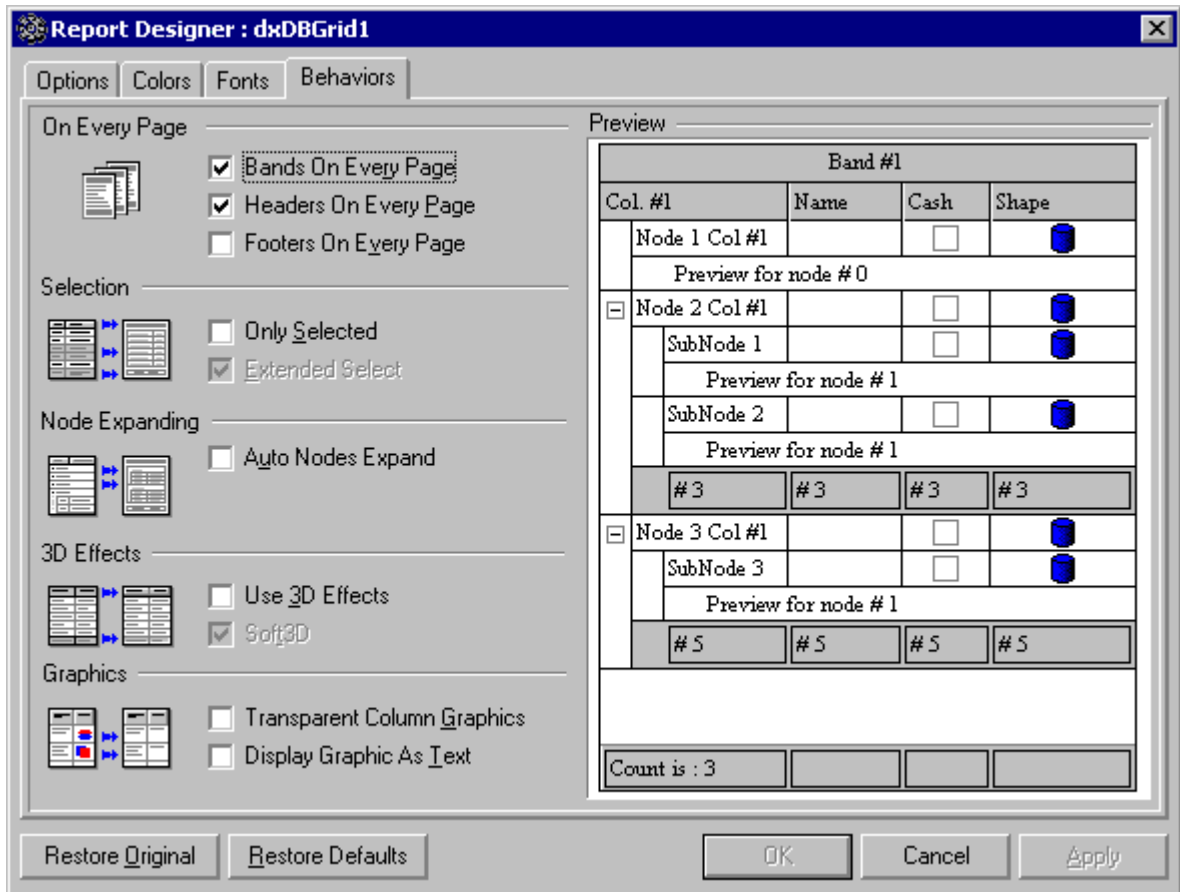
The “**Colors**” page of the designer window helps you to change any colors of the component. These changes are also reflected in the component image.



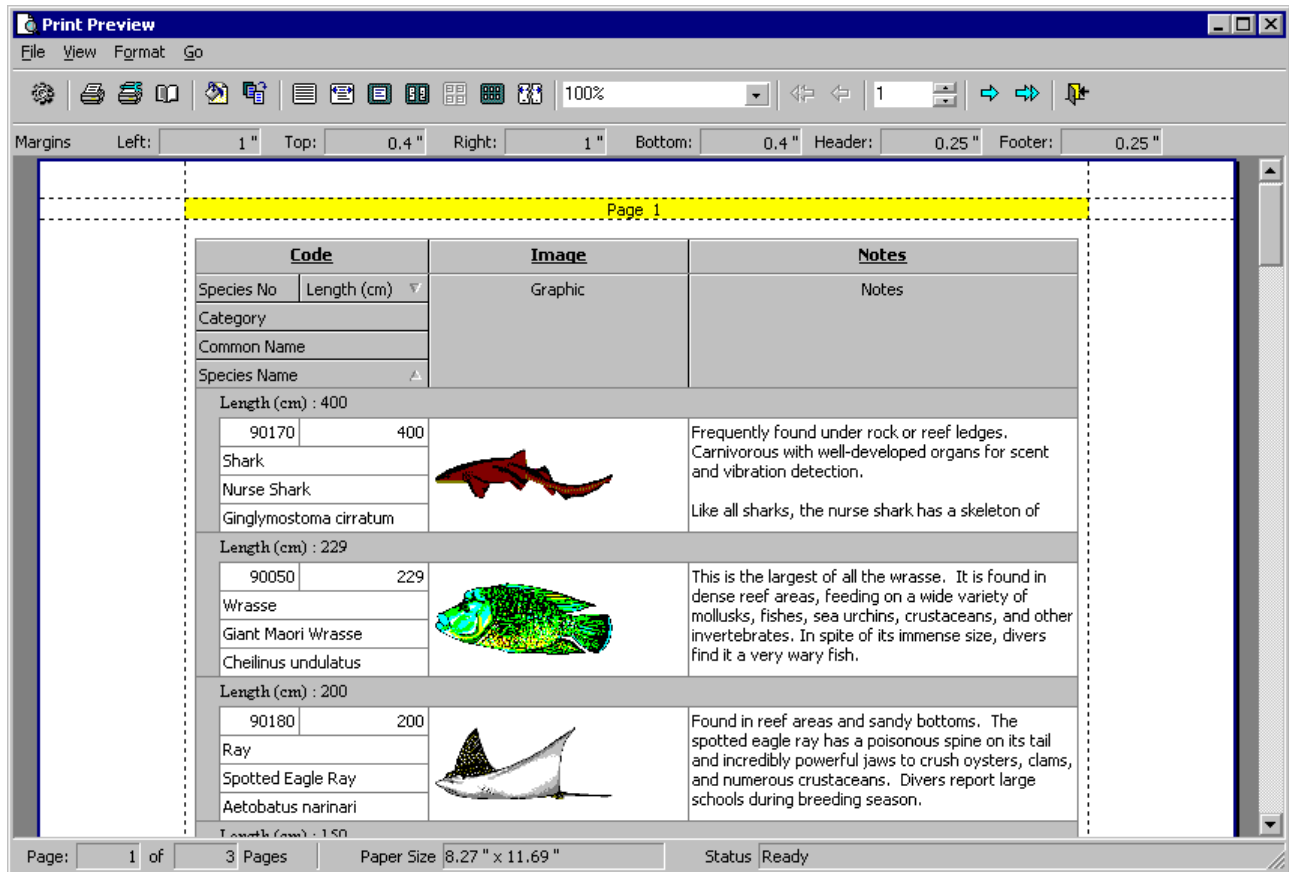
The component you choose to print may have different fonts for different text types. The **"Fonts"** page of the designer window allows you to change font types for printing.



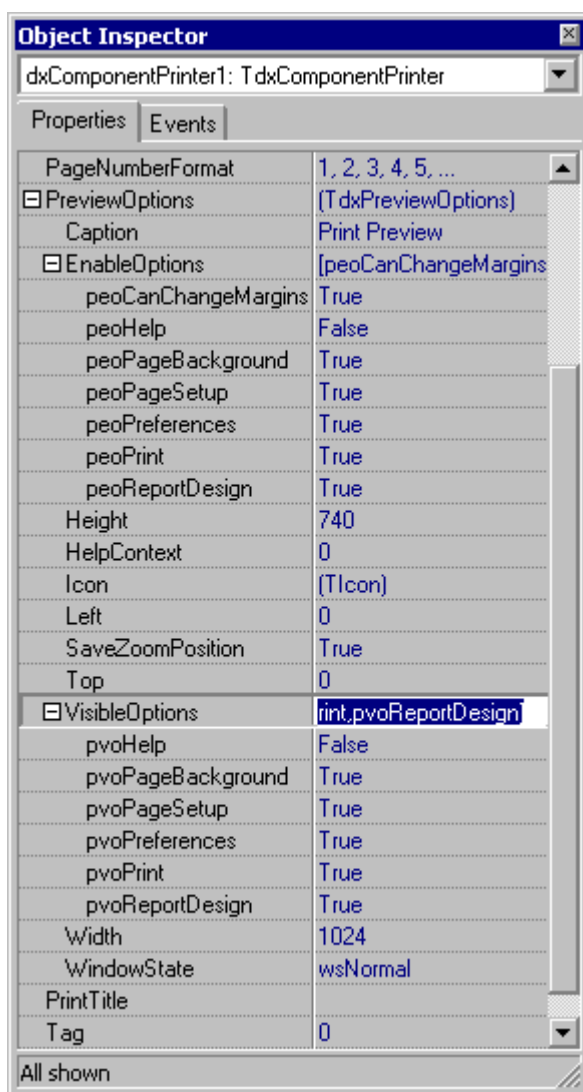
The “**Behaviors**” page of the designer window allows you to control the report object appearance within a report.



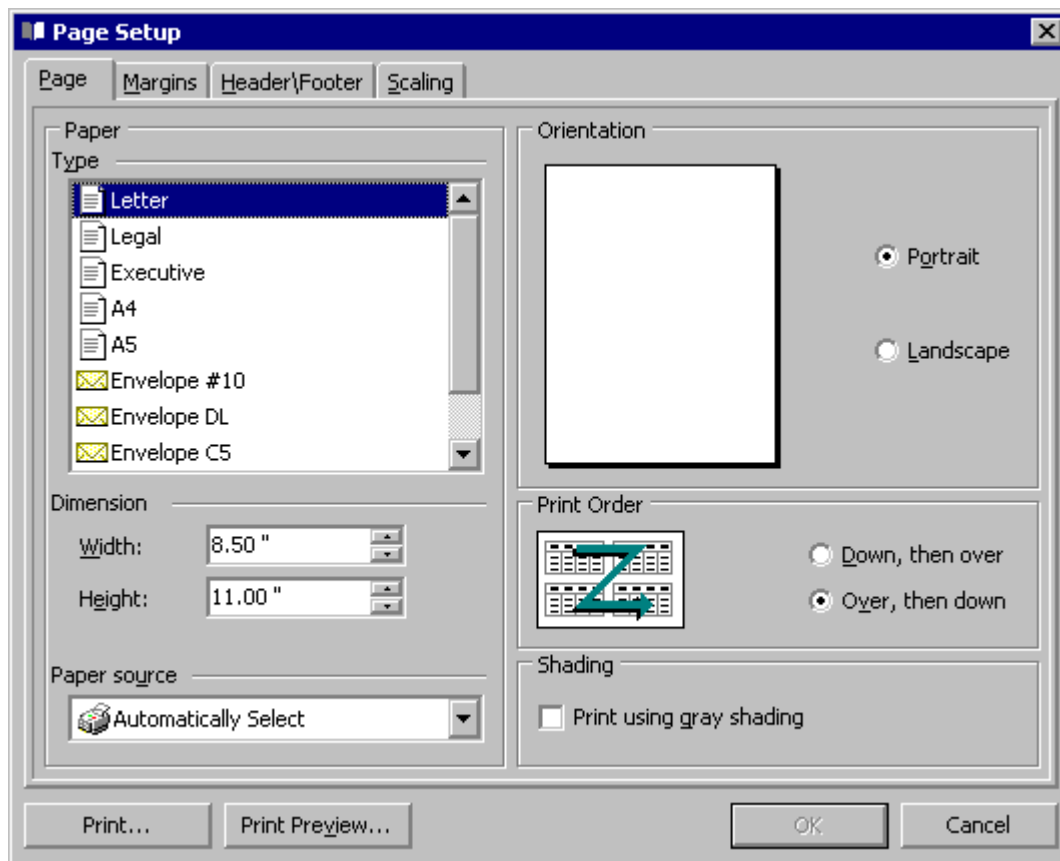
Call the preview window through the popup menu or by pressing the **"Preview"** button if you want to see how your component will look on paper. The preview window helps you customize your report as you wish. It provides the same functionality at design-time that you can access at run-time through your program code.



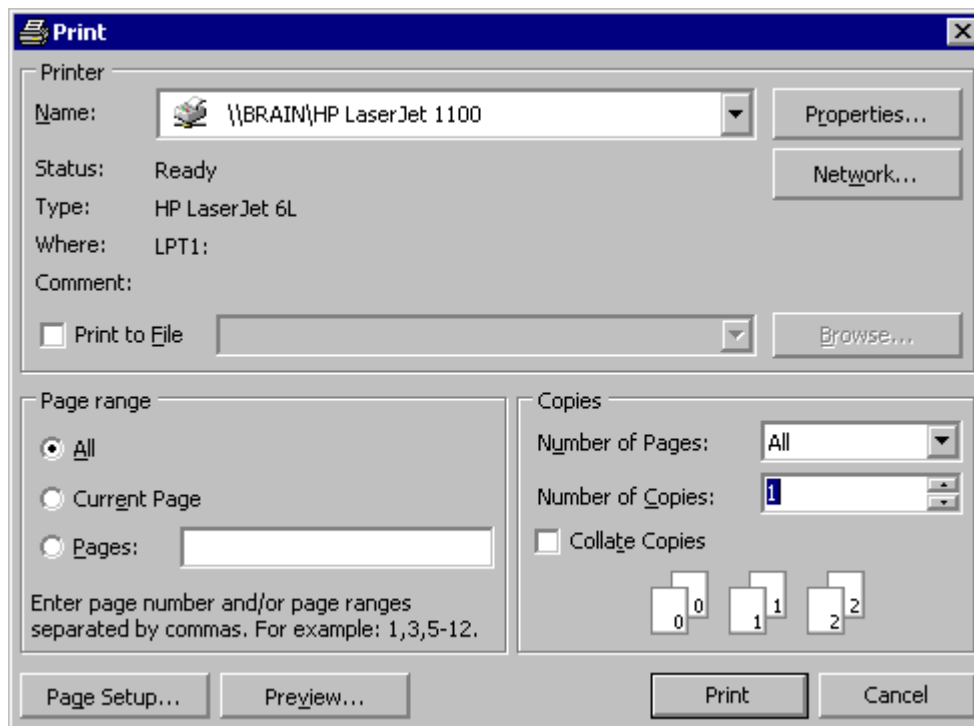
You can also customize the preview window. Use the Object Inspector at design-time to determine which controls will be displayed on the toolbar of the preview window and which menu items are enabled



At design-time you also can setup page layouts for your report. To call this window, you should press the **"Page Setup ..."** button of the designer window.



When you complete all these steps, it is possible to print your report at design-time without a single line of program code. Just press the **"Print..."** button and the print dialog appears:



As you can see, it is very simple to use the `TdxComponentPrinter` component at design-time. It is also easy to write program code for it because properties, methods and events of this component allow you to perform the same operations described herein as well as many others.

What Is New In Version 2

1. Dramatically improved performance (20-50 times faster than version 1) and decreased memory usage.
2. The ExpressPrinting System includes new report links for the following components:
 - ?? **TdxInspector** (ExpressInspector)
 - ?? **TdxDBInspector** (ExpressDBInspector)
 - ?? **TdxTreeView** (ExpressTreeView)
 - ?? **TdxDBTreeView** (ExpressDBTreeView)
 - ?? **TdxMasterView** (ExpressMasterView)
 - ?? **TrxRichEdit**
 - ?? **TRichEdit**
3. New properties for existing ReportLinks designed to greatly extend their core functionality.
4. Full CustomDraw support for all ReportLinks (including dxMasterView).
5. The ExpressPrinting System now includes a Style feature. You can print a given report using multiple styles. For instance, you can print the ExpressQuantumGrid on different paper formats without additional page setup. This feature is managed via a new component - TdxPrintStyleManager.
6. The TdxPageSetupDialog component allows you to adjust the page setup dialog window as your needs dictate.
7. A new PrintDialog component that extends the functionality of the standard PrintDialog component.
8. Full customization for PageHeader/Footer text and PageNumbering/DateTime output formats.
9. CustomDraw support for PageHeader/Footer.
10. New TdxPSEngineController that allows a developer to globally change the Look & Feel of all preview windows.
11. Now you can provide your own help for an application that uses the ExpressPrinting System. Use the HelpFile property to specify the name of the appropriate help file.

TdxBackground Object

TdxBackground specifies the background of report pages.

Unit

dxBkGnd

```
TdxBackground = class(TdxBaseObject);
```

Description

By default, the background of each report page is white. Incorporate this object to paint the background of each report page using a specific color, brush, or texture. The TdxBackground object also allows you to place a picture as the background of report pages.

Hierarchy

```
Object
  TPersistent
    TdxBaseObject
```

TdxBackground Properties

TdxBackground

BkColor	published / read / write	NeedRepaint	public / read only
Brush	published / read / write	Picture	published / read / write
Effects	published / read / write	PictureMode	published / read / write
Mode	published / read / write		

BkColor property

read / write

Specifies the background color for report pages.

```
property BkColor : TColor;
```

Description

Use the **BKColor** property to set or access the color used for report page backgrounds, when background mode equals **bmBrush** and brush style is not **bsSolid** or **bsClear**. By default, the **BkColor** property value is **clWhite**.

Note: The **BKColor** property is used to draw report pages only when the **Mode** property is equal to **bmBrush**.

See Also

TdxBackground.Brush; TdxBackground.Mode

Brush property

read / write

Specifies the brush used to draw report pages.

```
property Brush : TBrush;
```

Description

Use the **Brush** property to specify the brush used to paint the background of report pages. If **Brush** is not solid, the space between brush lines is filled with the color specified by the **BkColor** property.

Note: The **Brush** property is used to draw report pages only when the **Mode** property is equal to **bmBrush**.

See Also

TdxBackground.BkColor; TdxBackground.Mode

Effects property

read / write

Allows you to call the **Fill Effects** dialog at design-time.

```
property Effects : Boolean;
```

Description

The **Effects** option is used to setup a background texture or picture. Click it in the **Object Inspector** to call the **Fill Effects** dialog. This dialog allows you to select a specific background texture or a picture. This property has no effect at runtime. For runtime you should use the SetupEffects method.

See Also

TdxBackground.SetupEffects

Mode property

[read / write](#)

Specifies the manner in which the report background is drawn.

type

```
TdxBackgroundMode = (bmNone, bmBrush, bmBrushBitmap, bmPicture);
```

property Mode : TdxBackgroundMode;

Description

Use the **Mode** property to determine which properties are used when the TdxComponentPrinter component draws the report background.

Values	Meaning
bmNone	The report background is clear (white), no color, brush, texture or picture is used for fill.
bmBrush	BkColor and Brush properties are used to draw the report background. If Brush is hatched, BkColor is used.
bmBrushBitmap	Uses a picture, specified by the Picture property, to draw the report background. This picture is tiled on report pages. Note: Windows® 95 only supports 8x8 pictures as a brush, so if your picture is larger, an 8x8 section will be used to draw the report background. Use the bmPicture mode and set the PictureMode property to ppmTile to tile large pictures on report pages. Windows® 98 and NT allow you to use pictures of any size as a brush. Both bmBrushBitmap and bmPicture, with the PictureMode property set to ppmTile, generate the same results for Windows 98 and NT. To create a more compact print preview file and accelerate drawing, use bmBrushBitmap instead of bmPicture with PictureMode equal to ppmTile, wherever possible.
bmPicture	Uses a picture (the Picture property) to draw the report background. The PictureMode property determines how to draw the picture.

The default value is **bmNone**.

NeedRepaint property**read only**

Determines whether a report background should be repainted.

```
property NeedRepaint : Boolean;
```

Description

The **NeedRepaint** property returns **True** to indicate that the report background needs to be repainted. This property is used internally and is set to **True** whenever changes lead to report repainting. For instance, when the Mode property changes from **bmBrush** to **bmPicture**, the report should be repainted and the **NeedRepaint** property is set to **True**.

Picture property**read / write**

Specifies the picture used to draw a report's background.

```
property Picture : TGraphic;
```

Description

Use the **Picture** property to specify the picture used as the background image for report pages. The Mode and PictureMode properties specify the manner in which pictures are drawn on a report.

See Also

TdxBackground.Mode; TdxBackground.PictureMode; TdxBackground.SetupEffects

PictureMode property**read / write**

Defines the manner in which a picture, specified by the Picture property, is drawn on report pages.

type

```
TdxPicturePaintMode = (ppmCenter, ppmStretch, ppmTile, ppmProportional);
```

```
property PictureMode : TdxPicturePaintMode;
```

Description

Background picture can be drawn using one of the following mechanisms:

Value	Meaning
ppmCenter	Picture is centered within a report page.
ppmStretch	Picture is stretched to fit the entire contents of the report page.
ppmTile	Picture is tiled on report pages. Since, Windows 95™ only allows 8x8 pictures as a background, setting this value allows a user to tile pictures of any size using this operating system. Windows 98™ and Windows NT™ have no such limitation. Whenever possible, use the bmBrushBitmap mode instead, as this will reduce the size of the preview file and provides quicker drawing.

The default value is **ppmCenter**.

Note: The **PictureMode** property is effective only when the **Mode** property is equal to **bmPicture**.

See Also

TdxBackground.Mode; TdxBackground.Picture

TdxBackground Methods

TdxBackground

DoApply	protected	Paint	public
DoChange	protected	PaintEx	public
DoPaint	protected	RepaintNeeded	protected
DoPaintEx	protected	SetupEffects	public

DoApply method

Generates the **OnApply** event.

```
procedure DoApply; dynamic;
```

Description

The **DoApply** method is called when a user applies TdxBackground object properties to report pages; i.e. when a user presses the '**Apply**' button in the **Fill Effects** dialog. By default, it performs no action and only generates the **OnApply** event. Descendants can override the **DoApply** method to perform specific actions before/after generating the **OnApply** event.

DoChange method

Generates the **OnChange** event.

type

```
TdxBackgroundParam = ( bpBkColor, bpBrush, bpMode, bpPictureMode,
bpPicture );
```

```
TdxBackgroundParams = set of TdxBackgroundParam;
```

```
procedure DoChange(AChangeWhats : TdxBackgroundParams); dynamic;
```

Description

This method is called when changing properties of a **TdxBackground** object. The *AChangeWhats* parameter determines which property has been changed. The following values are available:

Value	Meaning
bpBkColor	The BkColor property has been changed.
bpBrush	The Brush property has been changed.
bpMode	The Mode property has been changed.
bpPictureMode	The PictureMode property has been changed.
bpPicture	The Picture property has been changed.

The **DoChange** method sets the **NeedRepaint** property to **True**, if changes made to **TdxBackground** object properties cause background repainting.

See Also

TdxBackground.NeedRepaint; TdxBackground.OnChange

DoPaint method

Generates the **OnPaint** event.

type

```
TdxPaintSequence = ( psBefore, psAfter );
```

```
procedure DoPaint(ADC : hDC; Rect : TRect; Sequence : TdxPaintSequence; var
ADone : Boolean );
```

Description

The **DoPaint** method is called when painting a report background on a device context, specified by the *ADC* parameter. The *Rect* parameter specifies the rectangle where to draw the background.

The *ADone* parameter determines whether this method processes the entire drawing operation. If *ADone* is **True**, the default drawing procedure is not called.

The *Sequence* parameter determines whether this method was called before or after background painting. The following values are available:

Value	Meaning
psBefore	Called before background painting.
psAfter	Called after background painting.

See Also

TdxBackground.OnPaint

DoPaintEx method

Generates the **OnPaintEx** event.

type

```
TdxPaintSequence = (psBefore, psAfter);
```

```
procedure DoPaintEx(ACanvas: TCanvas; Rect: TRect; Sequence:  
TdxPaintSequence; PixelsNumerator, PixelsDenominator: Integer; var ADone:  
Boolean); virtual;
```

Description

The **DoPaintEx** method is called when painting a report background.

The *Rect* parameter specifies the rectangle where to draw the background.

The *ADone* parameter determines whether this method processes the entire drawing procedure. If *ADone* is **True**, the default drawing procedure is not called. In other words, if the *ADone* parameter is set to **True** it is the programmer who is entirely responsible for drawing.

The *PixelsNumerator* and *PixelsDenominator* parameters are used to convert object size in pixels to logical units. In order to calculate logical units of an object width/height, you should multiply this width/height in pixels by the *PixelsNumerator* parameter and divide the result by the *PixelsDenominator* parameter.

The *Sequence* parameter determines whether this method was called before or after background painting. The following values are available:

Value	Meaning
psBefore	Called before background painting.
psAfter	Called after background painting.

Paint method

Paints the report background.

```
procedure Paint(ACanvas : TCanvas; const Rect : TRect); virtual;
```

Description

The **Paint** method paints the report background.

The *ACanvas* parameter specifies the drawing surface.

The *Rect* parameter specifies the rectangle where to draw the background.

PaintEx method

Paints a report's background.

```
procedure PaintEx(ACanvas: TCanvas; const Rect: TRect; PixelsNumerator,  
PixelsDenominator: Integer); virtual;
```

Description

The **PaintEx** method is used to paint the background of a report. This method operates with logical size units, as opposed to the **Paint** method, which operates with pixels.

The *ACanvas* parameter specifies the drawing surface.

The *Rect* parameter specifies the rectangle where to draw the background.

The *PixelsNumerator* and *PixelsDenominator* parameters are used to convert object size in pixels to logical units. In order to calculate width/height of an object in logical units, you should multiply this width/height in pixels by the *PixelsNumerator* parameter and divide the result by the *PixelsDenominator* parameter.

RepaintNeeded method

Specifies whether the report background needs repainting.

type

```
TdxBackgroundParam = (bpBkColor, bpBrush, bpMode, bpPictureMode,  
bpPicture);
```

```
TdxBackgroundParams = set of TdxBackgroundParam;
```

```
function RepaintNeeded(AChangeWhats: TdxBackgroundParams): Boolean;  
virtual;
```

Description

The **RepaintNeeded** method is used internally to specify whether report background needs repainting and which element of the background should be processed.

Value	Description
bpBkColor	Background color has been changed.
bpBrush	Brush style has been changed.
bpMode	Mode has been changed.
bpPictureMode	Picture mode has been changed.
bpPicture	Picture has been changed.

SetupEffects method

Displays the **Fill Effect** dialog on screen.

```
function SetupEffects: Boolean;
```

Description

This method displays the **Fill Effects** dialog on screen. This dialog appears when clicking the Effects property in the **Object Inspector** or {bmc setupeffects.bmp} on the toolbar of the preview window. The **SetupEffects** method returns **True**, if a user presses the **'OK'** button to close the **Fill Effects** dialog. Otherwise, it returns **False**. Use the following code to activate the **Fill Effects** dialog:

```
if dxComponentPrinter1Link1.PrinterPage.Background.SetupEffects;  
then  
    {a user has pressed the 'OK' button }  
    . . .  
else  
    {a user has pressed the 'Cancel' button}  
    . . .
```

See Also

TdxBackground.Effects

TdxBackground Events

TdxBackground

OnApply	protected	OnPaint	public
OnChange	public	OnPaintEx	public

OnApply event

type

```
TNotifyEvent = procedure(Sender: TObject) of object;  
property OnApply : TNotifyEvent;
```

Description

Write an **OnApply** event handler to perform specific actions when applying TdxBackground object properties to a report. This event is available when you directly use the TdxBackground object in your application.

See Also

TdxBackground.DoApply

OnChange event

Occurs when changing properties of a **TdxBackground** object.

type

```
TdxBackgroundParam = ( bpBkColor, bpBrush, bpMode, bpPictureMode,  
bpPicture );  
TdxBackgroundParams = set of TdxBackgroundParam;  
TdxBackgroundChangeEvent = procedure(Sender : TObject; AChangeWhat :  
TdxBackgroundParams) of object;  
property OnChange : TdxBackgroundChangeEvent;
```

Description

The **OnChange** event handler is used to perform special actions when changing properties of the TdxBackground object. The *AChangeWhat* parameter determines which property has been changed. Available values include:

Value	Meaning
bpBkColor	The BkColor property has been changed.
bpBrush	The Brush property has been changed.
bpMode	The Mode property has been changed.
bpPictureMode	The PictureMode property has been changed.
bpPicture	The Picture property has been changed.

You only need this event if you directly use the **TdxBackground** object in your application.

See Also

TdxBackground.DoChange

OnPaint event

Occurs when the report background is being painted.

type

```
TdxPaintSequence = ( psBefore, psAfter );
TCustomdxBackgroundPaintEvent = procedure(Sender : TObject; Canvas :
TCanvas; Rect : TRect; ASequence : TdxPaintSequence; var ADone : Boolean )
of object;
property OnPaint : TCustomdxBackgroundPaintEvent;
```

Description

The **OnPaint** event handler is used to perform special actions when a report background is being painted on a device context, specified by the *ADC* parameter. The *Canvas* parameter specifies the drawing surface. The *Rect* parameter determines the rectangle where to draw the background. The *ADone* parameter determines whether this method processes the entire operation and completes the Paint method.

The *ASequence* parameter determines whether the DoPaint method was called before or after background painting. Available values include:

Value	Meaning
psBefore	Called before background painting.
psAfter	Called after background painting.

See Also

TdxBackground.DoPaint

OnPaintEx event

Occurs when painting report background.

type

```
TCustomdxBackgroundPaintExEvent = procedure(Sender: TObject; Canvas:
TCanvas; Rect: TRect; ASequence: TdxPaintSequence; PixelsNumerator,
PixelsDenominator: Integer; var ADone: Boolean) of object;
property OnPaintEx: TCustomdxBackgroundPaintExEvent;
```

Description

The **OnPaintEx** event handler is used to perform specific actions when painting a report background. The main difference between the **OnPaint** and **OnPaintEx** events is that the latter operates with logical size units.

The *Canvas* parameter specifies the drawing surface.

The *Rect* parameter determines the rectangle where to draw the background.

The *ADone* parameter determines whether the **OnPaintEx** event handler completely draws the background. If *ADone* is **True**, the default paint procedure is not called.

The *PixelsNumerator* and *PixelsDenominator* parameters are used to convert object size in pixels to logical units. In order to calculate width/height of an object in logical units, you should multiply this width/height in pixels by the *PixelsNumerator* parameter and divide the result by the *PixelsDenominator* parameter.

The *ASequence* parameter determines whether the **DoPaintEx** method was called before or after background painting. Available values include:

Value	Meaning
psBefore	Called before background painting.
PsAfter	Called after background painting.

TdxPrinterPage Object

TdxPrinterPage defines report page attributes.

Unit

dxPrnPg

```
TdxPrinterPage = class(TdxPrinterPageObject);
```

Description

Objects of type **TdxPrinterPage** are used to customize different attributes of report pages: Page margins, page size, etc.

Hierarchy

```

TObject
  TPersistent
    TdxBaseObject
      TdxPrinterPageObject
```

TdxPrinterPage Properties

TdxPrinterPage

CenterOnPageH	published / read / write	Orientation	published / read / write
CenterOnPageV	published / read / write	PageFooter	published / read / write
DMPaper	published / read / write	PageHeader	published / read / write
FitToPagesByTall	published / read / write	PageOrder	published / read / write
FitToPagesByWide	published / read / write	PageSize	published / read / write
Footer	published / read / write	PaperSource	published / read / write
GrayShading	published / read / write	RealMeasurementUnits	public / read only
Header	published / read / write	RealPageSize	published / read / write
Margins	published / read / write	ReverseTitlesOnEvenPages	published / read / write
MeasurementUnits	published / read / write	ScaleFactor	published / read / write
MinMargins	published / read / write	ScaleMode	published / read / write

Derived from TdxPrinterPageObject

Background	published / read / write
Page	protected / read only

CenterOnPageH property

read / write

Specifies whether a report is horizontally centered on a page.

property CenterOnPageH: **Boolean**;

Description

Use the **CenterOnPageH** property to specify whether a report is horizontally centered on a page.

The default value for this property is **False**.

See Also

TdxPrinterPage.CenterOnPageV

CenterOnPageV property

read / write

Specifies whether a report is vertically centered on a page.

property CenterOnPageV: **Boolean**;

Description

Use the **CenterOnPageV** property to specify whether a report is vertically centered on a page.

See Also

TdxPrinterPage.CenterOnPageH

The default value for this property is **False**.

DMPaper property

[read / write](#)

Determines page size.

```
property DMPaper : Integer;
```

Description

For example, DMPAPER_LETTER, DMPAPER_A4,

See Also

TdxPrinterPage.PageSize

FitToPagesByTall property

[read / write](#)

Specifies how many pages a report's width can contain.

```
property FitToPagesByTall: Integer;
```

Description

Use the **FitToPagesByTall** property to specify the maximum number of pages a report's width can contain.

The default value for this property is **1**.

See Also

TdxPrinterPage.FitToPagesByWide

FitToPagesByWide property

[read / write](#)

Specifies how many pages the report's height can contain.

```
property FitToPagesByWide: Integer;
```

Description

Use the **FitToPagesByWide** property to specify the maximum number of pages the report's height can contain.

The default value for this property is **1**.

See Also

TdxPrinterPage.FitToPagesByTall

Footer property**read / write**

Determines report footer height.

property Footer: **Integer**;

Description

Use the **Footer** property to determine report footer height. This property value is expressed in current measurement units.

See Also

TdxPrinterPage.Header

GrayShading property**read / write**

Specifies whether a report is printed in color.

property GrayShading: **Boolean**;

Description

Use the **GrayShading** property to specify whether a report is printed in color or monochrome. Set the **GrayShading** property to **True** to print your report in monochrome.

The default value for this property is **False**.

Header property**read / write**

Determines header height.

property Header: **Integer**;

Description

Use the **Header** property to determine the height of a report header. This property value is expressed in current measurement units.

See Also

TdxPrinterPage.Footer

Margins property**read / write**

Specifies page margins.

property Margins : TdxRectWrapper;

Description

Use the **Margins** property to define margins of a page. These margins cannot be less than margins specified by the MinMargins property. All margins are specified in thousandths of measurement units.

For example, if you choose to use inches as your units of measurement, then setting the left margin to 1000 will set the left margin to one inch.

See Also

TdxPrinterPage.MinMargins; TdxPrinterPage.MinMarginsLoMetric; TdxPrinterPage.PageSetup; TdxPrinterPage.PageSize

MeasurementUnits property

[read / write](#)

Determines the measurement units used in a report.

type

```
TdxMeasurementUnits = (muDefault, muInches, muMillimeters);
```

```
property MeasurementUnits : TdxMeasurementUnits;
```

Description

Use the **MeasurementUnits** property to specify units used in a report. The following values are available:

Values	Meaning
muDefault	A report uses current Windows® measurement units.
muInches	Sets inches as the measurement units.
muMillimeters	Sets millimeters as the measurement units.

By default, the **MeasurementUnits** property value is equal to **muDefault**.

See Also

TdxPrinterPage.MinMargins; TdxPrinterPage.PageSize

MinMargins property

[read / write](#)

Specifies minimum page margins.

```
property MinMargins : TdxRectWrapper;
```

Description

Use the **MinMargins** property to specify the minimum margin allowed for a page. Page margins cannot be less than the value set for the **MinMargins** property. All margins are specified in thousandths of measurement units. For instance, if you choose to use millimeters as your unit of measurement, then changing the left margin to 10000 will set it at 10 millimeters.

See Also

TdxPrinterPage.Margins; TdxPrinterPage.MarginsLoMetric; TdxPrinterPage.MeasurementUnits; TdxPrinterPage.MinMarginsLoMetric; TdxPrinterPage.PageSize

Orientation property**read / write**

Determines page orientation.

type

```
TdxPrintDeviceOrientation = (poPortrait, poLandscape);
```

```
property Orientation : TdxPrintDeviceOrientation;
```

Description

Use the **Orientation** property to determine whether to display pages vertically or horizontally. The default value for this property is **poPortrait**.

There are two possible values for TdxPrintDeviceOrientation:

Values	Meaning
--------	---------

poPortrait	Page is arranged vertically.
-------------------	------------------------------

poLandscape	Page is arranged horizontally.
--------------------	--------------------------------

See Also

TdxPrinterPage.PageSizeLoMetric

PageFooter property**read / write**

Specifies page footer properties.

```
property PageFooter : TdxPageFooter;
```

Description

Use the **PageFooter** property to manage page footer settings. Page Footer settings are responsible for routines which manage report titles, text alignment, font, etc.

See Also

TdxPrinterPage.PageHeader

PageHeader property**read / write**

Specifies page header properties.

```
property PageHeader : TdxPageHeader;
```

Description

Use the **PageHeader** property to specify page header settings. Page Header settings are responsible for routines which manage text alignment, font, etc.

See Also

TdxPrinterPage.PageFooter

PageOrder property

read / write

Specifies the manner in which report pages are printed.

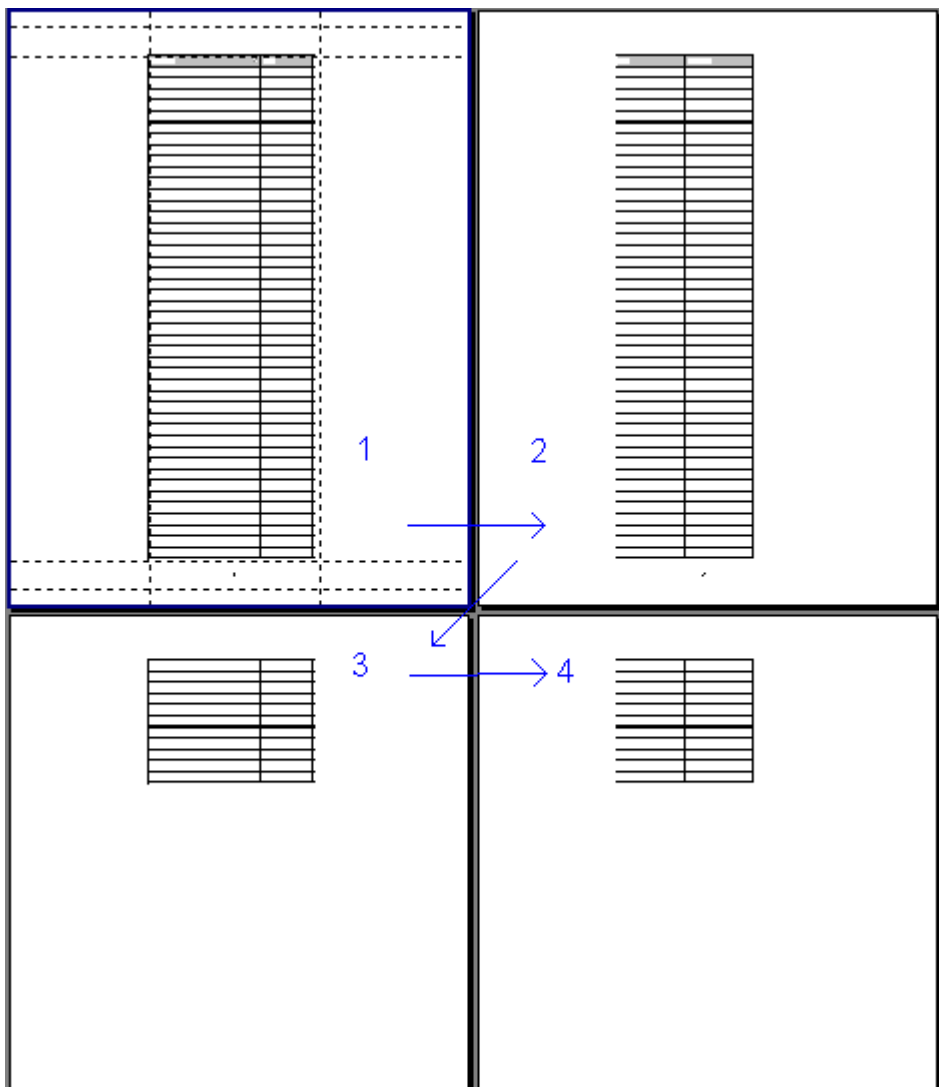
type

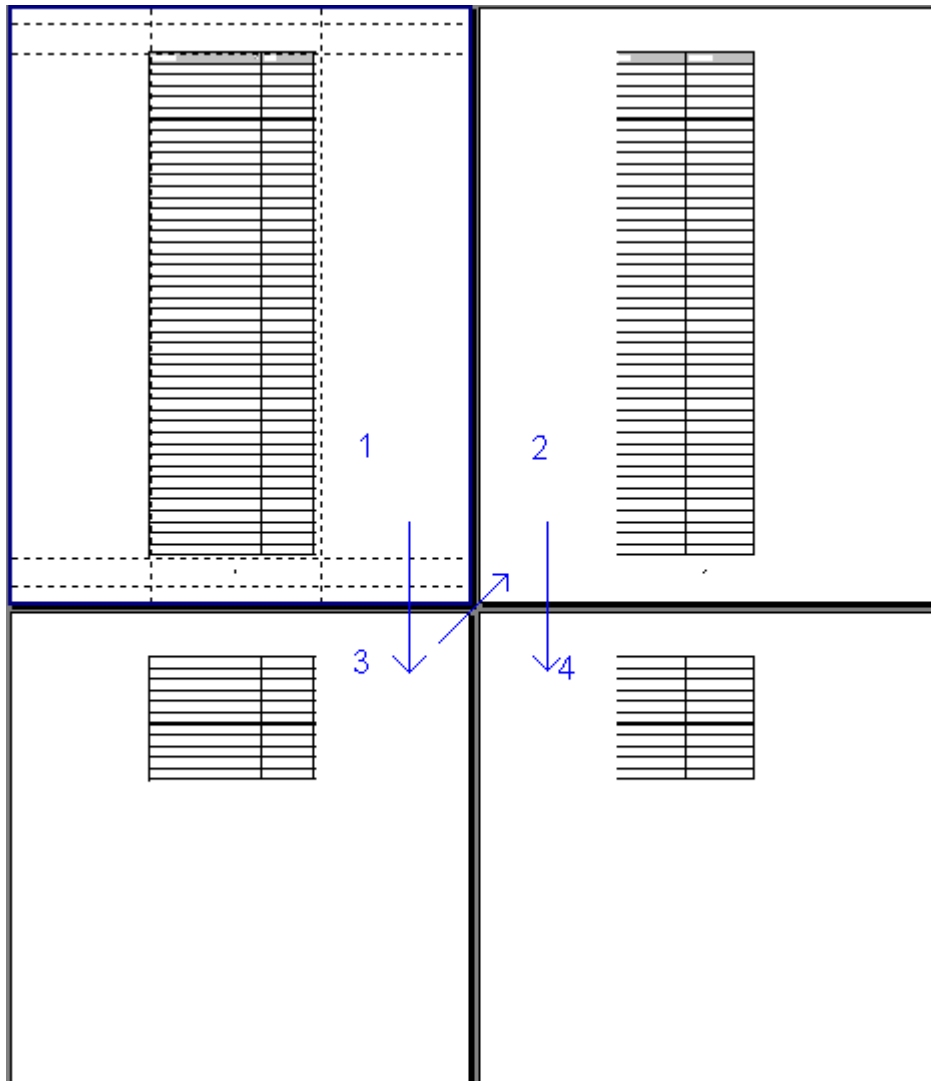
```
TdxPageOrder = (poOverThenDown, poDownThenOver);  
property PageOrder: TdxPageOrder;
```

Description

Use the **PageOrder** property to specify the manner in which report pages are printed. If the active report page does not fit inside the boundaries of one paper page, it is printed across multiple pages and printed in one of the following ways:

Value	Description
poOverThenDown	Report pages are printed one after another, i.e. ? 1, ? 2, ? 3, ? 4.
poDownThenOver	Prints first odd and then even report pages, i.e. ? 1, ? 3, ? 2, ? 4.





The default value for this property is **poOverThenDown**.

PageSize property

read / write

Determines the size of a report page.

property PageSize : TdxPointWrapper;

Description

Use the **PageSize** property to define the size of report pages. The width and height of a page are specified in thousandths of used measurement units. For instance, if inches are your unit of measurement, then an 8500 integer value of the page width property means that the page width is 8.5 inches. These properties can not be directly changed, you can work with them only via the page setup dialog or the DMPaper property.

See Also

TdxPrinterPage.DMPaper; TdxPrinterPage.Margins

RealMeasurementUnits property**read only**

Specifies the type of a measurement unit installed on a system.

type

```
TdxMeasurementUnits = (muDefault, muInches, muMillimeters);
```

```
property RealMeasurementUnits : TdxMeasurementUnits;
```

Description

The **RealMeasurementUnits** property is used to determine which type of measurement unit is used in a system. It returns **muInches** if inch is the base measurement unit; Otherwise it returns **muMillimeters**. This property is similar to the GetInnerMeasurementUnits method.

RealPageSize property**read / write**

Determines a page real size.

```
property RealPageSize: TPoint;
```

Description

Use the **GetRealPageSize** property to determine a page real size.

See Also

*TdxPrinterPage.RealMeasurementUnits; TdxPrinterPage.RealPageSizeLoMetric;
TdxPrinterPage.RealPageSizePixels*

ReverseTitlesOnEvenPages property**read / write**

Specifies whether text order in titles is reversed.

```
property ReverseTitlesOnEvenPages: Boolean;
```

Description

Use the **ReverseTitlesOnEvenPages** property to specify whether text order on even page titles is reversed. For instance, for an odd page, page titles may look like the following:

```
Machine Name, Date Printed, Total Pages,
```

Whereas even pages may look like the following:

```
Total Pages, Date Printed, Machine Name.
```

Set the **ReverseTitlesOnEvenPages** property to **True** to enable this reversing action.

The default value for this property is **False**.

See Also

TCustomdxPageObject.CenterTitle; CustomdxPageObject.LeftTitle; CustomdxPageObject.RightTitle; CustomdxPageObject.Titles

ScaleFactor property**read / write**

Determines the scale factor for a report.

property ScaleFactor: **Integer**;

Description

Use the **ScaleFactor** property to determine the scale factor for a report. If your report exceeds the boundaries but you want it to be printed on a single page, you may set an appropriate percentage value, so that your report fits perfectly inside the boundaries of a report page.

Note: The **ScaleFactor** property is only accessible if the **ScaleMode** property is set to **smAdjust**.

The default value for this property is **100**.

See Also

TdxPrinterPage.ScaleMode

ScaleMode property**read / write**

Specifies the manner in which a report is scaled.

type

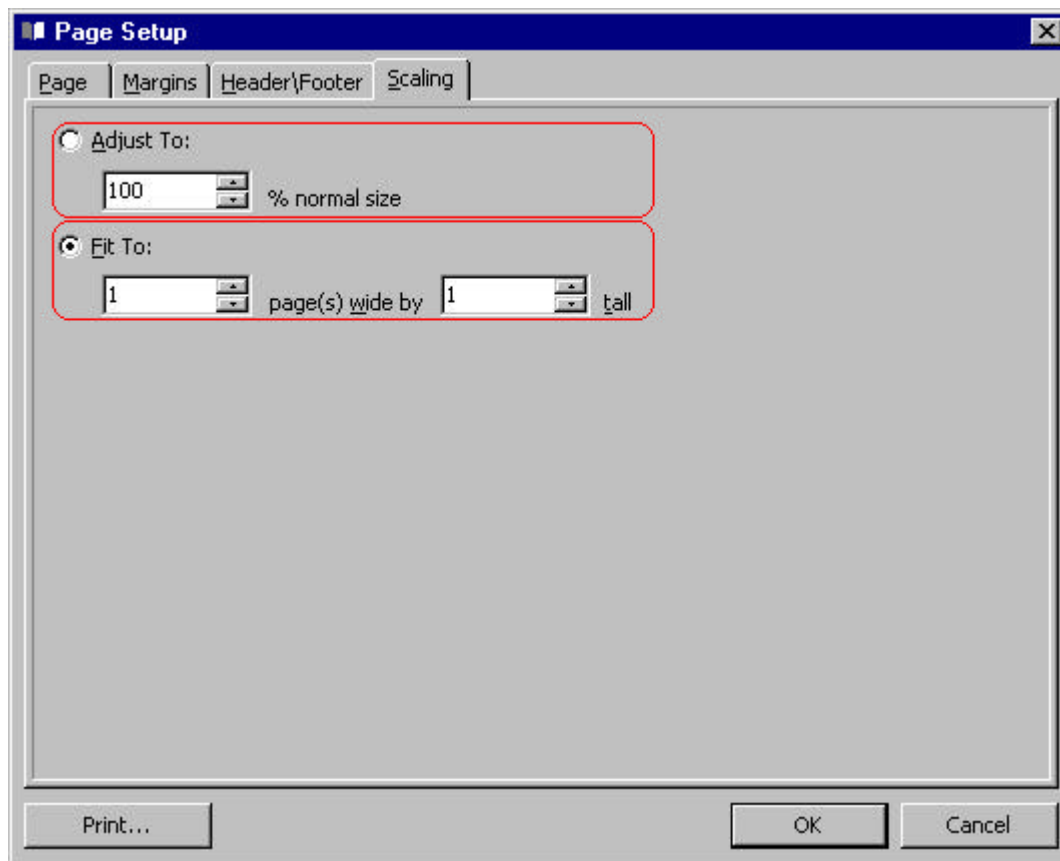
`TdxScaleMode = (smAdjust, smFit);`

property ScaleMode: TdxScaleMode;

Description

Use the **ScaleMode** property to specify the manner in which a report is scaled.

Value	Description
smAdjust	Use this value to set the desired percentage value of a report page. This method of adjusting report pages takes time - you have to manually select the right percentage value to design a really elegant layout.
smFit	Use this value to simply fit a report inside a page either by width or height. You do not need to try different percentages to adjust a report page for printing.



The default value for this property is **smAdjust**.

See Also

TdxPrinterPage.ScaleFactor

TdxPrinterPage Methods

In TdxPrinterPage

ApplyToPrintDevice	public	MinMarginsLoMetric	public
FixMarginsOutside	public	MinPrintableArea	public
FooterLoMetric	public	MinPrintableAreaLoMetric	public
FooterRect	public	PageSizeLoMetric	public
FooterRectLoMetric	public	PageSizePixels	public
GetInnerMeasurementUnits	public	PaintRectLoMetric	public
GetRealMinMargins	public	PaintRectPixels	public
HeaderLoMetric	public	ReadPrinterInfos	protected
HeaderRect	public	RealPageSizeLoMetric	public
HeaderRectLoMetric	public	RealPageSizePixels	public
IsEqual	public	RestoreDefaults	public
MapRect2LoMetric	public	UpdateMeasurementUnits	protected
MarginChange	protected	ValidateMargins	public
MarginsLoMetric	public		

ApplyToPrintDevice method

Applies page settings to a printing device.

```
procedure ApplyToPrintDevice;
```

Description

At runtime, a user can define different page setting such as margins, colors, paper source, etc. Use the **ApplyToPrintDevice** method to send these settings to a printing device and apply them to printed report pages.

FixMarginsOutside method

Adjusts report margins to the minimum page margins available.

```
procedure FixMarginsOutside;
```

Description

Use the **FixMarginsOutside** property to adjust the report page bounds within real layout page bounds, i.e. if the report page dimensions exceed the bounds of the real printer page its dimensions are reduced to fit it.

See Also

TdxPrinterPage.ValidateMargins

FooterLoMetric method

Determines footer dimensions.

```
function FooterLoMetric: Integer;
```

Description

Use the **FooterLoMetric** method to obtain footer dimensions against the current measurement units.

See Also

TdxPrinterPage.HeaderLoMetric

FooterRect method

Determines footer's rectangular dimensions.

```
function FooterRect: TRect;
```

Description

Use the FooterRect method to determine footer's rectangular dimensions.

See Also

TdxPrinterPage.FooterRectLoMetric; TdxPrinterPage.HeaderRect;
TdxPrinterPage.HeaderRectLoMetric

FooterRectLoMetric method

Determines footer dimensions.

```
function FooterRectLoMetric: TRect;
```

Description

Use the **FooterRectLoMetric** method to determine footer's dimensions against current measurement units.

See Also

TdxPrinterPage.FooterRect; TdxPrinterPage.HeaderRectLoMetric

GetInnerMeasurementUnits method

Determines the type of a measurement unit installed in a system.

type

```
TdxMeasurementUnits = (muDefault, muInches, muMillimeters);  
function GetInnerMeasurementUnits: TdxMeasurementUnits;
```

Description

The **GetInnerMeasurementUnits** method is used to determine which type of measurement unit is used in the system. It returns **muInches** if inch is the base measurement unit; Otherwise it returns **muMillimeters**.

See Also

TdxPrinterPage.GetDefaultMeasurementUnits

GetRealMinMargins method

Determines the minimum allowable page margins.

```
procedure GetRealMinMargins(var AMinLeft, AMinRight, AMinTop, AMinBottom:  
Integer);
```

Description

Use the **GetRealMinMargins** method to retrieve the minimum allowable report page margins.

See Also

*TdxPrinterPage.FixMarginsOutside; TdxPrinterPage.MarginChange; TdxPrinterPage.MarginsLoMetric;
TdxPrinterPage.MinMarginsLoMetric; TdxPrinterPage.MinPrintableArea;
TdxPrinterPage.MinPrintableAreaLoMetric; TdxPrinterPage.OnMarginChange;
TdxPrinterPage.ValidateMargins*

HeaderLoMetric method

Determines header dimensions.

```
function HeaderLoMetric: Integer;
```

Description

Use the **HeaderLoMetric** method to determine header dimensions against current measurement units.

See Also

TdxPrinterPage.FooterLoMetric

HeaderRect method

Determines a header's rectangular dimensions.

```
function HeaderRect: TRect;
```

Description

Use the **HeaderRect** method to determine a header's rectangular dimensions.

See Also

*TdxPrinterPage.FooterRect; TdxPrinterPage.FooterRectLoMetric;
TdxPrinterPage.HeaderRectLoMetric*

HeaderRectLoMetric method

Determines header dimensions.

```
function HeaderRectLoMetric: TRect;
```

Description

Use the **HeaderRectLoMetric** method to determine a header's dimensions against current measurement units.

See Also

TdxPrinterPage.FooterRect; TdxPrinterPage.FooterRectLoMetric; TdxPrinterPage.HeaderRect

MarginsLoMetric method

Determines margin dimensions.

```
function MarginsLoMetric: TRect;
```

Description

Use the **MarginsLoMetric** method to determine margin dimensions against current measurement units.

See Also

*TdxPrinterPage.MinMarginsLoMetric; TdxPrinterPage.MinPrintableArea;
TdxPrinterPage.MinPrintableAreaLoMetric*

MinMarginsLoMetric method

Determines minimum margin values.

```
function MinMarginsLoMetric: TRect;
```

Description

Use the **MinMarginsLoMetric** method to determine the minimum margin value against the current measurement units.

See Also

TdxPrinterPage.MarginsLoMetric

MinPrintableArea method

Determines the minimum printable area value.

```
function MinPrintableArea: Integer;
```

Description

Use the **MinPrintableArea** method to determine the minimum printable area value.

See Also

TdxPrinterPage.MinPrintableAreaLoMetric

MinPrintableAreaLoMetric method

Determines the minimum printable area value.

```
function MinPrintableAreaLoMetric: Integer;
```

Description

Use the **MinPrintableAreaLoMetric** method to determine the minimum printable area against current measurement units.

See Also

TdxPrinterPage.MinPrintableArea

PageSizeLoMetric method

Determines available page size.

```
function PageSizeLoMetric: TPoint;
```

Description

Use the **PageSizeLoMetric** method to determine the available page size against current measurement units.

See Also

TdxPrinterPage.PageSizePixels

PageSizePixels method

Returns page size in pixels.

```
function PageSizePixels: TPoint;
```

Description

Use the **PageSizePixels** method to retrieve page size in pixels.

See Also

TdxPrinterPage.PageSizeLoMetric

PaintRectLoMetric method

Determines the drawing area.

```
function PaintRectLoMetric: TRect;
```

Description

Use the **PaintRectLoMetric** method to determine the drawing area against current measurement units.

See Also

TdxPrinterPage.PaintRectPixels

PaintRectPixels method

Determines the scaled region wherein to perform drawing.

```
function PaintRectPixels: TRect;
```

Description

Use the **PaintRectPixels** method to determine the drawing region. This area is scaled based upon the scale factor.

See Also

TdxPrinterPage.PaintRectLoMetric

RealPageSizeLoMetric method

Determines actual page size.

```
function RealPageSizeLoMetric: TPoint;
```

Description

Use the **RealPageSizeLoMetric** method to determine the actual page size against current measurement units. While processing this method, page orientation should be taken into account.

See Also

TdxPrinterPage.RealPageSizePixels

RealPageSizePixels method

Returns the actual page size in pixels.

```
function RealPageSizePixels: TPoint;
```

Description

Use the **RealPageSizePixels** method to retrieve actual page size value in pixels.

See Also

TdxPrinterPage.RealPageSizeLoMetric

ValidateMargins method

Checks report margin size.

```
function ValidateMargins: Boolean;
```

Description

Provides an interface to the method that validates report page margins prior to printing. Use the **ValidateMargins** function to check whether report margins are set outside the printable page area (**True**); Otherwise, **False**.

See Also

*TdxPrinterPage.FixMarginsOutside; TdxPrinterPage.MinPrintableArea;
TdxPrinterPage.MinPrintableAreaLoMetric; TdxPrinterPage.OnMarginChange*

TdxPrinterPage Events

TdxPrinterPage

OnMarginChange **public**

OnChange event

Occurs when changing page contents.

property OnChange: TNotifyEvent;

Description

Write an **OnChange** event handler to perform specific actions when changing page contents.

TdxPointWrapper

Unit

dxWrap

```
TdxPointWrapper = class(TPersistent);
```

Description

The purpose of the **TdxPointWrapper** object is to maintain properties of structured type, such as TPoint, at both design and runtime. The **TdxPointWrapper** object is used to store information on 2-dimensional objects; i.e. it saves width and height of an area.

Hierarchy

Object

TPersistent

TdxPointWrapper Properties

TdxPointWrapper

Point	public / read / write	Y	published / read / write
X	published / read / write		

Point property

read / write

```
property Point : TPoint;
```

Description

Contains coordinates of an object (or width and height for 'area' objects). By default, the **Point** property value is (0, 0).

See Also

TdxPointWrapper.X;TdxPointWrapper.Y

X property

read / write

```
property X : Integer;
```

Description

Determines the horizontal coordinates of an object (or the width for 'area' objects). By default, the **X** property value is equal to 0. This property can not be directly changed, you can work with it only via the page setup dialog.

See Also

TdxPointWrapper.Point;TdxPointWrapper.Y

X property

read / write

```
property Y : Integer;
```

Description

Determines the vertical coordinates of an object (or the height for 'area' objects). By default, the **Y** property value is equal to 0. This property can not be directly changed, you can work with it only via the page setup dialog.

See Also

TdxPointWrapper.Point;TdxPointWrapper.X

TdxPointWrapper Methods

TdxPointWrapper

Clone	public	Empty	public
Create	public	IsEqual	public
DoChanged	protected	PointEqual	public
DoChanging	protected		

Clone method

```
function Clone: TPersistent;
```

Description

Creates a copy of a given **TdxPointWrapper** instance. The Clone method returns the newly created object. The result of this method is the same as if you sequentially call Create and Assign methods for the **TdxPointWrapper** object.

Create method

```
constructor Create(AX, AY : Integer);
```

Description

Creates an instance of type TdxPointWrapper from coordinates, specified by the *AX* and *AY* parameters accordingly.

See Also

TdxPointWrapper.CreateDefault;TdxPointWrapper.CreateFromPoint

DoChanged method

type

```
TdxPointCoord = (pcX, pcY);
```

```
TdxPointCoords = set of TdxPointCoord;
```

```
procedure DoChanged(Coords : TdxPointCoords); dynamic;
```

Description

This method is called when the X or Y property of the TdxPointWrapper object has been changed. It calls the OnChanged event handler (if assigned).

See Also

TdxPointWrapper.DoChanging;TdxPointWrapper.OnChanged

DoChanging method

Generates an **OnChanging** event.

type

```
TdxPointCoord = (pcX, pcY);
TdxPointCoords = set of TdxPointCoord;
procedure DoChanging(Coords : TdxPointCoords; var Values : array of integer); dynamic;
```

Description

This method is called when the X or Y property of the TdxPointWrapper object is to be changed. It simply calls the OnChanging event handler (if assigned).

See Also

TdxPointWrapper.DoChanged;TdxPointWrapper.OnChanging

Empty method

```
procedure Empty;
```

Description

Sets a zero value (0, 0) to the TdxPointWrapper object.

See Also

TdxPointWrapper.X;TdxPointWrapper.Y

IsEqual method

Compares the *APoint* parameter with a given point.

```
function IsEqual(const APoint : TPoint): Boolean;
```

Description

Returns **True**, if coordinates of a specified point are equal to the X and Y properties for a given TdxPointWrapper object.

See Also

TdxPointWrapper.PointEqual;TdxPointWrapper.X;TdxPointWrapper.Y

PointEqual method

Compares two points.

```
class function PointEqual(const P1, P2 : TPoint): Boolean;
```

Description

Returns **True**, if a point, specified by the *P1* parameter, has the same coordinates as a point, specified by the *P2* parameter.

TdxPointWrapper Events

TdxPointWrapper

OnChanged	public
OnChanging	public

OnChanged event

Occurs when the X or Y property of the TdxPointWrapper object has been changed.

type

```
TdxPointCoord = (pcX, pcY);
TdxPointCoords = set of TdxPointCoord;
TdxPointChangeEvent = procedure(Sender : TObject; Coords :
TdxPointCoords) of object;
property OnChanged : TdxPointChangeEvent;
```

Description

Write an **OnChanged** event handler to respond to X and Y properties changes in a TdxPointWrapper object.

See Also

TdxPointWrapper.OnChanging;TdxPointWrapper.X;TdxPointWrapper.Y;TdxPointWrapper.DoChanged

OnChanging event

Occurs when the X or Y property of the TdxPointWrapper object is about to be changed.

type

```
TdxPointCoord = (pcX, pcY);
TdxPointCoords = set of TdxPointCoord;
TdxPointChangingEvent = procedure(Sender : TObject; Coords :
TdxPointCoords; var Values : array of integer) of object;
property OnChanging : TdxPointChangingEvent;
```

Description

Write an **OnChanging** event handler to force an event before changing the X and Y properties of a TdxPointWrapper object.

See Also

TdxPointWrapper.OnChanged;TdxPointWrapper.X;TdxPointWrapper.Y;TdxPointWrapper.DoChanged

TdxRectWrapper

Unit

dxWrap

```
TdxRectWrapper = class(TPersistent);
```

Description

The **TdxRectWrapper** object is used to maintain objects of structured type, such as TRect, at both design and run-time.

Hierarchy

TObject

TPersistent

TdxRectWrapper Properties

TdxRectWrapper

Bottom	published / read / write	Side	public / read / write
BottomRight	public / read / write	Top	published / read / write
Height	public / read / write	TopLeft	public / read / write
Left	published / read / write	Width	public / read / write
Rect	public / read / write		
Right	published / read / write		

Bottom property

read / write

```
property Bottom : Integer;
```

Description

Specifies bottom coordinates of a rectangle.

See Also

TdxRectWrapper.BottomRight;TdxRectWrapper.Height;TdxRectWrapper.Left;TdxRectWrapper.Rect;TdxRectWrapper.Right;TdxRectWrapper.Top;TdxRectWrapper.TopLeft;TdxRectWrapper.Width

BottomRight property

read / write

```
property BottomRight : TPoint;
```

Description

Determines coordinates of the bottom-right corner of a rectangle.

See Also

TdxRectWrapper.Bottom;TdxRectWrapper.Height;TdxRectWrapper.Left;TdxRectWrapper.Rect;TdxRectWrapper.Right;TdxRectWrapper.Top;TdxRectWrapper.TopLeft;TdxRectWrapper.Width

Height property

read / write

```
property Height : Integer;
```

Description

Determines the height of a rectangle specified by a given TdxRectWrapper object.

See Also

TdxRectWrapper.Bottom;TdxRectWrapper.BottomRight;TdxRectWrapper.Left;TdxRectWrapper.Rect;TdxRectWrapper.Right;TdxRectWrapper.Top;TdxRectWrapper.TopLeft;TdxRectWrapper.Width

Left property**read / write**

property Left : Integer;

Description

Determines the left coordinates of a rectangle.

See Also

*TdxRectWrapper.Bottom;TdxRectWrapper.BottomRight;TdxRectWrapper.Height;
TdxRectWrapper.Rect;TdxRectWrapper.Right;TdxRectWrapper.Top;TdxRectWrapper.TopLeft;
TdxRectWrapper.Width*

Rect property**read / write**

property Rect : TRect;

Description

Represents a rectangle maintained by the TdxRectWrapper object.

See Also

*TdxRectWrapper.Bottom;TdxRectWrapper.BottomRight;TdxRectWrapper.Height;
TdxRectWrapper.Left;TdxRectWrapper.Right;TdxRectWrapper.Top;TdxRectWrapper.TopLeft;
TdxRectWrapper.Width*

Right property**read / write**

property Right : Integer;

Description

Determines the right coordinates of a rectangle.

See Also

*TdxRectWrapper.Bottom;TdxRectWrapper.BottomRight;TdxRectWrapper.Height;
TdxRectWrapper.Left;TdxRectWrapper.Rect;TdxRectWrapper.Top;TdxRectWrapper.TopLeft;
TdxRectWrapper.Width*

Side property**read / write**

Represents an array of rectangle coordinates.

type

TdxRectSide = (rsLeft, rsTop, rsRight, rsBottom);

TdxRectSides = **set of** TdxRectSide;

property Side[ASide:TdxRectSide] : Integer; **default;**

Description

Use this property to set or access TdxRectWrapper object coordinates. For instance, the following code changes coordinates of the TdxRectWrapper object:

```
dxRectWrapper1.Side[rsLeft] := 10;  
dxRectWrapper1.Side[rsTop] := 10;  
dxRectWrapper1.Side[rsRight] := dxRectWrapper1.Side[rsLeft] +  
dxRectWrapper1.Width;  
dxRectWrapper1.Side[rsBottom] := dxRectWrapper1.Side[rsTop] +  
dxRectWrapper1.Height;
```

See Also

*TdxRectWrapper.Bottom; TdxRectWrapper.BottomRight; TdxRectWrapper.Height;
TdxRectWrapper.Left; TdxRectWrapper.Rect; TdxRectWrapper.Right; TdxRectWrapper.Top;
TdxRectWrapper.TopLeft; TdxRectWrapper.Width*

Top property

[read / write](#)

```
property Top : Integer;
```

Description

Specifies the top coordinates of a rectangle.

See Also

*TdxRectWrapper.Bottom; TdxRectWrapper.BottomRight; TdxRectWrapper.Height;
TdxRectWrapper.Left; TdxRectWrapper.Rect; TdxRectWrapper.Right; TdxRectWrapper.TopLeft;
TdxRectWrapper.Width*

TopLeft property

[read / write](#)

```
property TopLeft : TPoint;
```

Description

Specifies coordinates of the top-left corner of the TdxRectWrapper object.

See Also

*TdxRectWrapper.Bottom; TdxRectWrapper.BottomRight; TdxRectWrapper.Height;
TdxRectWrapper.Left; TdxRectWrapper.Rect; TdxRectWrapper.Right; TdxRectWrapper.Top;
TdxRectWrapper.Width*

Width property

[read / write](#)

```
property Width : Integer;
```

Description

Determines the width of a rectangle specified by a given TdxRectWrapper object.

See Also

*TdxRectWrapper.Bottom;TdxRectWrapper.BottomRight;TdxRectWrapper.Height;
TdxRectWrapper.Left;TdxRectWrapper.Rect;TdxRectWrapper.Right;TdxRectWrapper.Top;
TdxRectWrapper.TopLeft*

TdxRectWrapper Methods

TdxRectWrapper

Clone	public	Empty	public
Create	public	IsEqual	public
DoChanged	protected		
DoChanging	protected		

Clone method

```
function Clone: TPersistent;
```

Description

This method creates a copy of a given TdxRectWrapper object. It returns the newly created object. The **Clone** method produces the same result as if you sequentially call the **Create** and **Assign** methods.

Create method

```
constructor Create(ALeft, ATop, ARight, ABottom : Integer);
```

Description

Creates a new TdxRectWrapper object with specified coordinates.

See Also

*TdxRectWrapper.CreateDefault; TdxRectWrapper.CreateFromBounds;
TdxRectWrapper.CreateFromPoints; TdxRectWrapper.CreateFromRect*

DoChanged method

Generates an OnChanged event.

type

```
TdxRectSide = (rsLeft, rsTop, rsRight, rsBottom);
```

```
TdxRectSides = set of TdxRectSide;
```

```
procedure DoChanged(Sides : TdxRectSides); dynamic;
```

Description

This method is called when coordinates of TdxRectWrapper have been changed. It simply calls the OnChanged event handler (if assigned). The *Sides* parameter determines assigned coordinates.

See Also

TdxRectWrapper.DoChanging; TdxRectWrapper.OnChanged

DoChanging method

Generates an OnChanging event.

type

```
TdxRectSide = (rsLeft, rsTop, rsRight, rsBottom);
TdxRectSides = set of TdxRectSide;
```

```
procedure DoChanging(Sides : TdxRectSides; var Values : array of integer );
dynamic;
```

Description

This method is called when coordinates of a TdxRectWrapper object are about to be changed. It simply calls the OnChanging event handler (if assigned). The *Sides* parameter specifies the changed coordinates. The *Values* parameter determines new values for changed coordinates.

See Also

TdxRectWrapper.DoChanged; TdxRectWrapper.OnChanging

Empty method

```
procedure Empty;
```

Description

Sets the default coordinate values of a TdxRectWrapper object. After calling this method the Left, Top, Bottom, Right, Width, and Height properties of the TdxRectWrapper object are equal to zero.

See Also

TdxPointWrapper.X; TdxPointWrapper.Y

IsEqual method

Compares a specified rectangle with a rectangle maintained by the TdxRectWrapper object.

```
function IsEqual(const ARect : TRect): Boolean;
```

Description

Returns **True**, if a rectangle, specified by the *ARect* parameter, has the same coordinates as the TdxRectWrapper object.

See Also

TdxPointWrapper.PointEqual; TdxPointWrapper.X; TdxPointWrapper.Y

TdxRectWrapper Events

TdxRectWrapper

OnChanged	public
OnChanging	public

OnChanged event

Occurs when coordinates of a rectangle, maintained by the TdxRectWrapper object, have been changed.

type

```
TdxRectSide = (rsLeft, rsTop, rsRight, rsBottom);
TdxRectSides = set of TdxRectSide;
TdxRectChangeEvent = procedure(Sender : TObject; Sides : TdxRectSides) of
object;
property OnChanged : TdxRectChangeEvent;
```

Description

Write an **OnChanged** event handler to respond to changes to TdxRectWrapper object coordinates. The *Sides* parameter determines the changed coordinates.

See Also

TdxRectWrapper.DoChanging;TdxRectWrapper.OnChanging

OnChanging event

Occurs when coordinates of a rectangle maintained by the TdxRectWrapper object are about to be changed.

type

```
TdxRectSide = (rsLeft, rsTop, rsRight, rsBottom);
TdxRectSides = set of TdxRectSide;
TdxRectChangingEvent = procedure(Sender : TObject; Sides : TdxRectSides;
var Values : array of integer) of object;
property OnChanging : TdxRectChangingEvent;
```

Description

Write an **OnChanging** event handler to change coordinates as your needs dictate. For instance, the following code allows users to avoid changing the left coordinate:

```
if [rsLeft] in Sides then
    Values[rsLeft] := dxRectWrapper1.Left;
```

See Also

TdxRectWrapper.DoChanging;TdxRectWrapper.OnChanged

TCustomdxComponentPrinter Object

TCustomdxComponentPrinter represents the base component of the ExpressPrinting System – Standard Edition.

Unit

dxPSCore

```
TCustomdxComponentPrinter = class(TComponent);
```

Description

TCustomdxComponentPrinter is the ancestor of TdxComponentPrinter and provides its core functionality. Do not use the **TCustomdxComponentPrinter** component to print VCL components in your application. Instead, use the TdxComponentPrinter component. To print a specific component you must be certain that you have a report link for that component class.

Hierarchy

```
    TObject
      TPersistent
        TComponent
```

TCustomdxComponentPrinter Properties

TCustomdxComponentPrinter

AbortPrinting	public / read / write	PreviewOptions	public / read / write
AutoUpdateDateTime	public / read / write	PreviewWindow	public / read only
BeepAfterLongOperations	public / read / write	PreviewWindowDesigner	public / read only
CurrentLink	public / read / write	PrintFileList	public / read only
CurrentLinkIndex	public / read / write	PrintTitle	public / read / write
DateFormat	public / read / write	ReportLink	public / read / write
LinkCount	public / read only	ReportLinkDesigner	public / read only
LongOperationTime	public / read only	State	public / read only
Options	public / read / write	TimeFormat	public / read only
PageNumberFormat	public / read only		

AbortPrinting property

read / write

Indicates whether to stop report printing.

```
property AbortPrinting : Boolean;
```

Description

When a report is being printed, every new page is preceded by the **OnNewPage** event. Set the **AbortPrinting** property to **True** within this event to abort printing.

AutoUpdateDateTime property

read / write

Specifies whether date and time values in a report are updated automatically.

```
property AutoUpdateDateTime: Boolean;
```

Description

Use the **AutoUpdateDateTime** property to update date and time values within a report when needed. If this property is set to **True**, date and times are updated every time the report is generated. If this property is set to **False**, report date and time are set when a specific component is assigned to the appropriate report link. You can also change date and time via the **DateTime** property of the report link.

The default value for this property is **True**.

BeepAfterLongOperations property

read / write

Specifies whether a beep is generated for a long operation.

```
property BeepAfterLongOperations: Boolean;
```

Description

Set the **BeepAfterLongOperations** property to **True** to generate a beep when an operation consumes more time than specified by the **LongOperationTime** property value.

The default value for this property is **True**.

See Also

TCustomdxComponentPrinter.LongOperationTime

CurrentLink property

read / write

Determines an active link.

```
property CurrentLink : TBasedxReportLink ;
```

Description

Use the **CurrentLink** property to set a link.

CurrentLinkIndex property

read / write

Specifies the index of a current report link.

```
property CurrentLinkIndex: Integer;
```

Description

Use the **CurrentLinkIndex** property to obtain the index of the active link within the appropriate component printer.

DateFormat property

read / write

Specifies date presentation format.

```
property DateFormat: Integer;
```

Description

A component printer has a set of predefined date formats that can be applied to a report. The report date format depends on the format specified in the System Regional Settings. The **DateFormat** property specifies the zero-based index of the appropriate format.

Index	Example
0	2/14/01
1	Wednesday, February 14, 2001
2	February 14, 2001
3	2/14/2001
4	2001-02-14
5	14-Feb-01
6	2.14.01
7	Wednesday, Feb. 14, 01
8	2.14.2001

9 February , 01
10 Feb-01

The default value for this property is **0**.

LinkCount property

read only

Specifies the number of links.

```
property LinkCount : Integer;
```

Description

Use the **LinkCount** property to obtain the total number of all accessible links. Users can employ this property to access different links by their ordinal number.

See Also

TCustomdxComponentPrinter.ReportLink

LongOperationTime property

read / write

Specifies the time interval after which an operation is considered to be excessive.

```
property LongOperationTime: Integer;
```

Description

Use the **LongOperationTime** property to set a time interval in milliseconds. This time interval is used to determine whether an operation is excessive. For instance, if a given operation exceeds a certain amount of time, a beep can be generated.

The default value for this property is **5000**.

See Also

TCustomdxComponentPrinter.BeepAfterLongOperations

Options property

read / write

Defines the behavior of a printing component.

type

```
TdxCP0ption = (cpoAutoRebuildBeforePreview, cpoAutoRebuildBeforePrint,  
cpoAutoRebuildBeforeSave, cpoGenerateReportProgressEvent,  
cpoShowHourGlass);
```

```
TdxCP0ptions = set of TdxCP0ption;
```

```
property Options : TdxCP0ptions;
```

Description

Use the **Options** property to force the printing component to perform specific actions during report generation. Available values include:

Values	Meaning
cpoAutoRebuildBeforePreview	Activate this option to rebuild a report before displaying the PrintPreview window.
cpoAutoRebuildBeforePrint	If active, the printing component rebuilds a report before printing.
cpoAutoRebuildBeforeSave	If active, a report is rebuilt before saving it to file.
cpoGenerateReportProgressEvent	If active, the printing component generates the OnGenerateReportProgress event while building a report. If this option is inactive, this event is not generated.
cpoShowHourGlass	If active, an HourGlass cursor appears during continuous operations, such as report generation.

By default, all options are active.

PageNumberFormat property

read / write

Specifies page number formatting in a report.

type

```
TdxPageNumberFormat = (pnfNumeral, pnfChars, pnfUpperChars, pnfRoman,
pnfUpperRoman);
```

```
property PageNumberFormat: TdxPageNumberFormat;
```

Description

Use the **PageNumberFormat** property to specify the appropriate page number formatting within a report. Possible values include.

Value	Example
pnfNumeral	1, 2, 3, 4, 5, ...
pnfChars	a, b, c, d, e, ...
pnfUpperChars	A, B, C, D, E, ...
pnfRoman	I, II, III, IV, V, ...
pnfUpperRoman	I, II, III, IV, V, ...

The default value for this property is **pnfNumeral**.

PreviewOptions property

read / write

Specifies the options of a preview form.

```
property PreviewOptions : TdxPreviewOptions ;
```

Description

The preview form contains a toolbar with different buttons and controls and a preview window. Use this property to specify the look and behavior of the preview form at runtime. This property enables a user to customize buttons, preview form size, caption, etc.

PreviewWindow property**read only**

Returns the preview window.

```
property PreviewWindow : TBasedxPreviewWindow;
```

Description

Use the **PreviewWindow** property to work directly with the preview window. For instance, the following code changes the preview window size (dxComponentPrinter1BeforePreview is the OnBeforePreview event handler of the TdxComponentPrinter object).

```
procedure TForm1.dxComponentPrinter1BeforePreview(Sender: TObject);  
begin  
    dxComponentPrinter1.PreviewWindow.Left := 10;  
    dxComponentPrinter1.PreviewWindow.Width := 250;  
    dxComponentPrinter1.PreviewWindow.Height := 250;  
    dxComponentPrinter1.PreviewWindow.Top := 50;  
end;
```

PreviewWindowDesigner property**read only**

Responsible for the lifetime of the preview window designer.

```
property PreviewWindowDesigner : TAbstractdxPreviewWindowDesigner;
```

Description

The **PreviewWindowDesigner** property is available only at design-time and controls the lifetime of the preview window. At runtime, this property returns **nil**.

PrintFileList property**read only**

Specifies the list of files to which a report has been printed.

```
property PrintFileList: TStrings;
```

Description

Use the PrintFileList property to print a report out to a file. The **PrintFileList** property determines the list of files to which a report has already been printed. This list is displayed in the **'Print'** dialog window within the combo box of the "print to file" option. When a user specifies his/her own file, this file is added to this list. Later a user may print any of these files out.

PrintTitle property**read / write**

Specifies the name of a print job.

```
property PrintTitle : string;
```

Description

Each time the printing component starts printing, it starts a new print job. Use this property to specify a print job's name, which identifies the print job in the print queue.

You can use the OnGetPrintTitle event as an alternate method to specify print job names. The name specified by the OnGetPrintTitle event handler overrides the name specified by the **PrintTitle** property.

See Also

TCustomdxComponentPrinter.GetPrintTitle;TCustomdxComponentPrinter.OnGetPrintTitle

ReportLink property**read / write**

Determines the report link used to print components.

```
property ReportLink[i:Integer] : TBasedxReportLink ;
```

Description

Use the **ReportLink** property to select the desired report link with appropriate report properties. Different report links are used to print different components. For instance, the TdxStringGridReportLink class is used for printing the **TStringGrid** component, and the TdxTreeListReportLink class is used for TdxTreeList (ExpressQuantumTreeList). Use the LinkCount property to access the total number of links.

See Also

TCustomdxComponentPrinter.LinkCount

ReportLinkDesigner property**read only**

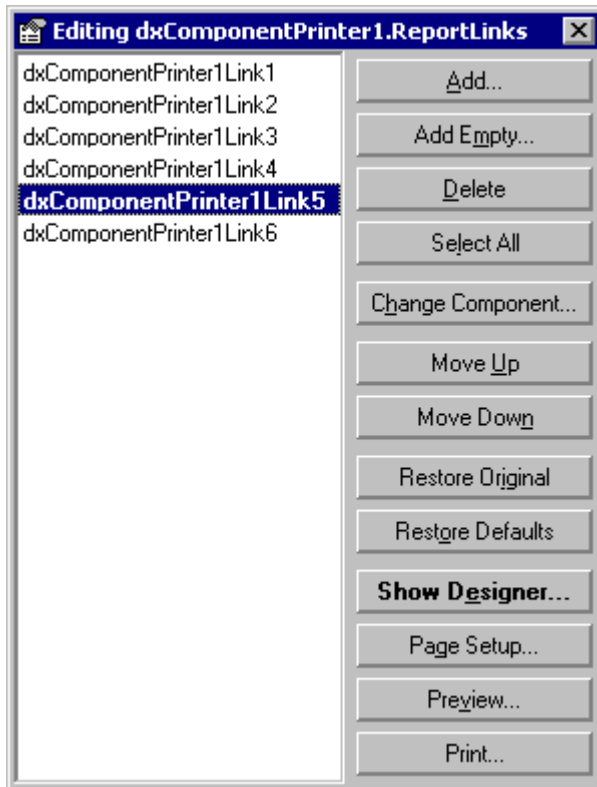
Responsible for the lifetime of the report links designer.

```
property ReportLinkDesigner : TAbstractdxReportLinkDesigner;
```

Description

The **ReportLinkDesigner** property is available only at design-time and controls the lifetime of the report link designer window.

The Report link designer window displays a list of all report links.



State property

read only

Describes the current state of the TCustomdxComponentPrinter object.

type

```
TdxCPState = (cpsSetComponent, cpsBuilding, cpsDesigning, cpsPreviewing, cpsPrinting);
```

```
TdxCPStates = set of TdxCPState;
```

```
property State : TdxCPStates;
```

Description

Use the State property to detect the current state of dxComponentPrinter. Available values include:

Values	Meaning
cpsBuilding	Printing component builds a report.
cpsDesigning	Design mode, the most frequent state to print a component at design- time.
cpsPreviewing	Preview form is active.
cpsPrinting	Report is being printed.

TimeFormat property

read only

Specifies time formatting options.

```
property TimeFormat: Integer;
```

Description

A component printer has a set of predefined time formats that can be applied to a report. The **TimeFormat** property specifies the zero-based index of a given format.

Index	Example
0	2:45 PM
1	2:45:58 PM
2	14:45
3	14:45:58

The default value for this property is **0**.

TCustomdxComponentPrinter Methods

TCustomdxComponentPrinter

AddEmptyLink	public	DrawPageFooter	public
AddEmptyLinkEx	public	DrawPageHeader	public
AddLink	public	FindLinkByComponent	public
AddLinkEx	public	GetNewLinkName	public
AssignReportLinks	public	GetPageColRowCount	public
Create	public	GetPageCount	public
DeleteAllLinks	public	GetPrintTitle	protected
DeleteLink	public	GetRealRegistryPath	public
DesignerExists	public	IndexOfLink	public
DesignReport	public	IndexOfLinkByName	public
Destroy	public	IsGenerateReportProgressEvent	protected
DestroyReport	public	IsRebuildBeforePreview	protected
DoAddReportLink	protected	IsRebuildBeforePrint	protected
DoAfterPreview	protected	IsShowHourGlass	protected
DoBeforePreview	protected	LinkByName	public
DoChangeComponent	protected	PageSetup	public
DoCustomDrawPage	protected	PageSetupEx	public
DoCustomDrawReportTitle	protected	PaintPage	public
DoDeleteReportLink	protected	PrepareReport	protected
DoDesignReport	protected	Preview	public
DoEndPrint	protected	PreviewExists	public
DoEndUpdateReport	protected	Print	public
DoNewPage	protected	PrintEx	public
DoPageSetUp	protected	PrintPage	protected
DoPrintDeviceBusy	protected	PrintPages	public
DoPrintDeviceError	protected	PrintPagesEx	public
DoProgress	protected	RebuildReport	public
DoStartPrint	protected	StdProcessBusy	protected
DoStartUpdateReport	protected	UnPrepareReport	protected

AddEmptyLink method

Adds a new empty link to the list of links.

type

```
TdxReportLinkClass = class of TBasedxReportLink;
```

```
function AddEmptyLink(ALinkClass : TdxReportLinkClass) : TBasedxReportLink;
```

Description

Call the **AddEmptyLink** method to create a new empty link of the class specified by the *ALinkClass* parameter. Unlike the **AddLink** method this method does not connect the created link with any component. This method returns a new instance of the **TBasedxReportLink** object.

See Also

*TCustomdxComponentPrinter.AddEmptyLinkEx;TCustomdxComponentPrinter.AddLink;
TCustomdxComponentPrinter.AddLinkEx*

AddEmptyLinkEx method

Adds a new link to the list of links.

type

```
TdxReportLinkClass = class of TBasedxReportLink;
function AddEmptyLinkEx(ALinkClass : TdxReportLinkClass; AOwner :
TComponent) : TBasedxReportLink;
```

Description

Call the **AddEmptyLink** method to create a new empty link of the class specified by the *ALinkClass* parameter and define the object responsible for destroying the link by the *AOwner* parameter. Unlike the AddLinkEx method this method does not connect the created link with any component. This method returns a new instance of the TBasedxReportLink object.

See Also

*TCustomdxComponentPrinter.AddEmptyLink;TCustomdxComponentPrinter.AddLink;
TCustomdxComponentPrinter.AddLinkEx*

AddLink method

Adds a new link to the list of links.

```
function AddLink(AComponent : TComponent): TBasedxReportLink;
```

Description

Call the **AddLink** method to create a new link to the component specified by the *AComponent* parameter. This method returns a new instance of the TBasedxReportLink object.

See Also

*TCustomdxComponentPrinter.AddEmptyLink;TCustomdxComponentPrinter.AddEmptyLinkEx;
TCustomdxComponentPrinter.AddLinkEx*

AddLinkEx method

Adds a new link to the list of links.

```
function AddLinkEx(AComponent : TComponent; AOwner : TComponent):
TBasedxReportLink;
```

Description

Call the **AddLinkEx** method to create a new link to the component specified by the *AComponent* parameter. This method returns a new instance of the TBasedxReportLink object. Unlike the AddLink method, it defines the object responsible for destroying the link by the *AOwner* parameter.

See Also

*TCustomdxComponentPrinter.AddEmptyLink;TCustomdxComponentPrinter.AddEmptyLinkEx;
TCustomdxComponentPrinter.AddLink*

AssignReportLinks method

Copies report links.

```
procedure AssignReportLinks(Source : TCustomdxComponentPrinter);
```

Description

The **AssignReportLinks** method deletes all report links and then copies all report links contained in the TCustomdxComponentPrinter object specified by the *Source* parameter.

Create method

Creates an instance of type TCustomdxComponentPrinter.

```
constructor Create(AOwner : TComponent); override;
```

Description

This method creates a TCustomdxComponentPrinter object and initializes its properties. Descendants can override this method to allocate any additional memory needed.

Note: Make certain to release memory when destroying the printing component.

See Also

TCustomdxComponentPrinter.Destroy

DeleteAllLinks method

Deletes all links.

```
procedure DeleteAllLinks;
```

Description

Call the **DeleteAllLinks** method when you wish to clear the list of links.

DeleteLink method

Deletes a specific report link.

```
procedure DeleteLink(AIndex: Integer);
```

Description

Call the **DeleteLink** method to remove the report link specified via the *AIndex* parameter from the list of available report links.

DesignerExists method

Determines whether a specific component class has a designer.

```
function DesignerExists(AReportLink: TBasedxReportLink {$IFDEF DELPHI4} =  
nil{$ENDIF}): Boolean;
```

Description

The **DesignerExists** method defines whether there is a designer for a specific link defined by the *AReportLink* parameter. **DesignerExist** returns **True** if such a designer exists.

DesignReport method

Displays the report link designer.

```
function DesignReport(AReportLink : TBasedxReportLink {$IFDEF DELPHI4} =  
nil{$ENDIF}): Boolean;
```

Description

Generally, the designer is used to customize the page layout of a report. The **DesignReport** method activates the report link designer for the link specified by the *AReportLink* parameter. The report designer is displayed on screen if the Component property has been set correctly and the report link includes a designer for a given component class. If the report designer window has been successfully activated, the **DesignReport** function returns the result of Designer task; Otherwise, **False**.

Destroy method

Destroys an instance of TCustomdxComponentPrinter.

```
destructor Destroy; override;
```

Description

This method destroys an object of type TCustomdxComponentPrinter. Descendants can override this method to release any additional memory allocated when creating a TCustomdxComponentPrinter object.

See Also

TCustomdxComponentPrinter.Create

DestroyReport method

Destroys the current report.

```
procedure DestroyReport;
```

Description

Call this method to destroy the report of a link specified by the *CurrentLink* property. Similar to the *DestroyReport* method of the *TBasedxReportLink* object but safer to use because it implements a number of convenient features, such as an HourGlass cursor when destroying takes too much time.

DoAddReportLink method

Generates an *OnAddReportLink* event.

```
procedure DoAddReportLink(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called when a new link, specified by the *AReportLink* parameter, is added to a link list. Descendants can override it to perform specific actions before/after calling the *OnAddReportLink* event handler.

See Also

TCustomdxComponentPrinter.OnAddReportLink

DoAfterPreview method

Generates an *OnAfterPreview* event.

```
procedure DoAfterPreview(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called when closing the preview window of the link, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the *OnAfterPreview* event handler.

See Also

TCustomdxComponentPrinter.OnAfterPreview

DoBeforePreview method

Generates an *OnBeforePreview* event.

```
procedure DoBeforePreview(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called before activating the preview window of the link, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the *OnBeforePreview* event handler.

See Also

TCustomdxComponentPrinter.OnBeforePreview

DoChangeComponent method

Generates an OnChangeComponent event.

```
procedure DoChangeComponent(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called when changing the Component property of the TBasedxReportLink object, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the OnChangeComponent event handler.

See Also

TCustomdxComponentPrinter.OnChangeComponent

DoCustomDrawPage method

Generates an OnCustomDrawPage event.

```
procedure DoCustomDrawPage(AReportLink : TBasedxReportLink; ACanvas :  
TCanvas; APageIndex : Integer; ARect : TRect; ANom, ADenom : Integer);  
virtual;
```

Description

This method is called when a specific report page, defined by the *APageIndex* parameter, is drawn. The *AReportLink* parameter specifies the TBasedxReportLink object. The *ACanvas* parameter specifies the canvas for drawing. *ARect* specifies the bound rectangle for drawing.

When using custom draw methods, you must manage image repainting during zoom events. It is always necessary when previewing a report and necessary only for images when printing the report. The *ANom* and *ADenom* parameters determine the coefficient of magnification and demagnification accordingly. Descendants can override them to perform specific actions before/after calling the OnCustomDrawPage event handler.

See Also

TCustomdxComponentPrinter.OnCustomDrawPage

DoCustomDrawReportTitle method

Generates an OnCustomDrawReportTitle event.

type

```
TdxTextAlignX = ( taLeft, taCenterX, taRight );
```

```
TdxTextAlignY = ( taTop, taCenterY, taBottom );
```

```
procedure DoCustomDrawReportTitle(AReportLink : TBasedxReportLink; ACanvas  
: TCanvas; ARect : TRect; ANom, ADenom : Integer; var TextAlignX :  
TdxTextAlignX; var TextAlignY : TdxTextAlignY; var AColor : TColor; AFont :  
TFont; var ADone : Boolean); virtual;
```

Description

The **DoCustomDrawReportTitle** method is called when a printing component builds the image of a report's title. The *AReportLink* parameter specifies the report link. The *ACanvas* parameter determines the drawing surface. The *ARect* parameter specifies the bound rectangle for drawing.

When using custom draw methods, you must manage image repainting during zoom events. It is always necessary when previewing the report and necessary only for images when printing the report. The *ANom* and *ADenom* parameters determine the coefficient of magnification and demagnification accordingly. The *TextAlignX* and *TextAlignY* parameters specify horizontal and vertical alignment. The *AColor* parameter sets the font color for the report's title. The *AFont* parameter sets the font for the report's title. The *ADone* parameter determines whether the **DoCustomDrawReportTitle** method processes the entire operation. Descendants can override this method to perform specific actions right after/before the *OnCustomDrawReportTitle* event.

Available values for the *TextAlignX* parameter include:

Values	Meaning
taLeft	Text is left justified. All text lines are at the left edge of an item.
taCenterX	Text is centered within a cell.
taRight	Text is right justified. All text lines are aligned at the right edge of an item.

By default, the **TextAlignX** property value is **taCenterX**.

Available values the *TextAlignY* parameter include:

Values	Meaning
taTop	Text is aligned at the top edge of an item.
taCenterY	Text is vertically centered within a cell.
taBottom	Text is aligned at the bottom of an item's cell.

By default, the **TextAlignY** property value is equal to **taCenterY**.

See Also

TCustomdxComponentPrinter.OnCustomDrawReportTitle

DoDeleteReportLink method

Generates an *OnDeleteReportLink* event.

```
procedure DoDeleteReportLink(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called when a report link is about to be deleted from the link list. The *AReportLink* parameter specifies the link to be deleted. Descendants can override it to perform specific actions before/after calling the *OnDeleteReportLink* event handler.

See Also

TCustomdxComponentPrinter.OnDeleteReportLink

DoDesignReport method

Generates an OnDesignReport event.

```
procedure DoDesignReport(AReportLink : TBasedxReportLink; ADone : Boolean
); dynamic;
```

Description

This method is called when closing the report link designer of a link, specified by the *AReportLink* parameter. If a user closes the designer window by pressing the 'OK' button, the *ADone* parameter is set to **True** and the report is automatically rebuilt. Descendants can override it to perform specific actions before/after calling the OnDesignReport event handler.

See Also

TCustomdxComponentPrinter.OnDesignReport

DoEndPrint method

Generates an OnEndPrint event.

```
procedure DoEndPrint(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called when the printing component finishes printing the component, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the OnEndPrint event handler.

See Also

TCustomdxComponentPrinter.OnEndPrint

DoEndUpdateReport method

Generates an OnEndGenerateReport event.

```
procedure DoEndUpdateReport(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called when the printing component finishes updating the report generated by the link, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the OnEndGenerateReport event handler.

See Also

TCustomdxComponentPrinter.OnEndGenerateReport

DoNewPage method

Generates an OnNewPage event.

```
procedure DoNewPage(AReportLink : TBasedxReportLink; APageIndex : Integer); dynamic;
```

Description

This method is called when creating a new page to print within a report, stored in the link, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the OnNewPage event handler. The *APageIndex* parameter specifies a new page number.

See Also

TCustomdxComponentPrinter.OnNewPage

DoPageSetUp method

Generates an OnPageSetUp event.

```
procedure DoPageSetUp(AReportLink : TBasedxReportLink; ADone : Boolean); dynamic;
```

Description

This method is called when setting up the page layout in a report stored in the link, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the OnPageSetUp event handler. If a user closes the PageSetup form by pressing the 'OK' button, *ADone* is equal to True.

See Also

TCustomdxComponentPrinter.OnPageSetUp

DoPrintDeviceBusy method

Generates an OnPrintDeviceBusy event.

```
procedure DoPrintDeviceBusy; dynamic;
```

Description

The **DoPrintDeviceBusy** method is called before printing to check whether the printing device is free. If it is busy, the StdProcessBusy method is called. Descendants can override it to perform specific actions before/after calling the OnPrintDeviceBusy event handler.

See Also

TCustomdxComponentPrinter.OnPrintDeviceBusy

DoPrintDeviceError method

Generates an OnPrintDeviceError event.

```
procedure DoPrintDeviceError; dynamic;
```

Description

The **DoPrintDeviceError** method is called when printing to check whether the printing device is available. Descendants can override it to perform specific actions before/after calling the OnPrintDeviceError event handler.

See Also

TCustomdxComponentPrinter.OnPrintDeviceError

DoProgress method

Generates an OnGenerateReportProgress event.

```
procedure DoProgress(AReportLink : TBasedxReportLink; PercentDone :  
Double); dynamic;
```

Description

This method is called when the report stored in the link, specified by the *AReportLink* parameter is being generated. The *PercentDone* parameter specifies the percentage of the work performed.

The **DoProgress** method calls the OnGenerateReportProgress event handler (if it has been assigned) and passes the *PercentDone* parameter and the appropriate TBasedxReportLink object to it. Descendants can override this method to perform specific actions before/after generating the OnGenerateReportProgress event.

See Also

TCustomdxComponentPrinter.OnGenerateReportProgress

DoStartPrint method

Generates an OnStartPrint event.

```
procedure DoStartPrint(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called before printing a report contained in the link, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the OnStartPrint event handler.

See Also

TCustomdxComponentPrinter.OnStartPrint

DoStartUpdateReport method

Generates an OnStartGenerateReport event.

```
procedure DoStartUpdateReport(AReportLink : TBasedxReportLink); dynamic;
```

Description

This method is called before updating the report contained in the link, specified by the *AReportLink* parameter. Descendants can override it to perform specific actions before/after calling the *OnStartUpdateReport* event handler.

See Also

TCustomdxComponentPrinter.OnStartGenerateReport

DrawPageFooter method

Draws a report page footer.

type

```
TdxPageTitlePart = (tpLeft, tpCenter, tpRight);
```

```
TdxPageTitleParts = set of TdxPageTitlePart;
```

```
procedure DrawPageFooter(AReportLink: TBasedxReportLink; APageIndex:  
Integer; ARect: TRect; ATitleParts: TdxPageTitleParts; ADrawBackground:  
Boolean);
```

Description

The **DrawPageFooter** method is called when a printing component builds the image of a report footer. The *AReportLink* parameter specifies the report link. The *APageIndex* parameter specifies the index of the page for which the footer is drawn. The *ARect* parameter specifies the bound rectangle for drawing. Since a footer can be divided into 3 sections, the *ATitleParts* parameter specifies one of these section. The *ADrawBackground* parameter specifies whether to draw the footer background.

Available values for the *ATitleParts* parameter include:

Value	Description
tpLeft	Footer is drawn within the left section.
tpCenter	Footer is drawn within the center section.
tpRight	Footer is drawn within the right section.

FindLinkByComponent method

Returns the report link by the assigned component.

```
function FindLinkByComponent(Value: TComponent): TBasedxReportLink;
```

Description

Every ExpressPrinting System report link is used to print a given component and thus each report link is assigned to a component. To find a report link assigned to a component call the **FindLinkByComponent** method. The *Value* parameter determines the printed component.

DrawPageHeader method

Draws a report page header.

type

```
TdxPageTitlePart = (tpLeft, tpCenter, tpRight);
```

```
TdxPageTitleParts = set of TdxPageTitlePart;
```

```
procedure DrawPageHeader(AReportLink: TBasedxReportLink; APageIndex:  
Integer; ARect: TRect; ATitleParts: TdxPageTitleParts; ADrawBackground:  
Boolean);
```

Description

The **DrawPageHeader** method is called when a printing component builds the image of a report header. The *AReportLink* parameter specifies the report link. The *APageIndex* parameter specifies the index of the page for which the header is drawn. The *ARect* parameter specifies the bound rectangle for drawing. Since a header can be divided into 3 sections, the *ATitleParts* parameter specifies one of these sections. The *ADrawBackground* parameter specifies whether to draw the header background.

Available values for the *ATitleParts* parameter include:

Value	Description
tpLeft	Header is drawn within the left section.
tpCenter	Header is drawn within the center section.
tpRight	Header is drawn within the right section.

GetNewLinkName method

Returns the name of a new link, namely the TBasedxReportLink object.

```
class function GetNewLinkName(AReportLink : TBasedxReportLink): string;
```

Description

This method specifies the name of a new link. This name consists of the TdxComponentPrinter object name + "Link" + the number of the link.

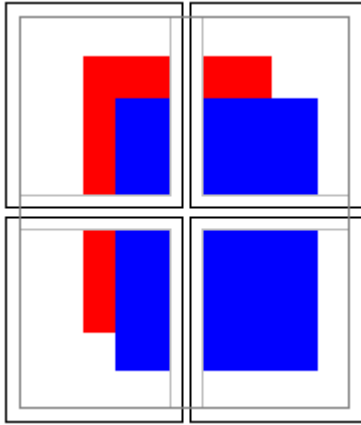
GetPageColRowCount method

Specifies the number of pages taken by the report image horizontally and vertically.

```
procedure GetPageColRowCount(var ACol, ARow : Integer; AReportLink:  
TBasedxReportLink {$IFDEF DELPHI4} = nil {$ENDIF});
```

Description

The **GetPageColRowCount** method calculates the number of pages occupied by the report images, on the basis of current page settings (page size, margins etc.). The *AReportLink* parameter specifies a report link to the printed component. Once this method is executed, the *APageColCount* parameter contains the number of 'column' pages and the *APageRowCount* parameter contains the numbers of 'row' pages. For instance, the **GetPageColRowCount** method returns 2 in the *APageColCount* parameter and 2 in the *APageRowCount* parameter:



See Also

TBasedxReportLink.GetPageColRowCount

GetPageCount method

Determines the number of pages in a report.

```
function GetPageCount(AReportLink: TBasedxReportLink {$IFDEF DELPHI4} =  
nil{$ENDIF}): Integer;
```

Description

Call the **GetPageCount** method to determine the number of pages in a report image stored in the link, specified by the *AReportLink* parameter.

GetPrintTitle method

Returns the name of a print job.

```
function GetPrintTitle(AReportLink : TBasedxReportLink): string; dynamic;
```

Description

The **GetPrintTitle** method returns the name of a print job. Users can view this name in the print queue. This method generates the OnGetPrintTitle event. If the OnGetPrintTitle event handler is not implemented, the **GetPrintTitle** method returns the name, specified by the PrintTitle property of the active link.

See Also

TCustomdxComponentPrinter.OnGetPrintTitle;TCustomdxComponentPrinter.PrintTitle

GetRealRegistryPath method

Returns the Registry path of the ExpressPrinting System.

```
function GetRealRegistryPath: string;
```

Description

The ExpressPrinting System allows a user to store different settings in the Registry. Call the **GetRealRegistryPath** method to obtain the default Registry path.

IndexOfLink method

Returns the index of a link.

```
function IndexOfLink(AReportLink : TBasedxReportLink): Integer;
```

Description

Use the **IndexOfLink** method to determine the index of a specific report link in the link list. This method applies the *AreportParameter* parameter that represents the TBasedxReportLink object to discover the 0-based index of the link.

See Also

TCustomdxComponentPrinter.IndexOfLinkByName

IndexOfLinkByName method

Returns the index of a link.

```
function IndexOfLinkByName (const AName : string): Integer;
```

Description

Use the **IndexOfLinkByName** method to determine the index of a specific report link in the link list. This method uses the *AName* parameter, which represents the name of the desired link to discover its 0-based index.

See Also

TCustomdxComponentPrinter.IndexOfLink

IsGenerateReportProgressEvent method

Determines the manner in which a report is generated.

```
function IsGenerateReportProgressEvent: Boolean;
```

Description

The **IsGenerateReportProgressEvent** method determines whether the **cpoGenerateReportProgressEvent** option of the Options property is active. If this method returns **True**, the printing component generates an OnGenerateReportProgress event while building a report.

See Also

TCustomdxComponentPrinter.Options

IsRebuildBeforePreview method

Determines the manner in which a report is generated.

```
function IsRebuildBeforePreview: Boolean;
```

Description

The **IsRebuildBeforePreview** method determines whether the **cpoAutoRebuildBeforePreview** option of the Options property is active. If this method returns **True**, the report is rebuilt before displaying the **PrintPreview** window.

See Also

TCustomdxComponentPrinter.Options

IsRebuildBeforePrint method

Determines the manner in which a report is generated.

```
function IsRebuildBeforePrint: Boolean;
```

Description

The **IsRebuildBeforePrint** method determines whether the **cpoAutoRebuildBeforePrint** option of the Options property is active. If this method returns **True**, the printing component rebuilds a report before printing.

See Also

TCustomdxComponentPrinter.Options

IsShowHourGlass method

Checks whether the **cpoShowHourGlass** option is enabled.

```
function IsShowHourGlass : Boolean;
```

Description

The **IsShowHourGlass** method determines whether the **cpoShowHourGlass** option of the Options property is active. If this method returns **True**, an HourGlass cursor appears during time consuming report operations.

See Also

TCustomdxComponentPrinter.Options

LinkByName method

Returns the TBasedxReportLink object by its name.

```
function LinkByName(const AName : string): TBasedxReportLink;
```

Description

Use the **LinkByName** method to access the desired TBasedxReportLink object by the *AName* parameter representing the name of this link.

See Also

TCustomdxComponentPrinter.IndexOfLinkByName

PageSetup method

Activates the page setup dialog.

```
function PageSetup(AReportLink: TBasedxReportLink {$IFDEF DELPHI4} =  
nil {$ENDIF}): Boolean;
```

Description

Use this method to customize page settings of the report stored in the link, specified by the *AReportLink* parameter. After calling the PageSetup method of the TBasedxReportLink object, this method calls the DoPageSetUp method to generate the OnPageSetUp event.

See Also

TCustomdxComponentPrinter.DoPageSetUp; *TCustomdxComponentPrinter.OnPageSetUp*

PageSetupEx method

Displays the "**PageSetup**" dialog window.

```
function PageSetupEx(AActivePageIndex: Integer; APreviewBtnClicked,  
APrintBtnClicked: PBoolean; AReportLink: TBasedxReportLink {$IFDEF DELPHI4}  
= nil {$ENDIF}): Boolean;
```

Description

The **PageSetupEx** function is called to display the "**PageSetup**" dialog window. The *AActivePageIndex* parameter specifies which page of the "**PageSetup**" dialog window is displayed by default. The *APreviewBtnClicked* and *APrintBtnClicked* parameters specify whether to display the "**Preview...**" and "**Print Preview...**" buttons in this dialog window. These parameters allow a developer to ascertain which of these buttons was pressed.

The *AReportLink* parameter specifies the report link. The *AReportLink* parameter can take **nil** value by default since Delphi 4, said differently you may omit this very parameter. In this case the current report link is printed out.

Note: If the used parameter is not **nil**, the corresponding button is displayed.

PaintPage method

Paints a single page of a report.

```
procedure PaintPage(ACanvas : TCanvas; APageIndex : Integer; const  
ApageRect, AContentRect : TRect; AReportLink : TBasedxReportLink {$IFDEF  
DELPHI4} = nil {$ENDIF});
```

Description

The **PaintPage** method paints a single page specified by the *APageIndex* parameter. The *AReportLink* parameter determines the link that contains the report to which the page belongs. The *APageRect* parameter specifies the rectangular portion of the page image with margins. The *AContentRect* parameter specifies the rectangular portion of the page image without margins. The *ACanvas* parameter determines the drawing canvas.

PrepareReport method

Prepares report for printing.

```
procedure PrepareReport(AReportLink : TBasedxReportLink); virtual;
```

Description

The **PrepareReport** method is called to force the ExpressPrinting System to build the report if it has not been built yet. This method sets the flag that report is prepared. The *AReportLink* parameter specifies the link for which to build the report.

See Also

TCustomdxComponentPrinter.UnPrepareReport; *TCustomdxComponentPrinter.RebuildReport*

Preview method

Activates the preview window.

```
procedure Preview (AModal : Boolean; AReportLink: TBasedxReportLink {$IFDEF  
DELPHI4} = nil {$ENDIF});
```

Description

Call the **Preview** method to display the preview window on screen. If *AModal* is True, then the preview window appears as a modal form. The *AReportLink* parameter specifies the link to be previewed. Use the *PreviewExists* method to determine whether a preview form is active.

See Also

TCustomdxComponentPrinter.PreviewWindow; *TCustomdxComponentPrinter.PreviewOptions*;
TCustomdxComponentPrinter.PreviewExists; *TCustomdxComponentPrinter.PreviewWindowDesigner*

PreviewExists method

Determines whether the preview window is displayed.

```
function PreviewExists: Boolean;
```

Description

The **PreviewExists** method returns **True** if the preview window currently exists. Otherwise, it returns **False**. Use the *Preview* method to activate the preview form.

See Also

TCustomdxComponentPrinter.Preview

Print method

Prints a report.

type

```
PdxPrintDlgData = ^TdxFPrintDlgData;
function Print(AShowDialog : Boolean; APrintDlgData : PdxPrintDlgData;
AReportLink: TBasedxReportLink {$IFDEF DELPHI4} = nil{$ENDIF}): Boolean;
```

```
function Print(AShowDialog: Boolean; APrintDlgData: PdxPrintDlgData;
AReportLink: TBasedxReportLink{$IFDEF DELPHI4} = nil{$ENDIF}):
Boolean;
```

Description

Call the **Print** method to print a report to the default printer.

The *AShowDialog* parameter determines whether to activate a print dialog before printing. If *AShowDialog* is **True**, the print dialog is active.

The *PdxPrintDlgData* parameter is the pointer to a structure that describes properties of a print dialog.

The *AReportLink* parameter specifies the link pointing to the report to print. It allows users to print not only the report contained in the currently active link, but any other from the list of available links. The *AReportLink* parameter can take **nil** value by default since Delphi 4, said differently you may omit this very parameter. In this case the current report link is printed out.

Note: *TdxPrintDlgData* is used internally to define print attributes. In most cases, it's easier to use the *PrintEx*, *PrintPage*, *PrintPages* and *PrintPagesEx* methods instead of the *Print*.

See Also

TCustomdxComponentPrinter.PrintEx; *TCustomdxComponentPrinter.PrintPage*;
TCustomdxComponentPrinter.PrintPages; *TCustomdxComponentPrinter.PrintPagesEx*

PrintEx method

Prints report pages.

type

```
TdxPageNumbers = (pnAll, pnEven, pnOdd);
procedure PrintEx(APageNums : TdxPageNumbers; ACopies : Integer;
ACollate : Boolean; AReportLink: TBasedxReportLink {$IFDEF DELPHI4} =
nil{$ENDIF});
```

Description

Use this method to print all pages in a particular range. The *APageNums* parameter specifies pages to print:

pnAll	Prints all pages.
pnEven	Prints only even pages.
pnOdd	Prints only odd pages.

The *ACopies* parameter specifies the number of copies to print. If the *ACollate* parameter is **True**, then report pages are collated. Unlike the *PrintPagesEx* method, the **PrintEx** method prints all pages of a separate subset. For instance, it prints all even or odd pages.

The *AReportLink* parameter specifies the link that stores a report to print. It allows users to print not only the report of the current active link, but any other from the list of available links.

Include *dxPrnDlg* in the *uses* section of a unit to use type *TdxPageNumbers*.

See Also

TCustomdxComponentPrinter.Print; *TCustomdxComponentPrinter.PrintPage*;
TCustomdxComponentPrinter.PrintPages; *TCustomdxComponentPrinter.PrintPagesEx*

PrintPage method

Prints a specific page of a report.

```
procedure PrintPage(AReportLink : TBasedxReportLink; APageIndex : Integer);
```

Description

Use the **PrintPage** method to print a specific page of a report contained in the link specified by the *AReportLink* parameter. The *APageIndex* parameter specifies the page to print. Unlike the *Print* method, the **PrintPage** method allows users to print the page specified by the *APageIndex* parameter. You may use the *Print* method to print all pages, but in most cases, it's easier to use the *PrintEx*, *PrintPage*, *PrintPages* and *PrintPagesEx* methods instead of the *Print*.

See Also

TCustomdxComponentPrinter.Print; *TCustomdxComponentPrinter.PrintEx*;
TCustomdxComponentPrinter.PrintPages; *TCustomdxComponentPrinter.PrintPagesEx*

PrintPages method

Prints pre-defined pages of a report.

```
procedure PrintPages(const APageIndexes : array of Integer;  

AReportLink: TBasedxReportLink {$IFDEF DELPHI4} = nil{$ENDIF});
```

Description

Use the *PrintPages* method to print pre-defined pages of the report contained in the link specified by the *AReportLink* parameter. The *AReportLink* parameter can take **nil** value by default since Delphi 4, said differently you may omit this very parameter. In this case the current report link is printed out.

The *APageIndexes* parameter specifies the pages to print. For example, to print the first and fourth pages, you should write the following event handler:

```
dxComponentPrinter1.PrintPages([0, 3]);
```

Unlike the `Print` method, the **PrintPages** method allows users to print separate pages. You may use the `Print` method to print all pages, but in most cases, it's easier to use the `PrintEx`, `PrintPage`, `PrintPages` and `PrintPagesEx` methods instead of the `Print`.

See Also

*TCustomdxComponentPrinter.Print; TCustomdxComponentPrinter.PrintEx;
TCustomdxComponentPrinter.PrintPage; TCustomdxComponentPrinter.PrintPagesEx*

PrintPagesEx method

Prints specific report pages.

type

```
TdxPageNumbers = (pnAll, pnEven, pnOdd);

procedure PrintPagesEx(const APageIndexes : array of Integer; APageNums :
TdxPageNumbers; ACopies : Integer; ACollate : Boolean; AReportLink:
TBasedxReportLink {$IFDEF DELPHI4} = nil{$ENDIF});
```

Description

Unlike the `PrintEx` method, the **PrintPagesEx** method provides printing of individual pages within a range. The *APageIndexes* parameter specifies pages to print. The *APageNums* parameter determines range of pages to print:

pnAll	Prints all pages.
pnEven	Prints only even pages.
pnOdd	Prints only odd pages.

The *ACopies* parameter specifies the number of copies to print.

The *ACollate* parameter determines whether to collate printed pages.

If *ACollate* is **False**, pages are not collated.

The *AReportLink* parameter specifies the report link. The *AReportLink* parameter can take **nil** value by default since Delphi 4, said differently you may omit this very parameter. In this case the current report link is printed out.

Include *dxPrnDlg* in the *uses* section of a unit to use type `TdxPageNumbers`.

See Also

*TCustomdxComponentPrinter.Print; TCustomdxComponentPrinter.PrintEx;
TCustomdxComponentPrinter.PrintPage; TCustomdxComponentPrinter.PrintPages*

RebuildReport method

Rebuilds the contents of a report.

```
procedure RebuildReport(AReportLink: TBasedxReportLink {$IFDEF DELPHI4} =
nil{$ENDIF});
```

Description

This method is most useful when it is required to re-build a report after making changes to printed components (such as changing data in cells or changing cell layouts). Use the **BuildReport** method to entirely re-build a report.

If you cannot use the auto rebuild feature, call the `RebuildReport` method to update the report's contents. The *AReportLink* parameter determines the link to which the report belongs, therefore users can rebuild not only a report stored in the active link, but any other as well.

See Also

TCustomdxComponentPrinter.IsGenerateReportProgressEvent;
TCustomdxComponentPrinter.IsRebuildBeforePreview;
TCustomdxComponentPrinter.IsRebuildBeforePrint;TCustomdxComponentPrinter.Options

StdProcessBusy method

Raises an exception.

```
procedure StdProcessBusy; virtual;
```

Description

The **StdProcessBusy** method raises an exception when the printing device is busy.

See Also

TCustomdxComponentPrinter.DoPrintDeviceBusy;TCustomdxComponentPrinter.OnPrintDeviceBusy

UnPrepareReport method

Sets the report to the unprepared state.

```
procedure UnPrepareReport(AReportLink : TBasedxReportLink); virtual;
```

Description

The **UnPrepareReport** method is called to force the ExpressPrinting System to set a flag that a report is not prepared. The *AReportLink* parameter specifies the link for this flag.

See Also

TCustomdxComponentPrinter.PrepareReport

TCustomdxComponentPrinter Events

TCustomdxComponentPrinter

OnAddReportLink	public	OnEndGenerateReport	public
OnAfterPreview	protected	OnEndPrint	public
OnBeforePreview	protected	OnGenerateReportProgress	public
OnChangeComponent	protected	OnGetPrintTitle	public
OnChangeCurrentLink	protected	OnMeasureReportTitle	protected
OnCustomDrawPage	public	OnNewPage	public
OnCustomDrawPageFooter	protected	OnPageSetup	public
OnCustomDrawPageHeader	protected	OnPrintDeviceBusy	public
OnCustomDrawReportTitle	protected	OnPrintDeviceError	public
OnDeleteReportLink	public	OnStartGenerateReport	public
OnDesignReport	protected	OnStartPrint	public

OnAddReportLink event

Occurs when adding a new report link.

type

```
TdxReportLinkNotifyEvent = procedure(Sender : TObject; AReportLink :
TBasedxReportLink) of object;
property OnAddReportLink : TdxReportLinkNotifyEvent;
```

Description

Write an **OnAddReportLink** event handler to perform specific actions after a new report link has been added to the list. For instance, this event can be used to check the number of currently available report links.

See Also

TCustomdxComponentPrinter.AddLink;TCustomdxComponentPrinter.AddLinkEx;
TCustomdxComponentPrinter.DoAddReportLink

OnAfterPreview event

Occurs after closing the preview form.

type

```
TdxPreviewEvent = procedure(Sender : TObject; AReportLink :
TBasedxReportLink ) of object;
property OnAfterPreview : TdxPreviewEvent;
```

Description

Write an **OnAfterPreview** event handler to perform specific actions when closing the preview window of the report contained in the link specified by the *AReportLink* parameter.

See Also

TCustomdxComponentPrinter.DoAfterPreview

OnBeforePreview event

Occurs before the preview window is to be activated.

type

`TdxPreviewEvent = procedure(Sender : TObject; AReportLink : TBasedxReportLink ) of object;`

property OnBeforePreview : TdxPreviewEvent;

Description

Write an **OnBeforePreview** event handler to perform specific actions when displaying the preview window of the report contained in the link specified by the *AReportLink* parameter. For instance, the following code changes the caption of a preview window of the active link (dxComponentPrinter1BeforePreview is the event handler of the TdxComponentPrinter OnBeforePreview event).

```
procedure TForm1.dxComponentPrinter1BeforePreview(Sender: TObject;  
AReportLink: TBasedxReportLink);  
begin  
    dxComponentPrinter1.PreviewWindow.Caption := 'Report Preview Form';  
end;
```

See Also

TCustomdxComponentPrinter.DoBeforePreview

OnChangeComponent event

Occurs when the Component property changes.

type

`TdxReportLinkNotifyEvent = procedure(Sender : TObject; AReportLink : TBasedxReportLink) of object;`

property OnChangeComponent : TdxReportLinkNotifyEvent;

Description

Write an **OnChangeComponent** event handler to perform specific actions when changing the Component property of the link specified by the *AReportLink* parameter.

See Also

TCustomdxComponentPrinter.DoChangeComponent

OnChangeCurrentLink event

Occurs when changing a report link.

type

```
TNotifyEvent = procedure (Sender: TObject) of object;
property OnChangeCurrentLink : TNotifyEvent;
```

Description

Write an **OnChangeCurrentLink** event handler to perform specific actions when the current report link changes. For instance, this event can be used to display a message notifying users when changing a report link.

See Also

TCustomdxComponentPrinter.CurrentLink

OnCustomDrawPage event

Occurs when drawing individual report pages.

type

```
TdxCustomDrawPageEvent = procedure(Sender : TObject; AReportLink :
TBasedxReportLink; ACanvas : TCanvas; APageIndex : Integer; ARect : TRect;
ANom, ADenom : Integer) of object;
property OnCustomDrawPage: TdxCustomDrawPageEvent;
```

Description

Write an **OnCustomDrawPage** event handler to draw a specific report page of the link specified by the *AReportLink* parameter.

The *ACanvas* parameter determines the drawing canvas.

The *ARect* parameter specifies the bound rectangle for drawing.

The *APageIndex* parameter determines the index of the page being drawn.

See Also

TCustomdxComponentPrinter.DoCustomDrawPage; TCustomdxComponentPrinter.PaintPage

OnCustomDrawPageFooter event

Occurs when drawing a page footer.

type

```
TdxCustomDrawPageHFEvent = procedure(Sender: TObject; AReportLink:
TBasedxReportLink; ACanvas: TCanvas; APageIndex: Integer; var ARect: TRect;
ANom, ADenom: Integer; var ADefaultDrawText, ADefaultDrawBackground:
Boolean) of object;

property OnCustomDrawPageFooter: TdxCustomDrawPageHFEvent;
```

Description

Write an **OnCustomDrawPageFooter** event handler to draw a specific report page footer for the report link specified by the *AReportLink* parameter.

The *ACanvas* parameter determines the drawing surface.

The *APageIndex* parameter specifies the index of the page for which the footer is drawn.

The *ARect* parameter specifies the bound rectangle for drawing.

The *ANom* and *ADenom* parameters determine the coefficient of magnification and demagnification accordingly.

The *ADefaultDrawText* and *ADefaultDrawBackground* parameters specify whether to draw predefined footer text and background accordingly.

OnCustomDrawPageHeader event

Occurs when drawing a page header.

type

```
TdxCustomDrawPageHFEvent = procedure(Sender: TObject; AReportLink:
TBasedxReportLink; ACanvas: TCanvas; APageIndex: Integer; var ARect: TRect;
ANom, ADenom: Integer; var ADefaultDrawText, ADefaultDrawBackground:
Boolean) of object;
```

property OnCustomDrawPageHeader: TdxCustomDrawPageHFEvent;

Description

Write an **OnCustomDrawPageHeader** event handler to draw a specific report page header for the report link specified by the *AReportLink* parameter.

The *ACanvas* parameter determines the drawing surface.

The *APageIndex* parameter specifies the index of the page for which the header is drawn.

The *ARect* parameter specifies the bound rectangle for drawing.

The *ANom* and *ADenom* parameters determine the coefficient of magnification and demagnification accordingly.

The *ADefaultDrawText* and *ADefaultDrawBackground* parameters specify whether to draw predefined header text and background accordingly.

OnCustomDrawReportTitle event

Occurs when drawing the report title.

type

```
TdxTextAlignX = ( taLeft, taCenterX, taRight );
TdxTextAlignY = ( taTop, taCenterY, taBottom );
TdxCustomDrawReportLinkTitleEvent = procedure(Sender : TObject;
AReportLink : TBasedxReportLink; ACanvas : TCanvas; ARect : TRect; ANom,
ADenom : Integer; var TextAlignX : TdxTextAlignX; var TextAlignY :
TdxTextAlignY; var AColor : TColor; AFont : TFont; var ADone : Boolean) of
object;
```

property OnCustomDrawReportTitle : TdxCustomDrawReportTitleEvent;

Description

Write an **OnCustomDrawReportLinkTitle** event handler to draw a specific report title for the report link specified in the *AReportLink* parameter. The *ACanvas* parameter determines the drawing surface.

The *ARect* parameter specifies the bound rectangle for drawing.

When using custom draw methods, you must manage image repainting during zoom events. It is always necessary when previewing the report and necessary only for images when printing the report. The *ANom* and *ADenom* parameters determine the coefficient of magnification and demagnification accordingly. The *TextAlignX* and *TextAlignY* parameters specify horizontal and vertical alignment accordingly. The *AColor* parameter sets the background color for the report title. The *AFont* parameter sets the font for the report title. The *ADone* parameter determines whether the **OnCustomDrawReportLinkTitle** method processes the entire operation. The height of the report title's region is calculated automatically and depends upon the font used but if the **Font.Size** property is changed you must also handle the *OnMeasureReportTitle* event to correct the height of the report title. To draw an image in the report title you should also determine its real height via the *OnMeasureReportTitle* event.

Available values for the *TextAlignX* parameter include:

Values	Meaning
taLeft	Text is left justified. All text lines are at the left edge of an item.
taCenterX	Text is centered within a cell.
taRight	Text is right justified. All text lines are aligned at the right edge of an item.

By default, the **TextAlignX** property value is **taCenterX**.

Available values the *TextAlignY* parameter include:

Values	Meaning
taTop	Text is aligned at the top edge of an item.
taCenterY	Text is vertically centered within a cell.
taBottom	Text is aligned at the bottom of an item's cell.

By default, the **TextAlignY** property value is equal to **taCenterY**.

See Also

TCustomdxComponentPrinter.DoCustomDrawReportTitle

OnDeleteReportLink event

Occurs when a report link is to be deleted.

type

```
TdxReportLinkNotifyEvent = procedure(Sender : TObject; AReportLink :
TBasedxReportLink) of object;
property OnDeleteReportLink : TdxReportLinkNotifyEvent;
```

Description

Write an **OnDeleteReportLink** event handler to perform specific actions when the report link specified in the *AReportLink* parameter, is about to be removed from the list of available links.

See Also

TCustomdxComponentPrinter.DeleteAllLinks;TCustomdxComponentPrinter.DoDeleteReportLink

OnDesignReport event

Occurs when the report link designer is closed.

type

```
TdxDesignReportEvent = procedure(Sender : TObject; AReportLink :  
TBasedxReportLink; var ADone : Boolean ) of object;  
property OnDesignReport : TdxDesignReportEvent;
```

Description

Write an **OnDesignReport** event handler when closing the report link designer. The *ADone* parameter determines whether pressing the 'OK' button has closed the designer window. The *AReportLink* parameter determines a link of the designed report.

See Also

TCustomdxComponentPrinter.DoDesignReport

OnEndGenerateReport event

Occurs when a report has just been built.

type

```
TdxReportLinkNotifyEvent = procedure(Sender : TObject; AReportLink :  
TBasedxReportLink) of object;  
property OnEndGenerateReport : TNotifyEvent;
```

Description

Write an **OnEndGenerateReport** event handler to perform specific actions as soon as the printing component finishes building a report contained in the link specified by the *AReportLink* parameter. For instance, this event can be used to destroy the report build progress indicator.

See Also

TCustomdxComponentPrinter.DoEndUpdateReport

OnEndPrint event

Occurs after printing a report.

type

```
TdxReportLinkNotifyEvent = procedure(Sender : TObject; AReportLink :  
TBasedxReportLink) of object;  
property OnEndPrint : TNotifyEvent;
```

Description

Write an **OnEndGenerateReport** event handler to perform specific actions when printing a report contained in the link specified by the *AReportLink* parameter.

See Also

TCustomdxComponentPrinter.DoEndPrint

OnGenerateReportProgress event

Occurs during report building.

type

```
TdxGenerateReportProgressEvent = procedure(Sender : TObject;
AReportLink : TBasedxReportLink; APercentDone : Double ) of object;
property OnGenerateReportProgress : TdxGenerateReportProgressEvent;
```

Description

Write an **OnGenerateReportProgress** event handler to perform specific actions while building a report contained in the link specified by the *AReportLink* parameter. The *APercentDone* parameter determines work completed in percents. This event allows you to create a progress bar to inform users of the report building process:



Note: This event is generated only if the *cpoGenerateReportProgressEvent* option is active.

See Also

TCustomdxComponentPrinter.DoProgress; *TCustomdxComponentPrinter.OnEndGenerateReport*;
TCustomdxComponentPrinter.Options

OnGetPrintTitle event

Occurs when a printing component starts printing.

type

```
TdxGetPrintTitleEvent = procedure(Sender : TObject; AReportLink :
TBasedxReportLink; var ATitle : string ) of object;
property OnGetPrintTitle : TdxGetPrintTitleEvent;
```

Description

Write an **OnGetPrintTitle** event handler to specify the name of a print job displayed in the print queue. Use the *ATitle* parameter to specify the print job's name. The *AReportLink* parameter specifies a link to which this report belongs.

See Also

TCustomdxComponentPrinter.GetPrintTitle; *TCustomdxComponentPrinter.PrintTitle*

OnMeasureReportTitle event

Occurs when drawing the report title before the OnCustomDrawReportTitle event.

type

```
TdxMeasureReportTitleEvent = procedure(Sender : TBasedxReportLink; var  
AHeight : Integer) of object;  
property OnMeasureReportTitle : TdxMeasureReportTitleEvent;
```

Description

Write an **OnMeasureReportTitle** event handler when you wish to draw an image within the report title. Because the height of the report title is calculated automatically and depends on the font used, you should specify the real height of the image using the *AHeight* parameter.

See Also

TBasedxReportTitle.OnMeasureReportLinkTitle;
TCustomdxComponentPrinter.OnCustomDrawReportTitle

OnNewPage event

Occurs before sending a regular page of a multi-page document to printer.

type

```
TdxNewPageEvent = procedure(Sender : TObject; AReportLink :  
TBasedxReportLink; APageIndex : Integer) of object;  
property OnNewPage : TdxNewPageEvent;
```

Description

Write an **OnNewPage** event handler to perform specific actions before printing a new page of the report contained in the link specified by the *AReportLink* parameter. For instance, the **OnNewPage** event can be used to display and update a progress indicator of sending pages to a printer device.

The *APageIndex* parameter specifies the page number of a new page.

See Also

TCustomdxComponentPrinter.DoNewPage

OnPageSetUp event

Occurs when closing the **PageSetup** dialog.

type

```
TdxPageSetUpEvent = procedure(Sender : TObject; AReportLink :  
TBasedxReportLink; ADone : Boolean ) of object;  
property OnPageSetUp : TdxPageSetUpEvent;
```

Description

Write an **OnPageSetUp** event handler to perform specific actions when closing the **PageSetup** window. The *ADone* parameter is set to **True** if the PageSetup form was closed by pressing the 'OK' button. The *AReportLink* parameter determines the report link for which the **PageSetup** window was called.

See Also

TCustomdxComponentPrinter.DoPageSetUp

OnPrintDeviceBusy event

Occurs when the printing device is busy.

type

```
TdxPrintDeviceBusyEvent = procedure(Sender : TObject; var ADone : Boolean)  
of object;
```

```
property OnPrintDeviceBusy : TdxPrintDeviceBusyEvent;
```

Description

Write an **OnPrintDeviceBusy** event handler to perform specific actions when the printing device is busy. For instance, this event can be used to prevent the display of exception messages. The *ADone* parameter determines whether the **OnPrintDeviceBusy** event handler entirely processes the operation. If this parameter is **True**, the default procedure is not called; Otherwise, an exception is raised.

See Also

TCustomdxComponentPrinter.DoPrintDeviceBusy;TCustomdxComponentPrinter.StdProcessBusy

OnPrintDeviceError event

Occurs when the printing device is not available (for example, disconnected).

type

```
TdxPrintDeviceProblemEvent = procedure(Sender : TObject; var ADone :  
Boolean) of object;
```

```
property OnPrintDeviceError : TdxPrintDeviceProblemEvent;
```

Description

Write an **OnPrintDeviceError** event handler to perform specific actions when any problem occurs with printing device. For instance, this event can be used to prevent the display of exception messages when printer is disconnected while report printing. The *ADone* parameter determines whether the **OnPrintDeviceError** event handler entirely processes the operation. If this parameter is **True**, the default procedure is not called; Otherwise, an exception is raised.

See Also

TCustomdxComponentPrinter.DoPrintDeviceError

OnStartGenerateReport event

Occurs when the printing component is about to build a report.

type

```
TdxReportLinkNotifyEvent = procedure(Sender : TObject; AReportLink :  
TBasedxReportLink) of object;
```

```
property OnStartGenerateReport : TdxReportLinkNotifyEvent;
```

Description

Write an **OnStartGenerateReport** event handler to perform specific actions before the printing component starts building a report contained in the link specified by the *AReportLink* parameter. For instance, this event can be used to create an indicator for a report build process.

See Also

TCustomdxComponentPrinter.DoStartUpdateReport;
TCustomdxComponentPrinter.OnEndGenerateReport;
TCustomdxComponentPrinter.OnGenerateReportProgress

OnStartPrint event

Occurs before the printing component prints the report.

type

```
TdxReportLinkNotifyEvent = procedure(Sender : TObject; AReportLink :  
TBasedxReportLink) of object;
```

```
property OnStartPrint : TdxReportLinkNotifyEvent;
```

Description

Write an **OnStartPrint** event handler to perform specific actions when the printing component starts printing a report contained in the link specified by the *AReportLink* parameter.

See Also

*TCustomdxComponentPrinter.DoStartPrint;**TCustomdxComponentPrinter.OnEndPrint*

TdxComponentPrinter Object

TdxComponentPrinter is used to print VCL components.

Unit

dxPSCore

```
TdxComponentPrinter = class(TCustomdxComponentPrinter);
```

Description

TdxComponentPrinter allows users to print other components. In order to print specific components, you must be certain that a report link for this component class actually exists. Use a report link object to customize print properties of a printed component.

Hierarchy

```

  TObject
    TPersistent
      TComponent
        TCustomdxComponentPrinter
```

TdxComponentPrinter Properties

In TdxComponentPrinter

OverWriteExistingFiles **published / read / write**

Derived from TCustomdxComponentPrinter

AbortPrinting	public / read / write	PreviewWindowDesigner	public / read only
CurrentLink	public / read / write	PrintTitle	published / read / write
LinkCount	public / read only	ReportLink	public / read / write
Options	published / read / write	ReportLinkDesigner	public / read only
PreviewOptions	published / read / write	State	public / read only
PreviewWindow	public / read only		

OverWriteExistingFiles property

read / write

Specifies whether to overwrite files when saving new files to disk.

```
property OverWriteExistingFiles: Boolean;
```

Description

The SavePagesAsImagesToDisk method allows you to save pages to disk. Each page corresponds to a certain file. Set the **OverWriteExistingFiles** property to **True** if you are going to replace old page files with new ones.

The default value for this property is **False**.

TdxComponentPrinter Methods

TdxComponentPrinter

EnumPagesAsImages	public
SavePagesAsImagesToDisk	public

Derived from TCustomdxComponentPrinter

AddEmptyLink	public	DoStartPrint	protected
AddEmptyLinkEx	public	DoStartUpdateReport	protected
AddLink	public	GetNewLinkName	public
AddLinkEx	public	GetPageColRowCount	public
AssignReportLinks	public	GetPageCount	public
CalcPageDrawData	protected	GetPrintTitle	protected
Create	public	IndexOfLink	public
DeleteAllLinks	public	IndexOfLinkByName	public
DesignerExists	public	IsGenerateReportProgressEvent	protected
DesignReport	public	IsRebuildBeforePreview	protected
Destroy	public	IsRebuildBeforePrint	protected
DestroyReport	public	IsShowHourGlass	protected
DoAddReportLink	protected	LinkByName	public
DoAfterPreview	protected	PageSetup	public
DoBeforePreview	protected	PaintPage	public
DoChangeComponent	protected	PaintRect	protected
DoCustomDrawPage	protected	PrepareReport	protected
DoCustomDrawReportTitle	protected	Preview	public
DoDeleteReportLink	protected	PreviewExists	public
DoDesignReport	protected	Print	public
DoEndPrint	protected	PrintEx	public
DoEndUpdateReport	protected	PrintPage	protected
DoNewPage	protected	PrintPages	public
DoPageSetUp	protected	PrintPagesEx	public
DoPrintDeviceBusy	protected	RebuildReport	public
DoPrintDeviceError	protected	StdProcessBusy	protected
DoProgress	protected	UnPrepareReport	protected

EnumPagesAsImages method

Converts report pages to graphic images.

type

```
TdxEnumPagesAsImagesProc = procedure(AComponentPrinter:
TCustomdxComponentPrinter; AReportLink: TBasedxReportLink; AIndex,
APageIndex: Integer; const AGraphic: TGraphic; AData: Pointer; var
AContinue: Boolean) of object;
```

```

procedure EnumPagesAsImages(const APageIndexes: array of Integer;
AGraphicClass: TGraphicClass; ADrawBackground: Boolean; ACallBackProc:
TdxEnumPagesAsImagesProc; ACallBackData, AProgressData, APrepareData:
Pointer; AReportLink: TBasedxReportLink {$IFDEF DELPHI4} = nil {$ENDIF});

```

Description

The **EnumPagesAsImages** method is called within the **SavePagesAsImagesToDisk** method to obtain report pages for the report link specified by the *AReportLink* parameter in graphic representation.

The *APageIndexes* parameter specifies absolute indexes of report pages used in specific events.

The *AGraphicClass* parameter specifies the image type.

Set the *ADrawBackground* parameter to **True** to draw the original background within an image.

The *ACallBackData*, *AProgressData* and *APrepareData* parameters determine pointers to arbitrary data which is used in the **TdxEnumPagesAsImagesProc** procedure, as well as in the **OnExportProgress** and **OnExportPrepareGraphic** events.

The **TdxEnumPagesAsImagesProc** procedure includes the following parameters:

Parameter	Description
<i>AComponentPrinter</i>	Determines the component printer which sends data.
<i>AReportLink</i>	Determines the report link whose report is being converted.
<i>AIndex</i>	Determines the zero-based index of the exported report page among other pages.
<i>APageIndex</i>	Determines the absolute index of the current page within a report, passed via the <i>APageIndexes</i> parameter of the EnumPagesAsImages method.
<i>AGraphic</i>	Specifies the image created for a specific page.
<i>AData</i>	Contains the pointer to additional arbitrary data which is determined by the <i>ACallBackData</i> parameter of the EnumPagesAsImages method.
<i>AContinue</i>	Set this parameter to True to continue converting report pages to graphic images or to False , to break all operations.

See Also

TdxComponentPrinter.SavePagesAsImagesToDisk

SavePagesAsImagesToDisk method

Saves report pages to graphic images on disk.

```

procedure SavePagesAsImagesToDisk(const APageIndexes: array of Integer;
AGraphicClass: TGraphicClass; ADrawBackground: Boolean; const AFileMask:
string; AProgressData, APrepareData: Pointer; AReportLink:
TBasedxReportLink {$IFDEF DELPHI4} = nil {$ENDIF});

```

Description

The **SavePagesAsImagesToDisk** calls the **EnumPagesAsImages** method to obtain report pages for the report link specified by the *AReportLink* parameter in graphic representation.

The *APageIndexes* parameter specifies absolute indexes of report pages used in specific events.

The *AGraphicClass* parameter specifies image type.

Set the *ADrawBackground* parameter to **True** to draw the original background within an image.

The *AFileMask* parameter sets a mask for saved pages. For instance, the 'Page%d.bmp' mask results in a 'Page1.bmp' file.

The *AProgressData* and *APrepareData* parameters determine the pointers to arbitrary data used in the **OnExportProgress** and **OnExportPrepareGraphic** events.

See Also

TdxComponentPrinter.EnumPagesAsImages

TdxComponentPrinter Events

TdxComponentPrinter

OnExportGetPageFileName	published
OnExportPrepareGraphic	published
OnExportProgress	published

Derived from TCustomdxComponentPrinter

OnAddReportLink	public	OnEndPrint	published
OnAfterPreview	published	OnGenerateReportProgress	published
OnBeforePreview	published	OnGetPrintTitle	published
OnChangeComponent	published	OnMeasureReportTitle	published
OnChangeCurrentLink	published	OnNewPage	published
OnCustomDrawPage	published	OnPageSetup	published
OnCustomDrawReportTitle	published	OnPrintDeviceBusy	published
OnDeleteReportLink	published	OnPrintDeviceError	published
OnDesignReport	published	OnStartGenerateReport	published
OnEndGenerateReport	published	OnStartPrint	published

OnExportGetPageFileName event

Occurs when saving a page to disk.

type

```
TdxExportGetPageFileNameEvent = procedure(Sender:
TCustomdxComponentPrinter; AIndex, APageIndex: Integer; var AFileName:
string) of object;
property OnExportGetPageFileName: TdxExportGetPageFileNameEvent;
```

Description

Write an **OnExportGetPageFileName** event handler to perform specific actions when exporting a page to a file on disk.

The *Sender* parameter determines the component printer which sends data.

The *AIndex* parameter determines the zero-based index of the exported report page among other pages in a report.

The *APageIndex* parameter determines the absolute index of the current page.

The *AFileName* parameter determines the file name to which a page is exported.

See Also

TdxComponentPrinter.SavePagesAsImagesToDisk

OnExportPrepareGraphic event

Occurs when exporting a page to graphics.

type

```
TdxExportPrepareGraphicEvent = procedure(Sender:
TCustomdxComponentPrinter; AReportLink: TBasedxReportLink; const AGraphic:
TGraphic; AData: Pointer) of object;
property OnExportPrepareGraphic: TdxExportPrepareGraphicEvent;
```

Description

The *Sender* parameter determines the component printer which sends data.

The *AGraphic* parameter specifies the image which is created for a specific page.

The *AData* parameter contains the pointer to additional arbitrary data.

See Also

TdxComponentPrinter.EnumPagesAsImages; TdxComponentPrinter.SavePagesAsImagesToDisk

OnExportProgress event

Occurs when exporting a report to graphic images or to disk.

type

```
TdxExportProgressEvent = procedure(Sender: TCustomdxComponentPrinter;
AReportLink: TBasedxReportLink; APageCount, AIndex, APageIndex: Integer;
AData: Pointer) of object;
property OnExportProgress: TdxExportProgressEvent;
```

Description

Write an **OnExportProgress** event handler to perform specific actions when exporting a report for the report link specified by the *AReportLink* parameter to graphics or to disk.

The *Sender* parameter determines the component printer which sends data.

The *AIndex* parameter determines the zero-based index of the exported report page among other pages.

The *APageIndex* parameter determines the absolute index of the current page in a report.

The *AFileName* parameter determines the file name to which a page is exported.

The *AData* parameter contains the pointer to additional arbitrary data which is determined by the *AProgressData* parameter of the **SavePagesAsImagesToDisk** or **EnumPagesAsImages** methods.

See Also

TdxComponentPrinter.EnumPagesAsImages; TdxComponentPrinter.SavePagesAsImagesToDisk

TBasedxPrintStyle Object

TBasedxPrintStyle is the base style for printing reports.

Unit

dxPgsDlg

TBasedxPrintStyle = **class**(TComponent)

Description

The ExpressPrinting System now incorporates a style feature. You may now print using a variety of custom settings (styles) within any report. For instance, you can print the ExpressQuantumGrid on different paper formats without any additional page setup. The **TBasedxPrintStyle** represents a style that includes special page setup settings, such as margins, orientation, print order, header/footer text, etc.

Hierarchy

```
TObject
  TPersistent
    TComponent
```

TBasedxPrintStyle Properties

In TBasedxPrintStyle

AllowChangeHFText	published / read / write	Index	published / read / write
AllowChangeMargins	published / read / write	IsCurrentStyle	published / read / write
AllowChangeOrientation	published / read / write	PrinterPage	published / read / write
AllowChangeOrientation	published / read / write	ShowPageSetupDlg	published / read / write
AllowChangeScale	published / read / write	State	public / read only
BuiltIn	public / read / write	StyleCaption	published / read / write
Data	public / read / write	StyleGlyph	published / read / write
Description	published / read / write	StyleManager	public / read / write
ImageIndex	published / read / write		

AllowChangeHFText property

read / write

Specifies whether the header and footer text of a report can be changed.

```
property AllowChangeHFText: Boolean;
```

Description

Use the **AllowChangeHFText** property to enable or disable the "Header/Footer" page of the **Page Setup** window. The default value for this property is **True**.

AllowChangeMargins property

read / write

Specifies whether margin settings of a report can be changed.

```
property AllowChangeMargins: Boolean;
```

Description

Use the **AllowChangeMargins** property to enable or disable the "Margins" page of the **Page Setup** window. The default value for this property is **True**.

AllowChangeOrientation property

read / write

Specifies whether report orientation can be changed.

```
property AllowChangeOrientation: Boolean;
```

Description

Use the **AllowChangeOrientation** property to enable or disable the "Orientation" section of the **Page Setup** window. The default value of this property is **True**.

AllowChangePaper property

read / write

Specifies whether report paper type can be changed.

property AllowChangePaper: **Boolean**;

Description

Use the **AllowChangePaper** property to enable or disable the "**Type**" section of the **Page Setup** window which is used to select one of the predefined paper types. The default value for this property is **True**.

AllowCustomPaperSizes property

[read / write](#)

Specifies whether the custom width and height of a report page can be determined.

property AllowCustomPaperSizes: **Boolean**;

Description

Use the **AllowCustomPaperSizes** property to enable or disable the "**Dimension**" section of the **Page Setup** window which is used to determine custom width and height instead of using predefined types. The default value for this property is **True**.

AllowChangeScale property

[read / write](#)

Specifies whether a report can be scaled manually.

property AllowChangeScale: **Boolean**;

Description

Use the **AllowChangeScale** property to enable or disable scaling within the **Page Setup** window. The default value for this property is **True**.

BuiltIn property

[read / write](#)

Specifies whether the current style is built-in.

property BuiltIn: **Boolean**;

Description

The ExpressPrinting System contains built-in print styles that are created at design time. These styles cannot be deleted at runtime. The **BuiltIn** property specifies whether the current style is built-in or created at runtime via the **Copy** function of the "**Define Print Styles**" dialog window.

Data property

[read / write](#)

Contains a pointer to arbitrary data.

property Data: **Pointer**;

Description

Use the **Data** property to associate a print style with corresponding data. This data does not affect report representation. You can use it as your needs dictate.

Description property**read / write**

Determines the text used as a description for the current print style.

property Description: string;

Description

Use the **Description** property to define a unique text string for your specific needs. For instance, you can use this property value to represent a textual description of the current print style within the status bar.

ImageIndex property**read / write**

Specifies the index of the image used to represent the current print style.

property ImageIndex: **Integer**;

Description

If there is an ImageList assigned to the Images property, print styles can use the images from this list. The **ImageIndex** property specifies an appropriate image index for each print style. This image is displayed as an icon for a specific print style within the "Edit Print Styles" window at design time or within the "Define Print Styles" window at runtime.

Index property**read / write**

Determines the index of the current print style among other styles.

property Index: **Integer**;

Description

Each print style has its unique index. Use the **Index** property to determine this index. You can use it to access an appropriate print style. This index is zero-based.

IsCurrentStyle property**read / write**

Specifies whether a given print style is current.

property IsCurrentStyle: **Boolean**;

Description

Though the ExpressPrinting System can have multiple print styles, only one of them can be "active" at a given time. Use the **IsCurrentStyle** property to determine whether a given style is active. The default value for this property is **False**.

PrinterPage property**read / write**

Determines report page properties.

```
property PrinterPage: TdxPrinterPage;
```

Description

Use the **PrinterPage** property to customize report page properties, such as page size, margins, header/footer settings, scaling, etc. For more information see TdxPrinterPage.

ShowPageSetupDlg property

read / write

```
property ShowPageSetupDlg: Boolean;
```

Description

The **ShowPageSetupDlg** property is used at design time only. Press the **ShowPageSetupDlg** button within the **Object Inspector** to call the "Page Setup" dialog window.



State property

read only

Specifies print style state.

type

```
TdxPrintStyleState = (pssCopy);
TdxPrintStyleStates = set of TdxPrintStyleState;
```

```
property State: TdxPrintStyleStates;
```

Description

Use the **State** property to determine which operation is currently being performed on a print style. Possible values include:

Value	Description
pssCopy	Print style is copied via the copy function of the "Define Print Styles" dialog window.

StyleCaption property

read / write

Determines style caption.

```
property StyleCaption: string;
```

Description

Use the **StyleCaption** property to determine the caption used within different dialogs to represent a given style.

StyleGlyph property**read / write**

Specifies an icon for the current style.

property StyleGlyph: TBitmap;

Description

Use the **StyleGlyph** property to specify the image displayed as an icon for a print style within the "**Edit Print Styles**" window at design time or within the "**Define Print Styles**" window at runtime. You can specify this image via the **ImageIndex** property, but the **StyleGlyph** property doesn't require any additional ImageList component.

StyleManager property**read / write**

Specifies the style manager component for the current print style.

property StyleManager: TdxPrintStyleManager;

Description

All print styles belong to corresponding style manager components (TdxPrintStyleManager). Use the **StyleManager** property to specify the style manager for a print style.

TBasedxPrintStyle Methods

In TBasedxPrintStyle

GetFilteredPapers	public	RestoreDefaults	public
PageSetup	public	SetupOptions	public
PageSetupEx	public		

GetFilteredPapers method

Returns paper types that are supported by the default printer.

```
procedure GetFilteredPapers(AStrings: TStrings);
```

Description

The **GetFilteredPapers** method is used to obtain all available paper types for a report. The **GetFilteredPapers** method generates the **OnFilterPaper** event. By writing implementation for this event a developer can filter out which paper types are to be supported. If no handler for this event has been implemented, all the paper types are supported. Supported paper types are displayed in the "Type" section of the **Page Setup** window.

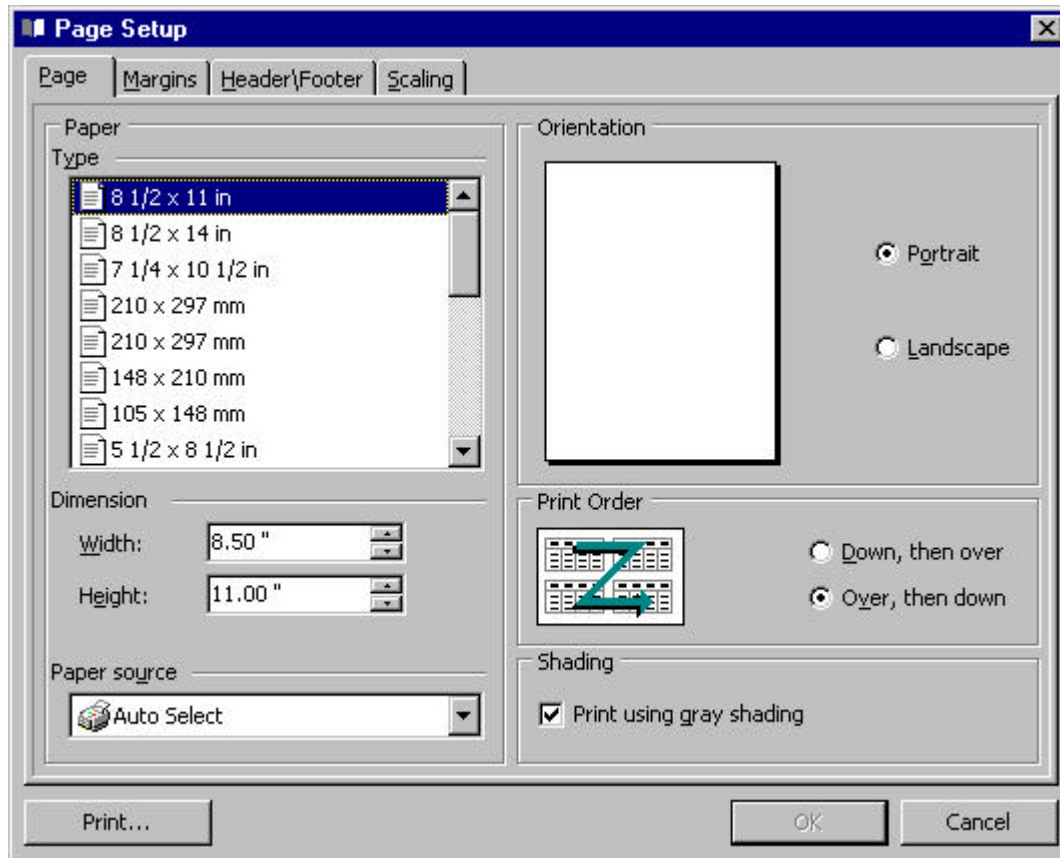
PageSetup method

Displays the "PageSetup" dialog window.

```
function PageSetup: Boolean; {$IFDEF DELPHI4} overload; {$ENDIF}
```

Description

The **PageSetup** function is called to display the "PageSetup" dialog window. Do not use this function directly - use the PageSetup function of TBasedxReportLink instead.



See Also

TBasedxPrintStyle.PageSetupEx

PageSetupEx method

Displays the "PageSetup" dialog window.

```
function PageSetupEx(AActivePageIndex: Integer; APreviewBtnClicked,
APrintBtnClicked: PBoolean): Boolean;
{$IFDEF DELPHI4}
function PageSetup(AActivePageIndex: Integer; APreviewBtnClicked,
APrintBtnClicked: PBoolean): Boolean; overload;
{$ENDIF}
```

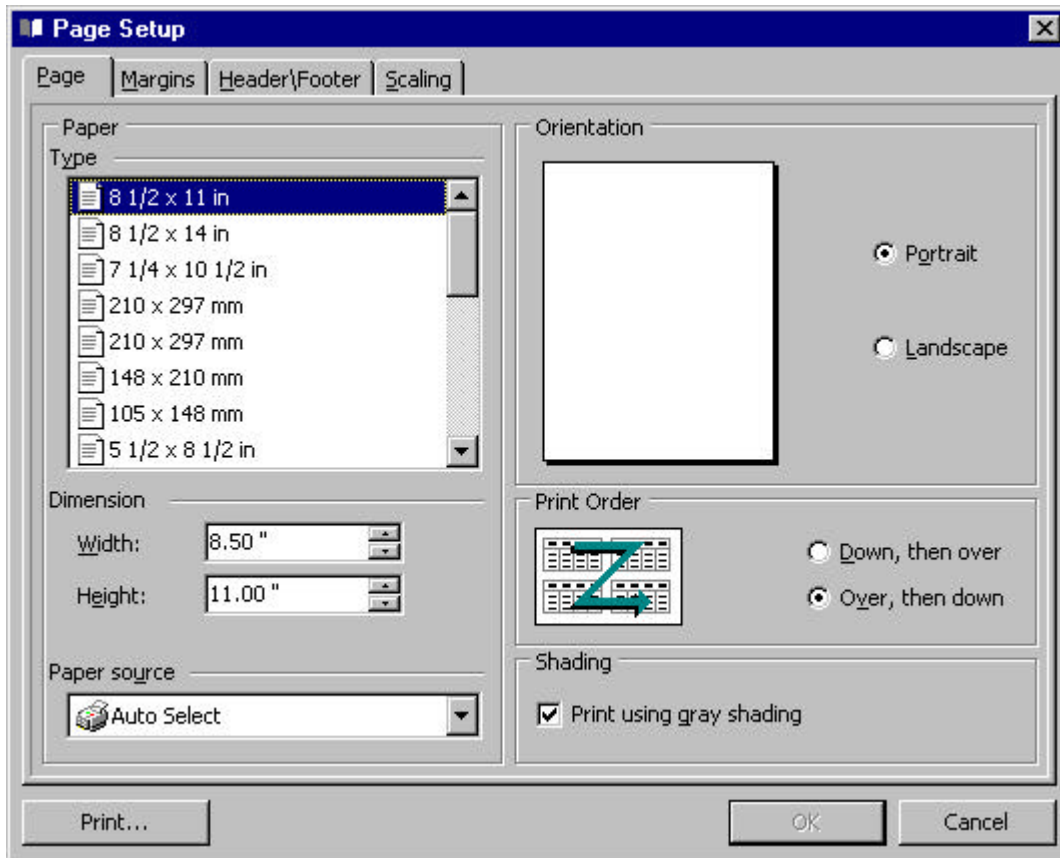
Description

The **PageSetupEx** function is called to display the "PageSetup" dialog window.

The *AActivePageIndex* parameter specifies which page of the "PageSetup" dialog window is displayed by default.

The *APreviewBtnClicked* and *APrintBtnClicked* parameters specify whether to display the "Preview..." and "Print Preview..." buttons in this dialog window. These parameters allow a developer to ascertain which of these buttons was pressed.

Do not use this function directly; use the PageSetupEx function of TBasedxReportLink instead.



Note: If the used parameter is not *nil*, the corresponding button is displayed.

See Also

TBasedxPrintStyle.PageSetup

RestoreDefaults method

Restores default settings.

```
procedure RestoreDefaults;
```

Description

The **RestoreDefaults** method resets print style settings to their predefined values. Use this method to quickly restore factory default values. All changes made to print style properties will be discarded.

SetupOptions method

```
function SetupOptions: Boolean;
```

Description

Call the **SetupOptions** method to activate the setup dialog window if it is registered for a given style.

TBasedxPrintStyle Events

In TBasedxPrintStyle

OnDestroy	published
OnFilterPaper	published

OnDestroy event

Occurs when a print style is being destroyed.

property OnDestroy: TNotifyEvent;

Description

Write an **OnDestroy** event handler to perform specific actions when a print style is being destroyed.

OnFilterPaper event

Occurs when retrieving the list of available paper types.

type

```
TdxFilterPaperEvent = procedure(Sender: TBasedxPrintStyle; const APaper: TdxPaperInfo; var AISupported: Boolean) of object;  
property OnFilterPaper: TdxFilterPaperEvent;
```

Description

Write an **OnFilterPaper** event handler to filter the list of paper types.

The *Sender* parameter specifies the print style for which the list of paper types is retrieved.

The *APaper* parameter is used to acquire paper type information.

The *AISupported* parameter specifies whether to include paper types in the list or not; if this parameter is set to **True**, the *APaper* type is displayed among available paper types.

TdxPSPrintStyle

Provides additional functionality for print styles.

Unit

dxPSCore

```
TdxPSPrintStyle = class(TBasedxPrintStyle)
```

Description

TdxPSPrintStyle is an enhanced version of the **TBasedxPrintStyle** object. Unlike the base print style, this class introduces 4 new events that allow you to extend the functionality of report printing and generation. Additionally, when a **TdxPSPrintStyle** style is created, style caption and glyph are set to predefined values. Page footer region is also populated with predefined settings: page number is placed within the central footer region; date is placed within the right footer region.

Hierarchy

```
Object
  TPersistent
    TComponent
      TBasedxPrintStyle
```

TdxPSPrintStyle Properties

Derived from TBasedxPrintStyle

AllowChangeHFText	published / read / write	Index	published / read / write
AllowChangeMargins	published / read / write	IsCurrentStyle	published / read / write
AllowChangeOrientation	published / read / write	PrinterPage	published / read / write
AllowChangeOrientation	published / read / write	ShowPageSetupDlg	published / read / write
AllowChangeScale	published / read / write	State	public / read only
BuiltIn	public / read / write	StyleCaption	published / read / write
Data	public / read / write	StyleGlyph	published / read / write
Description	published / read / write	StyleManager	public / read / write
ImageIndex	published / read / write		

TdxPSPrintStyle Methods

Derived from TBasedxPrintStyle

GetFilteredPapers	public	RestoreDefaults	public
PageSetup	public	SetupOptions	public
PageSetupEx	public		

TdxPSPrintStyle Events

In TdxPSPrintStyle

OnAfterGenerating	published	OnBeforeGenerating	published
OnAfterPrinting	published	OnBeforePrinting	published

Derived from TBasedxPrintStyle

OnDestroy	published
OnFilterPaper	published

OnAfterGenerating event

Occurs after a report has been generated.

property OnAfterGenerating: TNotifyEvent;

Description

Write an **OnAfterGenerating** event handler to perform specific actions after a report has been generated.

See Also

TdxPSPrintStyle.OnBeforeGenerating

OnAfterPrinting event

Occurs after a report has been printed.

property OnAfterPrinting: TNotifyEvent;

Description

Write an **OnAfterPrinting** event handler to perform specific actions after a report has been printed.

See Also

TdxPSPrintStyle.OnBeforePrinting

OnBeforeGenerating event

Occurs before generating a report.

property OnBeforeGenerating: TNotifyEvent;

Description

Write an **OnBeforeGenerating** event handler to perform specific actions before generating a report.

See Also

TdxPSPrintStyle.OnAfterGenerating

OnBeforePrinting event

Occurs before printing a report.

property OnBeforePrinting: TNotifyEvent;

Description

Write an **OnBeforePrinting** event handler to perform specific actions before printing a report.

See Also

TdxPSPrintStyle.OnAfterPrinting



TdxPrintStyleManager Object

Allows a user to apply different print styles to a report.

Unit

dxPgsDlg

```
TdxPrintStyleManager = class(TComponent)
```

Description

The ExpressPrinting System now incorporates a Style feature. You may now print using a variety of custom setting (styles) within any report. For instance, you can print the ExpressQuantumGrid on different paper formats without any additional page setup. This feature is managed via a new component - **TdxPrintStyleManager**.

Hierarchy

```
Object
  TPersistent
    TComponent
```

TdxPrintStyleManager Properties

In TdxPrintStyleManager

AutoSave	published / read / write	Images	published / read / write
CloneStyleCaptionPrefix	published / read / write	PageSetupDialog	published / read / write
Count	public / read only	StorageName	published / read / write
CurrentStyle	published / read / write	Styles	public / read / write
CurrentStyleIndex	public / read / write	Title	published / read / write
Designer	public / read only	UpdateCount	public / read only
HelpContext	published / read / write	Version	published / read / write

AutoSave property

read / write

Specifies whether to automatically save print style settings to file.

```
property AutoSave: Boolean;
```

Description

Set the **AutoSave** property to **True** if you want to store all settings and changes within a separate file. File name is determined via the StorageName property.

The default value for this property is **False**

See Also

TdxPrintStyleManager.StorageName

CloneStyleCaptionPrefix property

read / write

Determines the prefix for copied print styles.

```
property CloneStyleCaptionPrefix: string;
```

Description

A user can make a copy of a built-in print style at runtime using the 'Define Print Styles' dialog window. Two different print styles cannot use the same name. Use the **CloneStyleCaptionPrefix** property to determine the prefix to add before the name of an ancestor print style. The default value for this property is "Copy (%d) of".

Count property

read only

Contains the number of styles within the style manager component.

```
property Count: Integer;
```

Description

Use the **Count** property to obtain the number of available print styles that belong to the current style manager.

CurrentStyle property**read / write**

Specifies the current print style.

property CurrentStyle: TBasedxPrintStyle;

Description

Style Manager can contain many print styles but only one of them can be active at a given time. Use the **CurrentStyle** property to specify which of the available print styles is called by default when printing or previewing a report.

CurrentStyleIndex property**read / write**

Specifies the index of the current print style.

property CurrentStyleIndex: **Integer**;

Description

Style Manager can contain many print styles but only one of them can be active at a given time. Use the **CurrentStyleIndex** property to specify the index of the print style that is called by default when printing or previewing a report.

Designer property**read only**

Accesses the design time window at design time.

property Designer: TAbstractdxStyleManagerDesigner;

Description

The **Designer** property is used internally and accessible at design time only. This property is used to access the design time window that allows you to manage print styles.

HelpContext property**read / write**

Provides the context number for use when calling context-sensitive online Help.

type

THelpContext = -MaxLongInt..MaxLongInt;

property HelpContext : THelpContext;

Description

Assign a value to the **HelpContext** property to determine which help screen appears when a user presses F1 to request help. Each screen in the Help system should have a unique context number. When a preview window is active, pressing F1 displays the Help screen. The Help topic displayed depends on the **HelpContext** property value.

The default value of this property is **0**.

Images property

[read / write](#)

Determines an image list collection associated with TdxPrintStyleManager.

property Images: TImageList;

Description

If there is an ImageList assigned to the **Images** property, print styles can acquire its images. The **ImageIndex** property specifies the appropriate image index for each print style. This image is displayed as an icon for a print style within the "Edit Print Styles" window at design time or within the "Define Print Styles" window at runtime.

PageSetupDialog property

[read / write](#)

Specifies the page setup dialog component to use.

property PageSetupDialog: TdxPageSetupDialog;

Description

The **TdxPageSetupDialog** component allows you to adjust page setup dialog attributes according to your programming needs. Use the **PageSetupDialog** property to determine which **TdxPageSetupDialog** component the current style manager is connected. For more information see TdxPageSetupDialog.

StorageName property

[read / write](#)

Determines the file name used to store print style information.

property StorageName: string;

Description

You can store all settings and changes made to print styles within a separate file. Use the **StorageName** to specify this file name. This property is in effect only if the **AutoSave** property is set to **True**.

Styles property

[read / write](#)

Specifies a print style via its index.

property Styles[Index: Integer]: TBasedxPrintStyle;

Description

Use the **Styles** property to access a print style.

The *Index* parameter specifies the zero-based index of this print style among others.

The total number of styles is determined by the **Count** property.

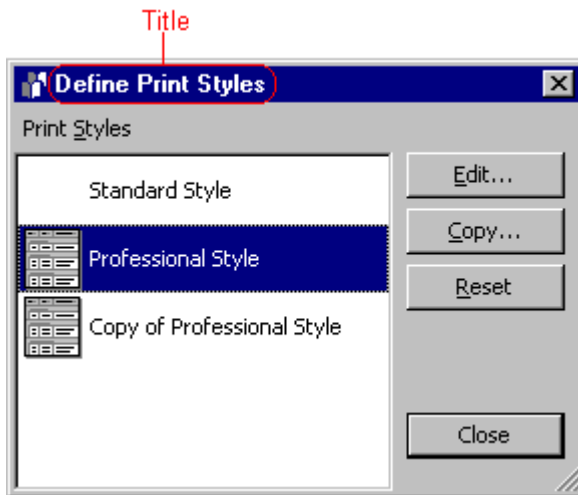
Title property**read / write**

Determines the title of the style manager dialog window.

property Title: string;

Description

A user can work with print styles at runtime via the appropriate dialog window. Use the **Title** property to determine the caption of this window.



UpdateCount property**read only**

Stands for an internal counter.

property UpdateCount: **Integer**;

Description

This property is for internal use only. It is used as a counter for the BeginUpdate and EndUpdate methods.

See Also

TdxPrintStyleManager.BeginUpdate; TdxPrintStyleManager.EndUpdate

Version property**read / write**

The **Version** property is used to controll changes applied to a form.

property Version: **Integer**;

Description

When a style manager is being created it reads version number from a file (if it exists, of course) and the form's dfm file.

If the Version number retrieved from the file is not equal to that obtained from the dfm file, the dfm file's data is used to restore styles and when the application is closed the stream is re-written with the current style data.

Otherwise the style data is derived from the file.

So, if a developer leaves the Version property unchanged all the changes to the form he has probably made during the current design-time session will be lost because they will be read from the file.

If a developer changes the Version property at design-time the style data is derived from the dfm file.

However, this property is mainly used to correctly upgrade to a newer version of the same product, when it is absolutely indispensable to avoid using data stored in the old dfm file.

TdxPrintStyleManager Methods

In TdxPrintStyleManager

AddStyle	public	EndUpdate	public
AddStyleEx	public	IndexOfStyle	public
AssignStyles	public	LoadFromFile	public
BeginClone	public	LoadFromStream	public
BeginUpdate	public	NonBuiltInsExists	public
Clear	public	RestoreDefaults	public
DefinePrintStylesDlg	public	SaveToFile	public
Delete	public	SaveToStream	public
DeleteNonBuiltIns	public	StyleByCaption	public
EndClone	public	StyleByName	public

AddStyle method

Adds a new style to the list of available print styles.

type

TdxPrintStyleClass = class of TBasedxPrintStyle;

function AddStyle(*AStyleClass*: TdxPrintStyleClass): TBasedxPrintStyle;

Description

Call the **AddStyle** method to add a new print style. The **AStyleClass** parameter specifies which predefined style to use for the creation of a new style.

AddStyleEx method

Adds a new style to the list of available print styles.

type

TdxPrintStyleClass = class of TBasedxPrintStyle;

function AddStyleEx(*AStyleClass*: TdxPrintStyleClass; *AOwner*: TComponent): TBasedxPrintStyle;

Description

Call the **AddStyle** method to add a new print style. The *AStyleClass* parameter specifies which predefined style to use as the basis of the new style. The *AOwner* allows you to change the owner of this style. By default, it is the same as in the style manager.

AssignStyles method

Copies print styles.

procedure AssignStyles(*Source*: TdxPrintStyleManager);

Description

The **AssignStyles** method deletes all print styles and then copies all report links contained in the TdxPrintStyleManager object specified by the *Source* parameter.

Clear method

Deletes all print styles within the current style manager.

```
procedure Clear;
```

Description

Call the **Clear** method to destroy all previously created print styles.

Delete method

Deletes a specified print style.

```
procedure Delete(Index: Integer);
```

Description

Call the **Delete** method to remove the print style specified by the zero-based *Index* parameter from the list of available print styles.

DeleteNonBuiltIns method

Deletes all not built-in print styles.

```
procedure DeleteNonBuiltIns;
```

Description

The ExpressPrinting System includes built-in print styles created at design time. These styles cannot be deleted at runtime. The **BuiltIn** property specifies whether the current style is built-in or created at runtime via the copy function of the "Define Print Styles" dialog window. Call the **DeleteNonBuiltIns** method to remove all print styles created at runtime.

IndexOfStyle method

Specifies the index of a specified print style.

```
function IndexOfStyle(Value: TBasedxPrintStyle): Integer;
```

Description

Call the **IndexOfStyle** method to obtain the zero-based index of the print style specified by the *Value* parameter.

NonBuiltInsExists method

Specifies whether non built-in print styles exist within the current style manager.

```
function NonBuiltInsExists: Boolean;
```

Description

The ExpressPrinting System has built-in print styles created at design time. These styles cannot be deleted at runtime. The **BuiltIn** property specifies whether the current style is built-in or created at runtime via the copy function of the "Define Print Styles" dialog window. Call the **NonBuiltInsExists** method to determine whether all print styles are built-in. This method returns **True** if there are no built-in print styles; Otherwise - **False**.

StyleByCaption method

Returns a print style by its caption.

```
function StyleByCaption(const ACaption: string): TBasedxPrintStyle;
```

Description

Call the **StyleByCaption** method to access the print style whose caption is equal to the *ACaption* parameter.

StyleByName method

Returns a print style by its name.

```
function StyleByName(const AName: string): TBasedxPrintStyle;
```

Description

Call the **StyleByName** method to access a print style whose name is equal to the *AName* parameter.

BeginClone method

Starts the copying of a print style.

```
function BeginClone(AIndex: Integer): TBasedxPrintStyle;
```

Description

Call the **BeginClone** method to start copying a print style with a zero-based index specified by the *AIndex* parameter. This function returns a new print style with the same properties as the initial print style.

Note: If you use the **BeginClone** method manually, you must also use the **EndClone** method.

See Also

TdxPrintStyleManager.EndClone

EndClone method

Completes the copying of a print style.

```
procedure EndClone(AStyle: TBasedxPrintStyle);
```

Description

Call the **EndClone** method to complete the copying of a print style. The *AStyle* parameter specifies the newly created print style. Calling the **EndClone** method activates this print style.

Note: If you use the **BeginClone** method manually, you must also use the **EndClone** method.

See Also

TdxPrintStyleManager.BeginClone

BeginUpdate method

Disables events generation.

```
procedure BeginUpdate;
```

Description

The **BeginUpdate** method is called before changing the list of available print styles. It prevents generation of the **OnChangeCurrentStyle** and **OnStyleListChanged** events. Make certain to call the **EndUpdate** method after calling the **BeginUpdate** method. Every call of the **BeginUpdate** method increases the internal counter (the **UpdateCount** property value) by one. Every call of the **EndUpdate** method decreases it by one. The above events can be fired only when the internal counter is 0.

See Also

TdxPrintStyleManager.BeginUpdate; TdxPrintStyleManager.UpdateCount

EndUpdate method

Enables events generation.

```
procedure EndUpdate;
```

Description

The **EndUpdate** method is called after the list of available print styles has been changed. It enables generation of the **OnChangeCurrentStyle** and **OnStyleListChanged** events. Make certain to call the **EndUpdate** method after calling the **BeginUpdate** method. Every call of the **BeginUpdate** method increases the internal counter (the **UpdateCount** property value) by one. Every call of the **EndUpdate** method decreases it by one. The above events can be fired only when the internal counter is 0.

See Also

TdxPrintStyleManager.EndUpdate; TdxPrintStyleManager.UpdateCount

DefinePrintStylesDlg method

Displays the "Define Print Styles" dialog window.

```
procedure DefinePrintStylesDlg(APreviewBtnClicked, APrintBtnClicked:  
PBoolean);
```

Description

The **DefinePrintStylesDlg** method is called to display the "Define Print Styles" dialog window. This dialog allows a user to edit print styles. When a user presses the "Edit..." button, the "Page Setup..." dialog window is displayed.

The *APreviewBtnClicked* and *APrintBtnClicked* parameters specify whether to display the "Preview..." and "Print Preview..." buttons in this dialog window. These parameters allow a developer to determine which of these buttons was pressed.

Note: If the appropriate parameter is not *nil*, the corresponding button is displayed.

RestoreDefaults method

Loads default settings for print styles.

```
procedure RestoreDefaults;
```

Description

Use the **RestoreDefaults** property to load default settings for every print style belonging to the current style manager.

LoadFromFile method

Loads all style manager settings from a file.

```
procedure LoadFromFile(const AName: string);
```

Description

Call the **LoadFromFile** method to retrieve component settings from the file specified by the *AName* parameter, and load them into a style manager. The *AName* parameter completely defines the file name. No extension is added to this parameter.

See Also

TdxPrintStyleManager.LoadFromStream; *TdxPrintStyleManager.SaveToFile*;
TdxPrintStyleManager.SaveToStream

LoadFromStream method

Reads all style manager settings from a stream and stores the contents within TdxPrintStyleManager.

```
procedure LoadFromStream(AStream: TStream);
```

Description

Call the **LoadFromStream** method to retrieve component settings from the stream specified by the *AStream* parameter, and load them into a style manager.

Note: The stream must contain data in **TdxPrintStyleManager** internal format (for instance, data is written to a stream by another **TdxPrintStyleManager** via the **SaveToStream** method).

See Also

*TdxPrintStyleManager.LoadFromFile; dxPrintStyleManager.SaveToFile;
dxPrintStyleManager.SaveToStream*

SaveToFile method

Saves all style manager settings to a file.

```
procedure SaveToFile(const AName: string);
```

Description

Call the **SaveToFile** method to save all **TdxPrintStyleManager** attributes to a file specified by the *AName* parameter. These attributes can later be reloaded from this file into another style manager using the **LoadFromFile** method. The *AName* parameter completely defines the file name. No extension is added to this parameter.

See Also

*TdxPrintStyleManager.LoadFromStream; TdxPrintStyleManager.LoadFromFile;
TdxPrintStyleManager.SaveToStream*

SaveToStream method

Writes all style manager settings to a stream passed as a *Stream* parameter.

```
procedure SaveToStream(AStream: TStream);
```

Description

Use **SaveToStream** to stream out **TdxPrintStyleManager** settings. It can be streamed back to another **TdxPrintStyleManager** object using the **LoadFromStream** method.

See Also

*TdxPrintStyleManager.LoadFromStream; TdxPrintStyleManager.SaveToFile;
TdxPrintStyleManager.LoadFromFile*

TdxPrintStyleManager Events

In TdxPrintStyleManager

OnChangeCurrentStyle	published
OnStyleListChanged	published

OnChangeCurrentStyle event

Occurs when the current print style of a style manager is about to change.

property OnChangeCurrentStyle: TNotifyEvent

Description

Write an **OnChangeCurrentStyle** event handler to perform custom actions when changing the current style.

Note: This event does not occur if the **UpdateCount** property is not equal to 0. For more information see the *BeginUpdate* and *EndUpdate* methods.

OnStyleListChanged event

Occurs when updating the list of available print styles.

property OnStyleListChanged: TNotifyEvent

Description

Write an **OnStyleListChanged** event handler to perform custom actions when updating the list of available print styles. For instance, this event occurs when a new style is added to the style manager.

Note: This event does not occur if the **UpdateCount** property is not equal to 0. For more information see the *BeginUpdate* and *EndUpdate* methods.



TdxPageSetupDialog Object

Allows a user to customize the "Page Setup" dialog window.

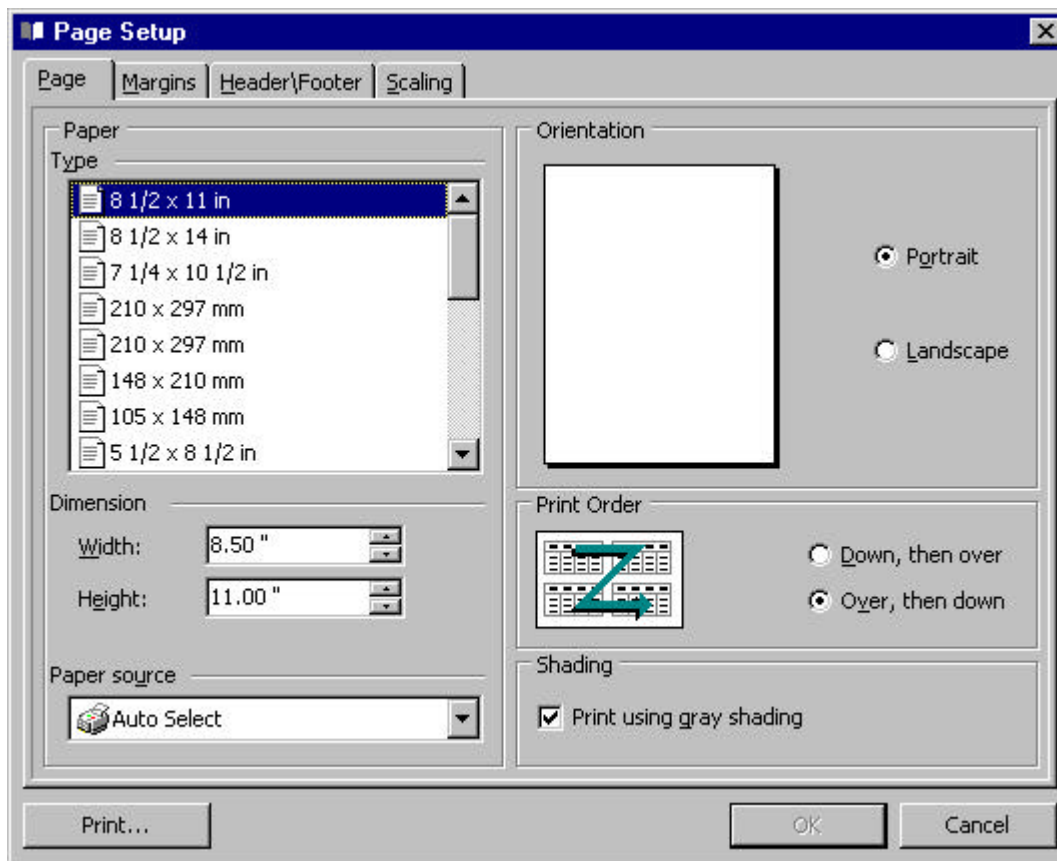
Unit

dxPgsDlg

```
TdxPageSetupDialog = class(TComponent)
```

Description

The **TdxPageSetupDialog** component allows you to adjust the page setup dialog window as your needs dictate. All page setup options can be disabled or hidden, if necessary.



Hierarchy

```

TObject
  TPersistent
    TComponent

```

TdxPageSetupDialog Properties

In TdxPageSetupDialog

ActivePageIndex	published / read / write	OptionsEnabled	published / read / write
ButtonsEnabled	published / read / write	OptionsVisible	published / read / write
ButtonsVisible	published / read / write	PrintStyle	published / read / write
HelpContext	published / read / write	Title	published / read / write
HfMode	published / read / write		

ActivePageIndex property

read / write

Specifies which page of the "PageSetup" dialog window is displayed.

property ActivePageIndex: Integer;

Description

Use the **ActivePageIndex** property to specify the setup page. A value of 0 corresponds to the **Page** tab, a value of 1 corresponds to the **Margins** tab, a value of 2 corresponds to the **Header/Footer** tab, and a value of 3 corresponds to the **Scaling** tab.

ButtonsEnabled property

read / write

Specifies which buttons are enabled in a Print dialog.

type

TdxPrintDlgButtonKind = (pdbPrinterProperties, pdbNetwork, pdbPreview, pdbPageSetup, pdbDefineStyles, pdbStyleOptions, pdbHelp);

TdxPrintDlgButtons = set of TdxPrintDlgButtonKind;

property ButtonsVisible: TdxPrintDlgButtons;

Description

Use the **ButtonsVisible** property to specify which buttons are visible in a Print dialog.

Value	Description
pdbPrinterProperties	Displays a Printer Properties button in the dialog.
pdbNetwork	Displays a Network button in the dialog.
pdbPreview	Displays a Preview button in the dialog.
pdbPageSetup	Displays a Page Setup button in the dialog.
pdbDefineStyles	Displays a Define Styles button in the dialog.
pdbStyleOptions	Displays a Style Options button in the dialog.
pdbHelp	Displays a Help button in the dialog.

The default value for this property is [**pdbPrinterProperties**, **pdbNetwork**, **pdbPreview**, **pdbPageSetup**, **pdbDefineStyles**, **pdbStyleOptions**].

See Also*TdxPageSetupDialog.ButtonsVisible***OptionsEnabled property****read / write**

 Specifies which options within the "**PageSetup**" dialog window are enabled.
type

```
psoAll = [Low(TdxPageSetupDlgOption)..High(TdxPageSetupDlgOption)];
psoDefaultOptionsEnabled = PSOAll;

TdxPageSetupDlgOption = (psoCenterOnPage, PSOMargins, PSOPageOrder,
psoShading, PSOStyleCaption, PSOHFAutoText, PSOHFBackground, PSOHFFont,
psoHFText, PSOHFFunctions, PSOHFMargins, PSOHFReverse, PSOHFVertAlignment);

TdxPageSetupDlgOptions = set of TdxPageSetupDlgOption;
property OptionsEnabled: TdxPageSetupDlgOptions;
```

Description

Use the **OptionsEnabled** property to specify which options are enabled and can be changed. Possible values include:

Value	Description
psoCenterOnPage	Specifies whether the "center on page" option is enabled.
psoMargins	Specifies whether page margins can be changed.
psoPageOrder	Specifies whether print order can be changed.
psoShading	Specifies whether the "shading" option is enabled.
psoStyleCaption	Specifies whether style name can be changed.
psoHFAutoText	Reserved for future use.
psoHFBackground	Specifies whether headers/footer background is enabled.
psoHFFont	Specifies whether the font dialog for headers/footers is enabled.
psoHFText	Specifies whether headers/footer text can be changed.
psoHFFunctions	Specifies whether headers/footer special insert functions are enabled.
psoHFMargins	Specifies whether headers/footer margins can be changed.
psoHFReverse	Specifies whether headers/footer text can be reversed on odd/even pages.
psoHFVertAlignment	Specifies whether the headers/footer vertical alignment option is enabled.

The default value for this property is

[Low(TdxPageSetupDlgOption)..High(TdxPageSetupDlgOption)].

See Also*TdxPageSetupDialog.OptionsVisible*

HFMode property**read / write**

Specifies the manner in which header and footers are displayed.

type

```
TdxHFMode = (hfmThreeSections, hfmOneSection);
```

```
property HFMode: TdxHFMode;
```

Description

Use the **HFMode** property to specify whether to divide headers or footers into 3 sections or to display them as a single region. Possible values include:

Value	Description
hfmThreeSections	Headers (footers) are displayed within 3 separate sections.
hfmOneSection	Headers (footers) are displayed within one section.

The default value for this property is **hfmThreeSections**.

HelpContext property**read / write**

Provides the context number to use when calling the context-sensitive online Help.

```
property HelpContext: THelpContext;
```

Description

Assign a value to the **HelpContext** property to determine which help topic appears when a user presses F1 to request help.

PrintStyle property**read / write**

Specifies the print style used to store information.

```
property PrintStyle: TBasedxPrintStyle;
```

Description

The Page setup dialog must store information about its settings. Use the **PrintStyle** property to specify a print style into which page setup can store its internal information.

Title property**read / write**

Determines the title of the page setup dialog window.

```
property Title: string;
```

Description

A user can adjust page settings via a specific dialog window. Use the **Title** property to determine the initial window caption. When it is displayed at runtime, the appropriate print style caption is added to the initial caption.

ButtonsVisible property

[read / write](#)

Specifies which buttons within the "**PageSetup**" dialog window are displayed.

type

```
TdxPageSetupDlgButtonKind = (psbHelp, psbStyleOptions, psbPreview,
psbPrint);
```

```
TdxPageSetupDlgButtons = set of TdxPageSetupDlgButtonKind;
```

property ButtonsVisible: TdxPageSetupDlgButtons;

Description

Use the **ButtonsVisible** property to specify which buttons are displayed and can be enabled or disabled. Possible values include:

Value	Description
psbHelp	Specifies whether the " Help " button is displayed.
psbStyleOptions	Specifies whether the " Options... " button is displayed. This button is displayed only when there are user registered dialog windows.
psbPreview	Specifies whether the " Print Preview... " button is displayed.
psbPrint	Specifies whether the " Print... " button is displayed.

The default value for this property is [**psbStyleOptions**, **psbPreview**, **psbPrint**].

See Also

TdxPageSetupDialog.ButtonsEnabled

OptionsVisible property

[read / write](#)

Specifies which options within the "**PageSetup**" dialog window are displayed.

type

```
psoAll = [Low(TdxPageSetupDlgOption)..High(TdxPageSetupDlgOption)];
```

```
psoDefaultOptionsVisible = psoAll;
```

```
TdxPageSetupDlgOption = (psoCenterOnPage, psoMargins, psoPageOrder,
psoShading, psoStyleCaption, psoHFAutoText, psoHFBackground, psoHFFont,
psoHFText, psoHFFunctions, psoHFMargins, psoHFReverse, psoHFVertAlignment);
```

```
TdxPageSetupDlgOptions = set of TdxPageSetupDlgOption;
```

property OptionsVisible: TdxPageSetupDlgOptions;

Description

Use the **OptionsEnabled** property to specify which options are enabled and can be changed. Possible values include:

Value	Description
psoCenterOnPage	Specifies whether the "center on page" option is enabled.
psoMargins	Specifies whether page margins can be changed.
psoPageOrder	Specifies whether print order can be changed.
psoShading	Specifies whether the "shading" option is enabled.
psoStyleCaption	Specifies whether a style name can be changed.
psoHFAutoText	Reserved for future use.
psoHFBackground	Specifies whether the headers/footer background is enabled.
psoHFFont	Specifies whether the font dialog for headers/footers is enabled.
psoHFText	Specifies whether headers/footer text can be changed.
psoHFFunctions	Specifies whether the headers/footer special insert functions are enabled.
psoHFMargins	Specifies whether headers/footer margins can be changed.
psoHFReverse	Specifies whether headers/footer text can be reversed on odd/even pages.
psoHFVertAlignment	Specifies whether the headers/footer vertical alignment option is enabled.

The default value for this property is **psoDefaultOptionsVisible**.

See Also

TdxPageSetupDialog.OptionsEnabled

TdxPageSetupDialog Methods

In TdxPageSetupDialog

Execute	public
RealTitle	public

Execute method method

Activates the "**Page Setup**" dialog window.

```
function Execute: Boolean;
```

Description

Call the **Execute** method to display the "**Page Setup**" dialog window at runtime.

RealTitle method

Returns the title of the "**Page Setup**" dialog window.

```
function RealTitle: string;
```

Description

The **Title** property determines this initial window caption. When this window is displayed at runtime, the appropriate print style caption is added to the initial caption. Call the **RealTitle** method to obtain the final caption.

TdxPageSetupDialog Events

In TdxPageSetupDialog

OnClose	published	OnShow	published
OnCustomDrawPreview	published		

OnClose event

Occurs when closing the "**Page Setup**" dialog window.

property OnClose: TNotifyEvent;

Description

Write an **OnClose** event handler to perform custom actions when the "**Page Setup**" dialog window is about to close.

OnCustomDrawPreview event

Occurs when the "**Margins**" tab of the "**Page Setup**" dialog window is displayed.

type

```
TdxCustomDrawPreviewEvent = procedure(APrintStyle: TBasedxPrintStyle;
ACanvas: TCanvas; APageRect, AContentRect, AHeaderRect, AFooterRect: TRect)
of object;
```

property OnCustomDrawPreview: TdxCustomDrawPreviewEvent;

Description

Write an **OnCustomDrawPreview** event handler to draw a specific preview within the '**Preview**' section of the "**Margins**" tab. Use this event to display your own preview picture for your report. The *ACanvas* parameter determines the drawing canvas. The *APageRect*, *AContentRect*, *AHeaderRect*, and *AFooterRect* parameters specify the bound rectangle for page, content, header and footer drawing.

Keep in mind that the **OnCustomDrawPreview** event handler must be a full drawing procedure.

OnShow event

Occurs when displaying the "**Page Setup**" dialog window.

property OnShow: TNotifyEvent;

Description

Write an **OnShow** event handler to perform custom actions when activating the "**Page Setup**" dialog window.

TAbstractdxStyleManagerDesigner

Controls the lifetime of the style manager designer window.

Unit

dxPgsDlg

```
TAbstractdxStyleManagerDesigner = class;
```

Description

The **TAbstractdxStyleManagerDesigner** object controls the lifetime of the style manager designer window and is used at design time only.



TdxPrintDialog Object

Defines printer dialog attributes.

Unit

dxPrnDlg

TdxPrintDialog = **class** (TComponent)

Description

TdxPrintDialog objects are used to customize numerous printer dialog attributes such as PageNums, PrinterStyle, etc.

Hierarchy

TObject

TPersistent

TComponent

TdxPrintDialog Properties

In TdxPrintDialog

ButtonsEnabled	published / read / write	OptionsVisible	published / read / write
ButtonsVisible	published / read / write	PageCount	public / read / write
Collate	published / read / write	PageNums	published / read / write
Copies	published / read / write	PageRanges	published / read / write
DialogData	public / read only	Pages	published / read / write
FileList	public / read / write	PreviewBtnClicked	public / read only
FileName	published / read / write	PrintBtnClicked	public / read only
HelpContext	published / read / write	PrintToFile	published / read / write
MaxRange	published / read / write	StyleManager	published / read / write
MinRange	published / read / write	Title	published / read / write
OptionsEnabled	published / read / write	UseFileList	published / read / write

ButtonsEnabled property

read / write

Specifies which buttons are enabled in a printer dialog.

type

```
TdxPrintDlgButtonKind = (pdbPrinterProperties, pdbNetwork, pdbPreview,
  pdbPageSetup, pdbDefineStyles, pdbStyleOptions, pdbHelp);
```

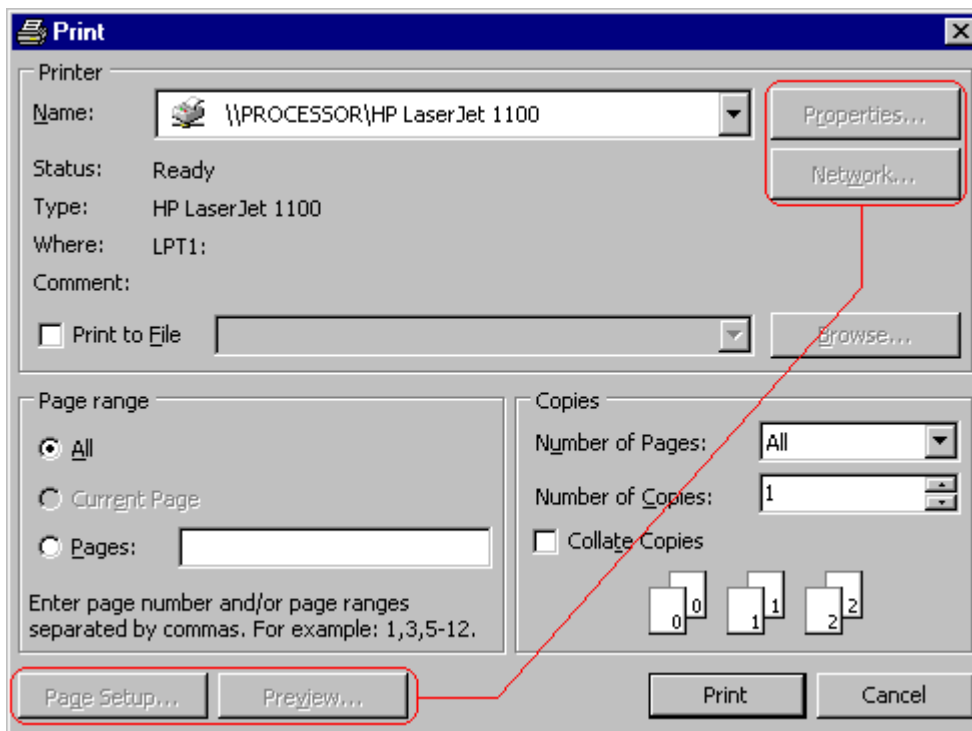
```
TdxPrintDlgButtons = set of TdxPrintDlgButtonKind;
```

```
property ButtonsEnabled: TdxPrintDlgButtons;
```

Description

Use the **ButtonsEnabled** property to specify which buttons are enabled in a printer dialog.

Value	Description
pdbPrinterProperties	Properties button.
pdbNetwork	Network button.
pdbPreview	Preview button.
pdbPageSetup	Page Setup button.
pdbDefineStyles	Define Styles button.
pdbStyleOptions	Style Options button.
pdbHelp	Help button.



The default value for this property is **[pdbPrinterProperties, pdbNetwork, pdbPreview, pdbPageSetup, pdbDefineStyles, pdbStyleOptions]**.

See Also

TdxPrintDialog.ButtonsVisible; dxPrintDialog.OptionsEnabled; dxPrintDialog.OptionsVisible

ButtonsVisible property

read / write

Specifies which buttons are visible in a printer dialog.

type

```
TdxPrintDlgButtonKind = (pdbPrinterProperties, pdbNetwork, pdbPreview,
pdbPageSetup, pdbDefineStyles, pdbStyleOptions, pdbHelp);
```

```
TdxPrintDlgButtons = set of TdxPrintDlgButtonKind;
```

```
property ButtonsVisible: TdxPrintDlgButtons;
```

Description

Use the **ButtonsVisible** property to specify which buttons are visible in a printer dialog.

Value	Description
pdbPrinterProperties	Printer Properties button.
pdbNetwork	Network button.
pdbPreview	Preview button.
pdbPageSetup	Page Setup button.
pdbDefineStyles	Define Styles button.
pdbStyleOptions	Style Options button.

pdbHelp

Help button.

The default value for this property is [**pdbPrinterProperties**, **pdbNetwork**, **pdbPreview**, **dbPageSetup**, **pdbDefineStyles**, **pdbStyleOptions**].

See Also

TdxPrintDialog.ButtonsEnabled; dxPrintDialog.OptionsEnabled; dxPrintDialog.OptionsVisible

Collate property

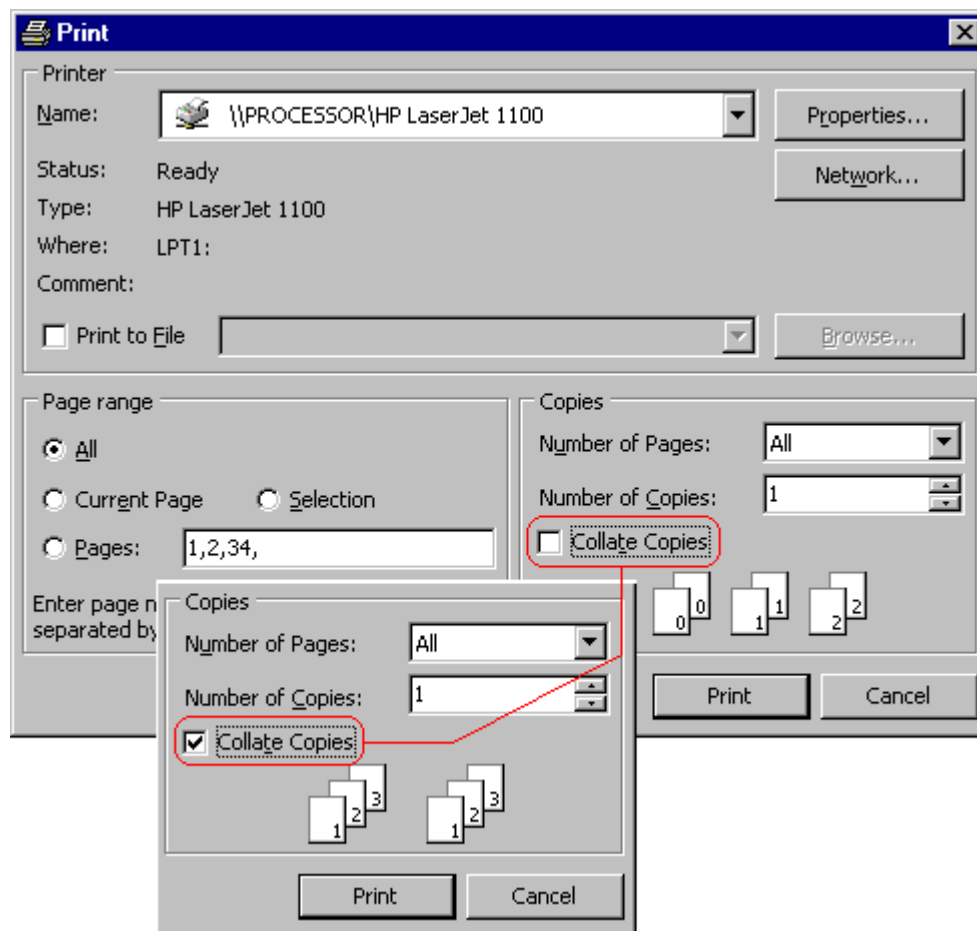
[read / write](#)

Determines whether to collate report copies.

property Collate: **Boolean**;

Description

Use the **Collate** property when printing multiple report copies to specify whether they should be collated. Set the **Collate** property to **True** to print all report pages before printing the next copy.



The default value for this property is **False**.

See Also

TdxPrintDialog.Copies

Copies property

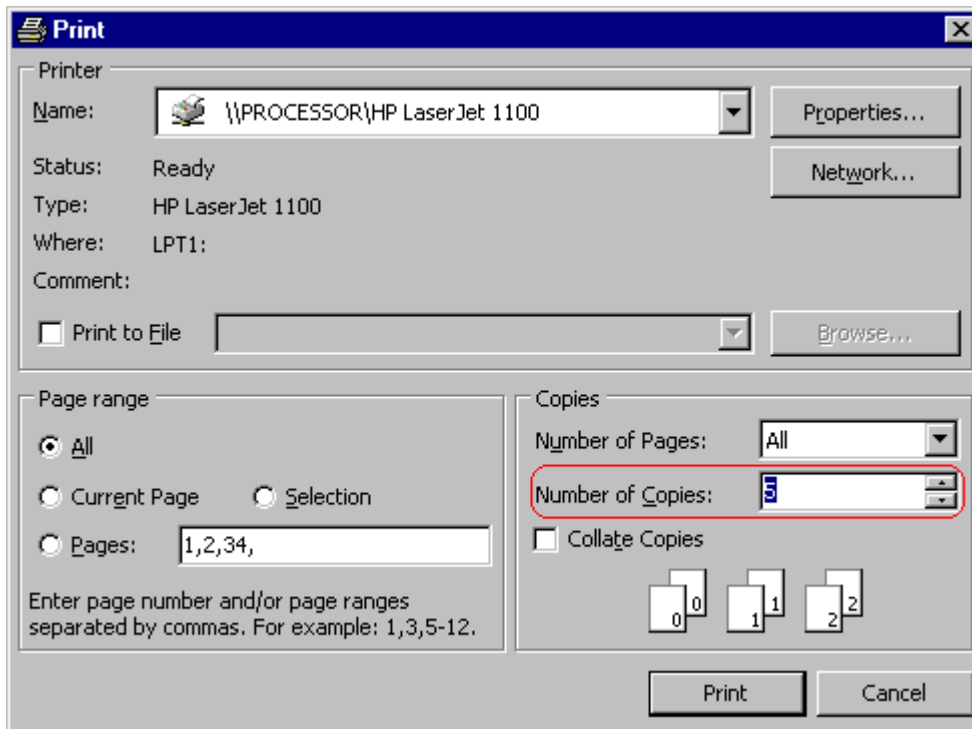
read / write

Determines the number of report copies to print.

property Copies: **Integer**;

Description

Use the **Copies** property to determine the number of copies to print.



The default value for this property is **1**.

See Also

TdxPrintDialog.Copies

DialogData property

read only

Specifies printer dialog attributes.

type

TdxPrintDialogData = packed record

```

    Copies: Integer;
    Collate: Boolean;
    FileList: TStrings;
    FileName: string;
    MaxRange: Integer;

```

```

    MinRange: Integer;
    PageCount: Integer;
    PageNums: TdxPageNumbers;
    PageRanges: TdxPageRanges;
    Pages: string;
    PrintToFile: Boolean;
    StyleManager: TdxPrintStyleManager;
end;
property DialogData: TdxPrintDialogData;

```

Description

Use this property to obtain **DialogData** options.

FileList property

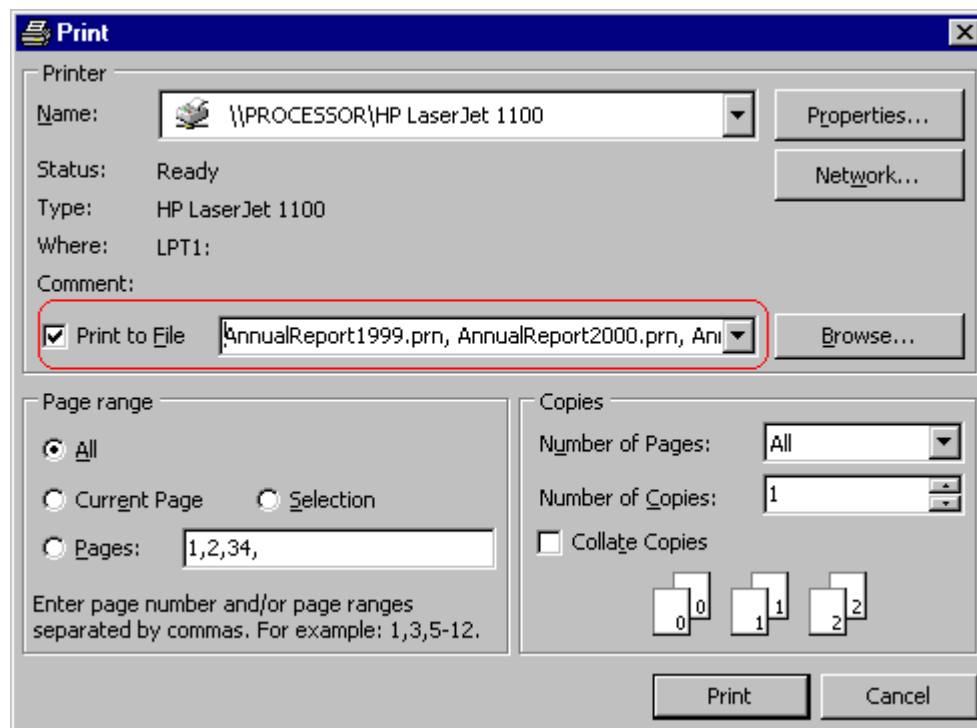
[read / write](#)

Specifies the list of file names to which a report may be printed.

```
property FileList: TStrings;
```

Description

The **FileList** property determines the list of files to which a report has already been printed. This list is displayed in the **"Print"** dialog window within the combo box of the "print to file" option. When a user specifies his/her own file, this file is added to this list. Later a user may print any of these files out.



See Also

TdxPrintDialog.FileName; TdxPrintDialog.UseFileList

FileName property

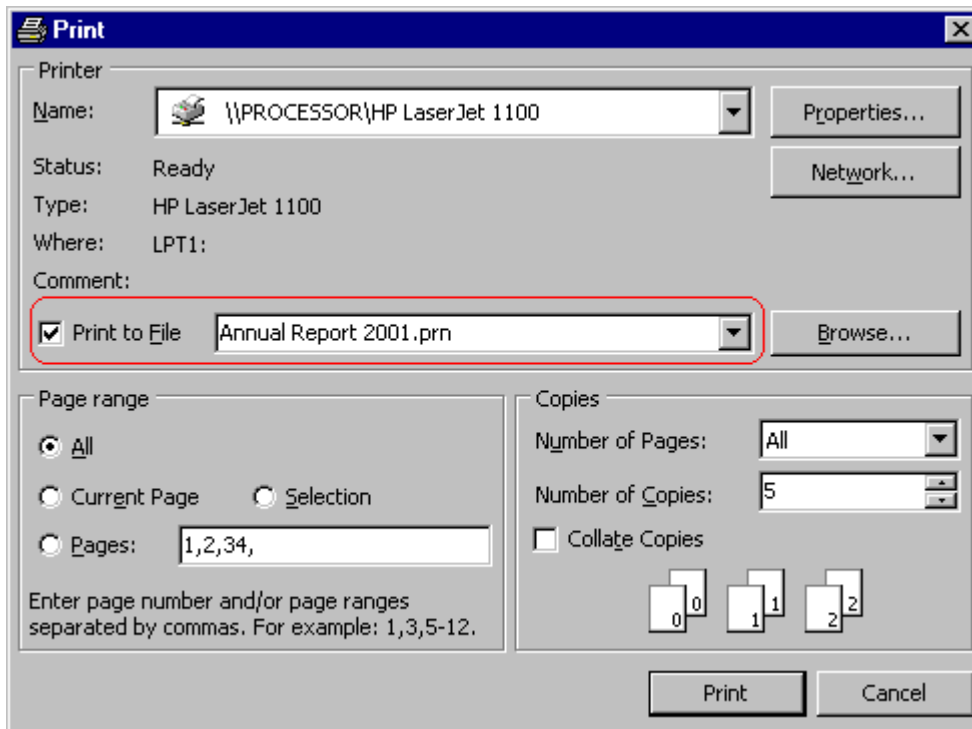
read / write

Specifies the file name wherein a report is printed.

property FileName: string;

Description

Use the **FileName** property to specify the file name wherein to print a report.



See Also

TdxPrintDialog.FileList; TdxPrintDialog.UseFileList

HelpContext property

read / write

Provides the context number used when calling context-sensitive online Help.

property HelpContext: THelpContext;

Description

Assign a value to the **HelpContext** property to determine which help topic appears when a user presses F1 to request help.

MaxRange property

read / write

Determines the last page in a range of pages to print.

property MaxRange: **Integer**;

Description

Use the **MaxRange** property to determine the last page in a range of pages to print.

The default value for this property is **1**.

See Also

TdxPrintDialog.MinRange

MinRange property

[read / write](#)

Determines the first page in a range of pages to print.

property MinRange: **Integer**;

Description

Use the **MinRange** property to determine the first page in a range of pages to print.

The default value for this property is **1**.

See Also

TdxPrintDialog.MaxRange

OptionsEnabled property

[read / write](#)

Specifies which options are enabled in a printer dialog.

type

```
TdxPrintDlgOption = (pdoPrintToFile, pdoAllPages, pdoCurrentPage,
pdoSelection, pdoPageRange, pdoPrintStyles);
```

```
TdxPrintDlgOptions = set of TdxPrintDlgOption;
```

property OptionsEnabled: TdxPrintDlgOptions;

Description

Use the **OptionsEnabled** property to specify which options are enabled. Disabled items are not displayed and therefore inaccessible.

Value	Description
pdoPrintToFile	Print To File option.
pdoAllPages	All Pages option.
pdoCurrentPage	Current Page option.
pdoSelection	Selection option.
pdoPageRange	Page Range option.
pdoPrintStyles	Print Styles option.

The default value for this property is [**pdoPrintToFile**, **pdoAllPages**, **pdoPageRange**].

See Also

TdxPrintDialog.ButtonsEnabled; TdxPrintDialog.ButtonsVisible

OptionsVisible property

[read / write](#)

Specifies visible option items in a printer dialog.

type

```
TdxPrintDlgOption = (pdoPrintToFile, pdoAllPages, pdoCurrentPage,  
pdoSelection, pdoPageRange, pdoPrintStyles);
```

```
TdxPrintDlgOptions = set of TdxPrintDlgOption;
```

property OptionsVisible: TdxPrintDlgOptions;

Description

Use the **OptionsVisible** property to specify which option items are visible in a printer dialog.

Value	Description
pdoPrintToFile	Print To file option.
pdoAllPages	All Pages option.
pdoCurrentPage	Current Page option.
pdoSelection	Selection option.
pdoPageRange	Page Range option.
pdoPrintStyles	Print Styles option.

The default value for this property is [**pdoPrintToFile**, **pdoAllPages**, **pdoCurrentPage**, **pdoPageRange**].

See Also

TdxPrintDialog.ButtonsEnabled; TdxPrintDialog.ButtonsVisible; TdxPrintDialog.OptionsEnabled

PageCount property

[read / write](#)

Determines the number of report pages in a report.

property PageCount: **Integer**;

Description

Use the **PageCount** property to obtain the total number of pages in a report.

See Also

*TdxPrintDialog.MaxRange; TdxPrintDialog.MinRange; TdxPrintDialog.PageRanges;
TdxPrintDialog.Pages*

PageNums property

[read / write](#)

Specifies the manner in which report pages are numerated.

type

```
TdxPageNumbers = (pnAll, pnEven, pnOdd);
```

```
property PageNums: TdxPageNumbers;
```

Description

Use the **PageNums** property to specify the manner in which report pages are numerated. Possible values include:

Value	Description
pnAll	Numerates all report pages.
pnEven	Numerate only even report pages.
pnOdd	Numerates only odd report pages.

The default value for this property is **pnAll**.

See Also

TdxPrintDialog.PageCount; TdxPrintDialog.PageRanges; TdxPrintDialog.Pages

PageRanges property

[read / write](#)

Determines the type of report pages to print.

type

```
TdxPageRanges = (prAll, prCurrent, prRange, prSelection);
```

```
property PageRanges: TdxPageRanges;
```

Description

Use the **PageRanges** property to determine which portion of a report to print.

Value	Description
prAll	Prints all report pages.
prCurrent	Prints only the current page of a report.
prRange	Prints range of pages.
prSelection	Prints only the selected area.

The default value for this property is **prAll**.

See Also

TdxPrintDialog.PageCount; TdxPrintDialog.PageNums; TdxPrintDialog.Pages

Pages property**read / write**

Enumerates pages to print.

property Pages: string;

Description

Use the **Pages** property to enumerate pages and/or a range of pages to print.

See Also

TdxPrintDialog.PageCount; TdxPrintDialog.PageNums; TdxPrintDialog.PageRanges

PreviewBtnClicked property**read only**

Determines whether the “**Preview**” button is pressed.

property PreviewBtnClicked: **Boolean**;

Description

If a user presses the “ **Preview** ” button, the **PreviewBtnClicked** property is set to **True**.

See Also

TdxPrintDialog.PrintBtnClicked

PrintToFile property**read / write**

Determines whether a report is printed to a file.

property PrintToFile: **Boolean**;

Description

Use the **PrintToFile** property to determine whether a report is printed to a file.

The default value for this property is **False**.

See Also

TdxPrintDialog.FileList; TdxPrintDialog.FileName; TdxPrintDialog.UseFileList

PrintBtnClicked property**read only**

Determines whether the “**Print**” button is pressed.

property PrintBtnClicked: **Boolean**;

Description

If a user presses the “**Print**” button, the **PrintBtnClicked** property returns **True**.

This property is for internal use only.

See Also

TdxPrintDialog.PreviewBtnClicked

StyleManager property

read / write

Specifies which style manager to use.

```
property StyleManager: TdxPrintStyleManager;
```

Description

Use the **StyleManager** property to specify the style manager component to work with. Since your application may have several style managers, you are free to choose between them using the **StyleManager** property.

Title property

read / write

Determines a report's title.

```
property Title: string;
```

Description

Use the **Title** property to determine your report title.

UseFileList property

read / write

Determines whether to use a file list.

```
property UseFileList: Boolean;
```

Description

The **UseFileList** property allows you to use files from a file list. The default value for this property is **False**.

See Also

TdxPrintDialog.FileList; TdxPrintDialog.FileName

TdxPrintDialog Methods

In TdxPrintDialog

Execute **public**

Execute method

Invokes a printer dialog.

function Execute: **Boolean**;

Description

Use the **Execute** property to invoke the printer dialog window.

TdxPrintDialog Events

In TdxPrintDialog

OnClose	published	OnShow	published
OnPageSetup	published		

OnClose event

Occurs when closing a printer dialog.

```
property OnClose: TNotifyEvent;
```

Description

Write an **OnClose** event handler to perform specific actions when closing a dialog.

See Also

TdxPrintDialog.OnPageSetup; TdxPrintDialog.OnShow

OnPageSetup event

Occurs when clicking the "**PageSetup...**" button.

type

```
TdxPageSetupEvent = procedure(Sender: TObject; var ADone: Boolean;
APreviewBtnClicked, APrintBtnClicked: PBoolean) of object;
```

```
property OnPageSetup: TdxPageSetupEvent;
```

Description

Write an **OnPageSetup** event handler to perform specific actions when clicking the "**PageSetup...**" button. You may use the **OnPageSetup** event handler to display your own Page Setup dialog window which is customized via the TdxPageSetupDialog component.

The **ADone** parameter is set to **True** if the PageSetup form was closed by pressing the 'OK' button.

The **APreviewBtnClicked** and **APrintBtnClicked** parameters determine whether the Preview button and/or Print button were clicked.

Note: *PBoolean* is declared in *dxPSGlibl* and this unit name must be manually added to the form's "uses" clause.

See Also

TdxPrintDialog.OnClose; TdxPrintDialog.OnShow

OnShow event

Occurs when opening a printer dialog.

property OnShow: TNotifyEvent;

Description

Write an **OnShow** event handler to perform specific actions when displaying a dialog.

See Also

TdxPrintDialog.Execute; TdxPrintDialog.OnClose; TdxPrintDialog.OnPageSetup

TdxPSEngine Object

Used to access certain settings of the ExpressPrinting System.

Unit

dxPSEngn

```
TdxPSEngine = class(TObject)
```

Description

All ExpressPrinting System components and windows (preview and dialog) use settings of one global object - **TdxPSEngine**. Settings for this object include the registry path and help file. They also specify whether to save window positions and the manner in which preview windows are displayed (standard or flat). The **TdxPSEngine** object can be accessed via the **dxPSEngine** global function. Since this object is global and unique, it should be used at runtime instead of a **TdxPSEngineController** component (you may place several components onto a form).

Hierarchy

TObject

TdxPSEngine Properties

In TdxPSEngine

DesignTimeRegistryPath	public / read only	RealRegistryPath	public / read only
HelpFile	public / read / write	RegistryPath	public / read / write
LookAndFeel	public / read / write	SaveFormsPosition	public / read / write

DesignTimeRegistryPath property

read only

Specifies the Registry path used at design time.

```
property DesignTimeRegistryPath: string;
```

Description

Use the **DesignTimeRegistryPath** property to obtain the Registry path where the ExpressPrinting System stores all settings when working at design time.

See Also

TdxPSEngine.RealRegistryPath; TdxPSEngine.RegistryPath

HelpFile property

read / write

Specifies the name of the default file used to display Help at runtime.

```
property HelpFile: string;
```

Description

You can provide your own help for an application that uses the ExpressPrinting System. Use the **HelpFile** property to specify this help file.

LookAndFeel property

read / write

Specifies preview window draw style.

type

```
TdxPSLookAndFeel = (pslfStandard, pslfFlat);
```

```
property LookAndFeel: TdxPSLookAndFeel;
```

Description

Changes to the **LookAndFeel** property affect preview toolbar paint styles. If drawing style is **pslfStandard**, a 'vertical line' appears on the left side of a toolbar. If the **LookAndFeel** property is set to **pslfFlat**, preview toolbars have an MS Office 10 (XP) look & feel.

Value	Example
-------	---------

pslfStandard**pslfFlat**

RealRegistryPath property

read only

Specifies the current Registry path.

```
property RealRegistryPath: string;
```

Description

Use the **RealRegistryPath** property to obtain the Registry path where the ExpressPrinting System stores all settings. This property returns the **DesignTimeRegistryPath** property value at design time or the **RegistryPath** property value at runtime.

See Also

TdxPSEngine.DesignTimeRegistryPath; TdxPSEngine.RegistryPath

RegistryPath property

read / write

Specifies the registry path for preview window settings.

```
property RegistryPath: string;
```

Description

The **RegistryPath** property is used to specify the path to save design-time and run-time settings of the preview window.

The ExpressPrintingSystem 2.0 allows you to entirely specify the registry path by setting the RegistryPath property of the TdxPSEngineController component.

See Also

TdxPSEngine.DesignTimeRegistryPath; TdxPSEngine.RealRegistryPath

SaveFormsPosition property

read / write

Specifies whether to save window positions.

```
property SaveFormsPosition: Boolean;
```

Description

Use the **SaveFormsPosition** property to specify the manner in which the preview and dialog windows are displayed. If this property is set to **True**, all windows will be displayed at the same coordinates

(position on screen) where they were last closed. If this property is set to **False**, all windows are displayed at their default positions.



TdxPSEngineController Object

Used to access specific settings of the ExpressPrinting System.

Unit

dxPSEngn

```
TdxPSEngineController = class(TComponent)
```

Description

All ExpressPrinting System components and windows (dialog and preview) use settings of the **TdxPSEngineController** component. Object settings include the registry path and help file. They also specify whether to save window positions and the manner in which preview windows are displayed (standard or flat). Use this component at design time to adjust specific settings. We recommended that you use the **TdxPSEngine** object at runtime.

You may place several **TdxPSEngineController** components onto a form. Use the **Activate** method to specify which of these components is active and is used by the ExpressPrinting System.

Hierarchy

TObject

TPersistent

TComponent

TdxPSEngineController Properties

In TdxPSEngine

HelpFile	published / read / write	RegistryPath	published / read / write
LookAndFeel	published / read / write	SaveFormsPosition	published / read / write

HelpFile property

read / write

Specifies the name of the default file used to display Help at runtime.

```
property HelpFile: string;
```

Description

You can provide your own help for an application that uses the ExpressPrinting System. Use the **HelpFile** property to specify the name of the appropriate help file.

LookAndFeel property

read / write

Specifies preview windows draw style.

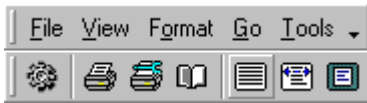
type

```
TdxPSLookAndFeel = (pslfStandard, pslfFlat);
```

```
property LookAndFeel: TdxPSLookAndFeel;
```

Description

Changes to the **LookAndFeel** property affect preview toolbar paint styles. If drawing style is **pslfStandard**, a 'vertical line' appears on the left side of a toolbar. If the **LookAndFeel** property is set to **pslfFlat**, preview toolbars have an MS Office 10 (XP) look & feel.

Value	Example
pslfStandard	
pslfFlat	

The default value for this property is **pslfStandard**.

RegistryPath property

read / write

Specifies the registry path for ExpressPrinting System settings.

property RegistryPath: string;

Description

Use the **RegistryPath** property to specify a path to save ExpressPrinting System settings.

SaveFormsPosition property

read / write

Specifies whether to save window positions.

property SaveFormsPosition: **Boolean**;

Description

Use the **SaveFormsPosition** property to specify the manner in which preview and dialog windows are displayed. If this property is set to **True**, all windows will be displayed at the same coordinates (position on screen) where they were last closed. If this property is set to **False**, all windows are displayed at their default positions.

The default value for this property is **True**.

TdxPSEngineController Methods

In TdxPSEngine

Activate **public**

Activate method

Activates the current TdxPSEngineController component.

```
procedure Activate;
```

Description

You may place several TdxPSEngineController components onto a form. Call the **Activate** method to specify which of these components is active and is used by the ExpressPrinting System.

TdxBaseObject Object

Unit

dxBase

```
TdxBaseObject = class (TPersistent);
```

Description

The **TdxBaseObject** class is the base class for all objects that define the look & feel of a report.

For Internal use only.

Hierarchy

TObject

TPersistent

TdxBaseObject Methods

In TdxBaseObject

BeginUpdate	public	LoadFromFile	public
CancelUpdate	public	LoadFromStream	public
Clone	public	SaveToFile	public
EndUpdate	public	SaveToStream	public
IsEqual	public		

BeginUpdate method

Disables all TdxBaseObject operations.

```
procedure BeginUpdate;
```

Description

Use the **BeginUpdate** method to prevent immediate background redrawing. Remember to call the **EndUpdate** method after calling **BeginUpdate**. Every call of the **BeginUpdate** method increases the internal counter by one. Every call of the **EndUpdate** method decreases this counter by one. An operation can be applied to the TdxBaseObject object only when the internal counter is 0.

See Also

TdxBaseObject.CancelUpdate; TdxBaseObject.EndUpdate

CancelUpdate method

Enables operations for the TdxBaseObject.

```
procedure CancelUpdate;
```

Description

The **CancelUpdate** method nulls the internal counter of the **BeginUpdate** and **EndUpdate** methods. Call this method to enable operations regardless of the internal counter.

See Also

TdxBaseObject.BeginUpdate; TdxBaseObject.EndUpdate

Clone method

Creates a new TdxBaseObject instance.

```
function Clone: TdxBaseObject; virtual;
```

Description

Use the Clone function to create a new instance of the TdxBaseObject.

EndUpdate method

Enables TdxBaseObject operations.

```
procedure EndUpdate;
```

Description

The **BeginUpdate** and **EndUpdate** methods work together to prevent numerous updates of the TdxBaseObject. Make certain to call the **EndUpdate** method after calling **BeginUpdate**. Every call of the **BeginUpdate** method increases the internal counter by one. Every call of the **EndUpdate** method decreases the internal counter by one. An operation can be applied to the **TdxBaseObject** object only when the internal counter is 0.

See Also

TdxBaseObject.BeginUpdate; TdxBaseObject.CancelUpdate

IsEqual method

Determines whether the ABaseObject is of type TdxBaseObject.

```
function IsEqual(ABaseObject: TdxBaseObject): Boolean; virtual;
```

Description

Use the **IsEqual** function to determine whether the *ABaseObject* is aTdxBaseObject class type. The **IsEqual** function returns **True** if the *ABaseObject* is aTdxBaseObject class type; **False** otherwise.

LoadFromFile method

Loads TdxBaseObject contents from a file.

```
procedure LoadFromFile(const AFileName: string); dynamic;
```

Description

Use the **LoadFromFile** method to load the TdxBaseObject contents from the file specified by the *AFileName* constant.

See Also

TdxBaseObject.LoadFromStream; TdxBaseObject.SaveToFile; TdxBaseObject.SaveToStream

LoadFromStream method

Loads data form a stream into the TdxBaseObject object.

```
procedure LoadFromStream(AStream: TStream); dynamic;
```

Description

Use the **LoadFromStream** method to read TdxBaseObject data from the stream specified by the *AStream* variable.

See Also

TdxBaseObject.LoadFromFile; TdxBaseObject.SaveToFile; TdxBaseObject.SaveToStream

SaveToFile method

Saves the TdxBaseObject contents to a file.

```
procedure SaveToFile(const AFileName: string); dynamic;
```

Description

Use the **SaveToFile** procedure to save the current TdxBaseObject object to the file specified by the *AFileName* constant.

See Also

TdxBaseObject.LoadFromFile; TdxBaseObject.LoadFromStream; TdxBaseObject.SaveToStream

SaveToStream method

Saves TdxBaseObject contents to a stream.

```
procedure SaveToStream(AStream: TStream); dynamic;
```

Description

Use the **SaveToStream** procedure to save the current TdxBaseObject object to the stream specified by the *AStream* variable.

See Also

TdxBaseObject.LoadFromFile; TdxBaseObject.LoadFromStream; TdxBaseObject.SaveToFile

TdxPrinterPageObject

Defines report page attributes.

Unit

dxPrnPg

```
TdxPrinterPageObject = class(TdxBaseObject)
```

Description

Objects of type **TdxPrinterPage** are used to customize different attributes of report pages: Page margins, page size, etc.

Hierarchy

TObject

TPersistent

TdxBaseObject

TdxPrinterPageObject Properties

In TdxPrinterPageObject

Background	protected / read / write
Page	protected / read only

Background property

read / write

Specifies a report's background.

property Background: TdxBackground;

Description

Use the Background property to customize the background of report pages. By default, a report's background is transparent. Use this property to specify any background color, brush or picture.

Page property

read only

Returns information about the printer pages settings.

property Page: TdxPrinterPage;

Description

Use the **Page** property to return information about the printer pages settings.

TdxPrinterPageObject Methods

Derived from TdxBaseObject

BeginUpdate	public	LoadFromFile	public
CancelUpdate	public	LoadFromStream	public
Clone	public	SaveToFile	public
EndUpdate	public	SaveToStream	public
IsEqual	public		

TCustomdxPageObject

Encapsulates general properties and methods of the Page object.

Unit

dxPrnPg

```
TCustomdxPageObject = class(TdxPrinterPageObject)
```

Description

The **TCustomdxPageObject** contains general methods and properties of the Page object: text setup routines: such as Font, Titles, page management routines: LoadFromFile, SaveToFile, etc.

Hierarchy

```
Object
  TPersistent
    TdxBaseObject
      TdxPrinterPageObject
```

TCustomdxPageObject Properties

In TCustomdxPageObject

CenterTextAlignY	public / read / write	RightTextAlignY	public / read / write
CenterTitle	public / read / write	RightTitle	public / read / write
Font	public / read / write	TextAlignY	public / read / write
LeftTextAlignY	public / read / write	Titles	public / read / write
LeftTitle	public / read / write		

Derived from TdxPrinterPageObject

Background	published / read / write
Page	public / read only

CenterTextAlignY property

read / write

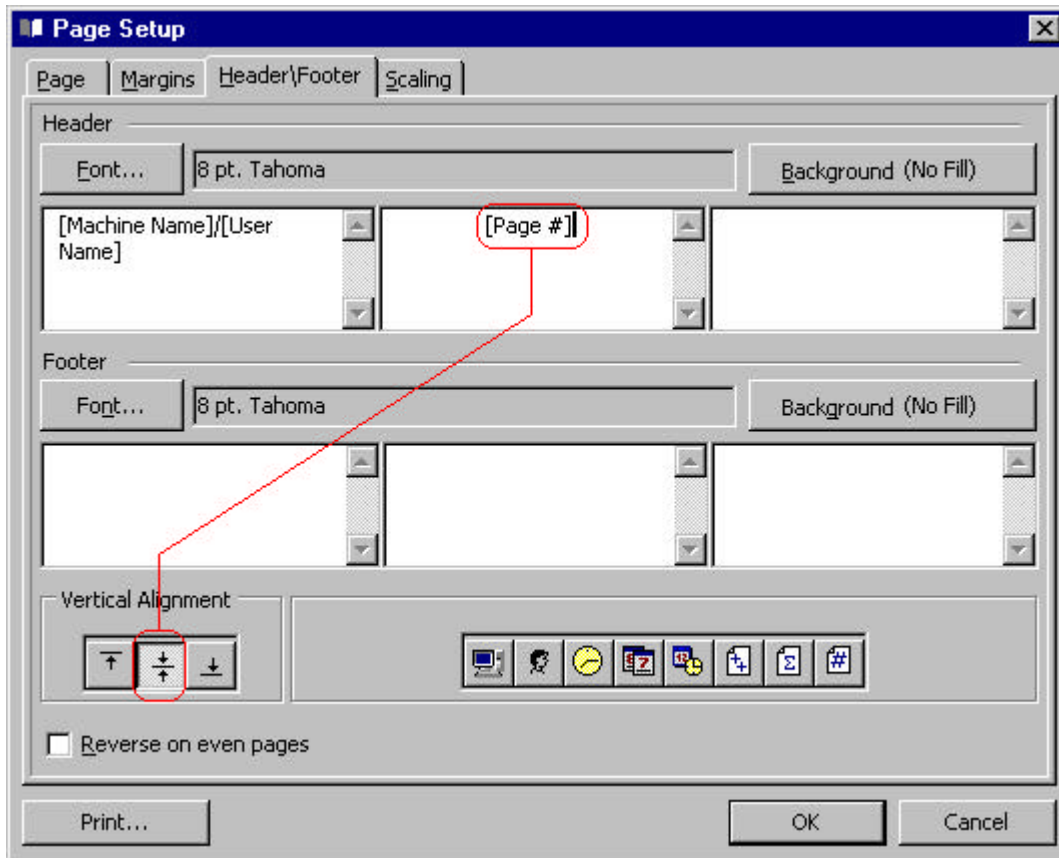
Specifies the way the title central part text is vertically aligned.

```
type
  TdxTextAlignY = (taTop, taCenterY, taBottom);
property CenterTextAlignY: TdxTextAlignY;
```

Description

Use the **CenterTextAlignY** property to specify the manner the central part of the title is vertically aligned. It can take one of the thee values:

Value	Description
taTop	Moves the text to the top of the row.
taCenterY	Moves the text to the center of the row.
taBottom	Moves the text to the bottom of the row.



The default value of this property is **taCenterY**.

See Also

TCustomdxPageObject.LeftTextAlignY; *TCustomdxPageObject.RightTextAlignY*;
TCustomdxPageObject.TextAlignY

CenterTitle property

[read / write](#)

Determines the central title text.

property CenterTitle: TStrings;

Description

Use the **CenterTitle** property to determine the text located in the central part of the title.

See Also

TCustomdxPageObject.LeftTitle; *TCustomdxPageObject.RightTitle*; *TCustomdxPageObject.Titles*

Font property

[read / write](#)

Specifies the font of the text.

property Font: TFont;

Description

Use **Font** property to specify the font for the document.

See Also

TCustomdxPageObject.Titles

LeftTextAlignY property

read / write

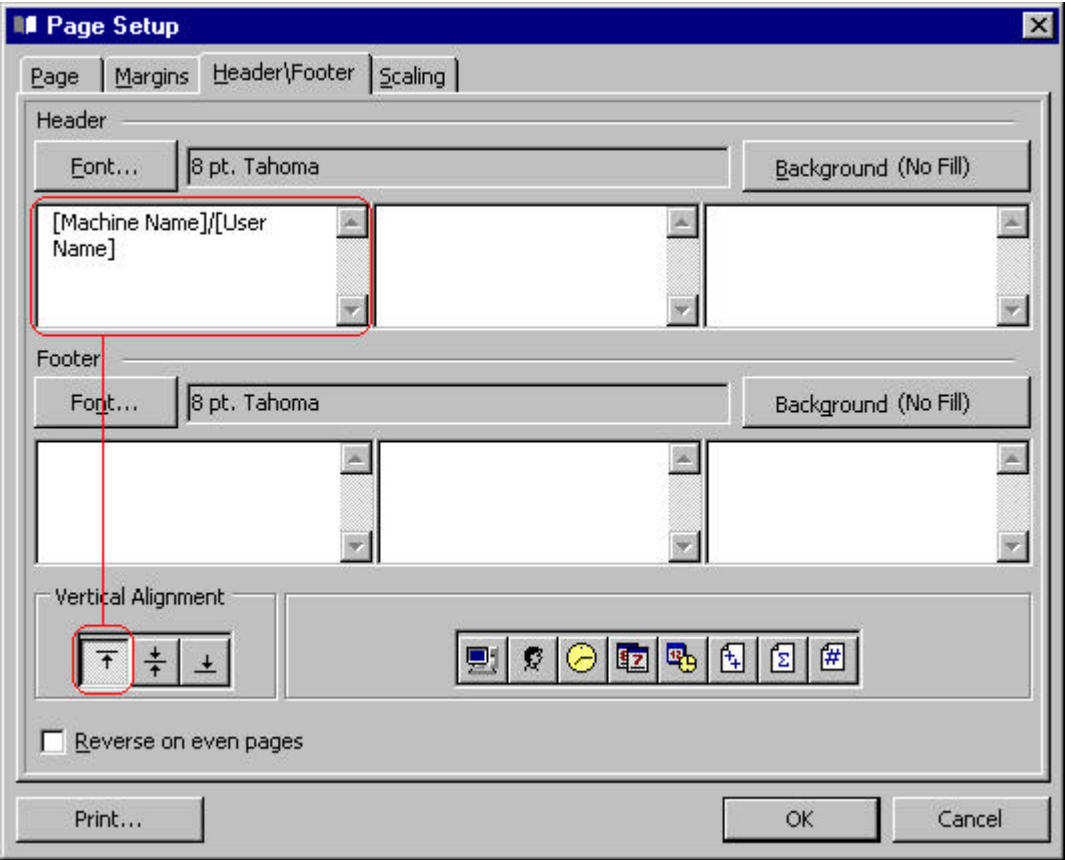
Specifies vertical alignment of the text located in the left part of either the header or footer.

```
type
    TdxTextAlignY = (taTop, taCenterY, taBottom);
property LeftTextAlignY: TdxTextAlignY;
```

Description

Use the **LeftTextAlignY** property to specify the vertical alignment of the text, which is located in the left part of either the header or footer.

Value	Description
taTop	Aligns the text to the top of the row.
taCenterY	Aligns the text to the center of the row.
taBottom	Aligns the text to the bottom of the row.



The default value of this property is **taCenterY**.

See Also

*TCustomdxPageObject.CenterTextAlignY; TCustomdxPageObject.RightTextAlignY;
TCustomdxPageObject.TextAlignY*

LeftTitle property

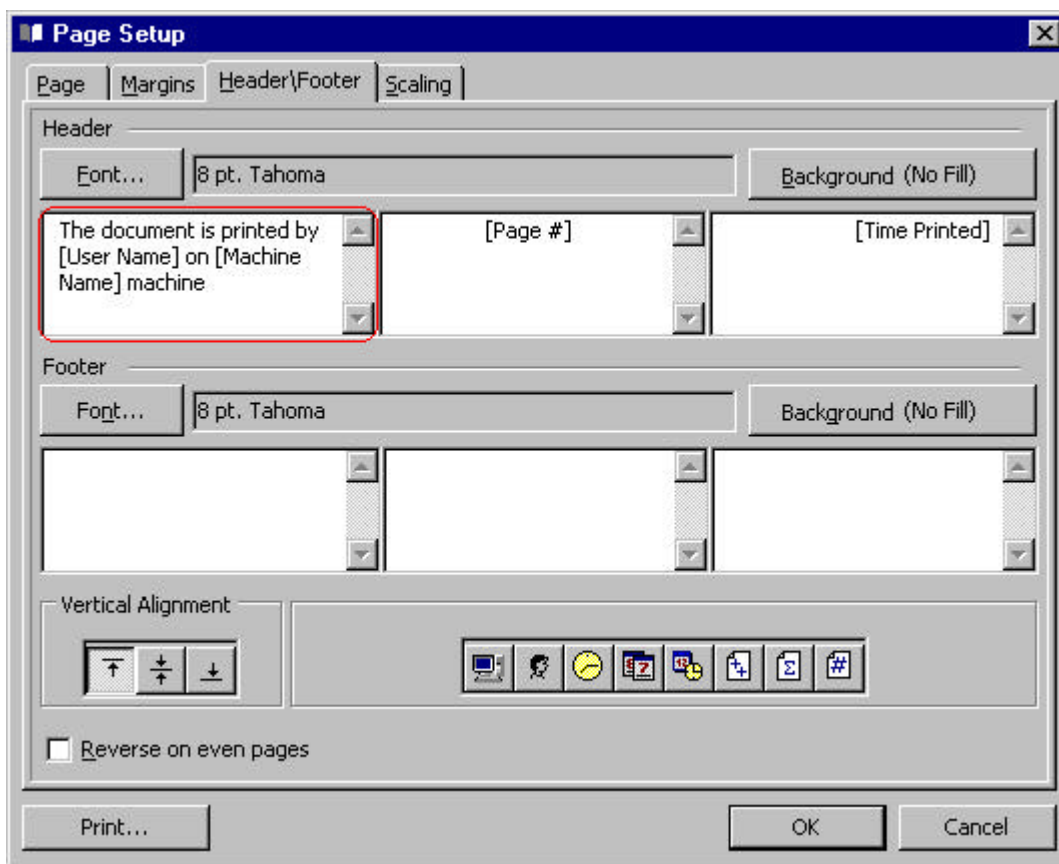
[read / write](#)

Specifies the text located in the left corner of the header.

property LeftTitle: TStrings;

Description

Use the **LeftTitle** property to specify the text that is printed in the left corner of the header/footer row. Possible values are machine name, user name, time printed, date printed, date and time printed, page # of pages #, total pages, page number.



See Also

TCustomdxPageObject.RightTitle; TCustomdxPageObject.Titles

RightTextAlignY property

[read / write](#)

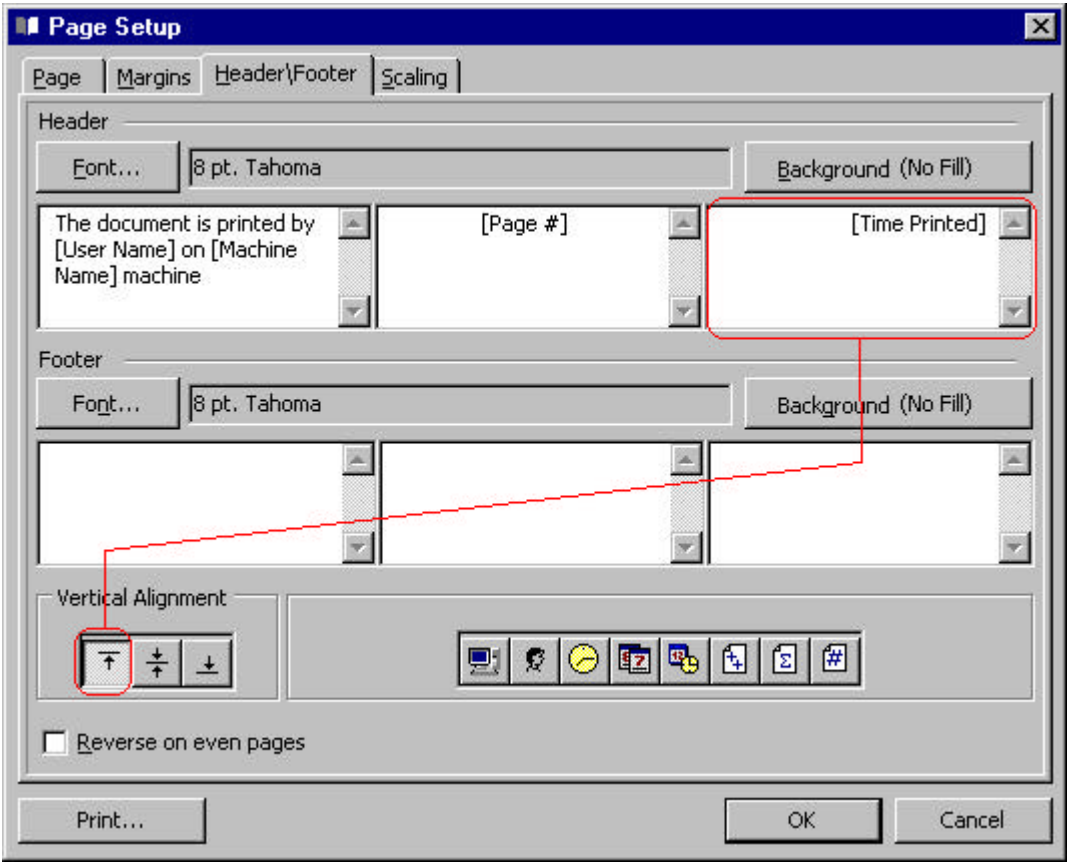
Specifies vertical alignment of text located in the right corner of either the header or footer.

```
type
    TdxTextAlignY = (taTop, taCenterY, taBottom);
property RightTextAlignY: TdxTextAlignY;
```

Description

Use the **TdxTextAlignY** property to specify alignment for the text in the right corner of the row.

Value	Description
taTop	Aligns the text to the top of the row.
taCenterY	Aligns the text to the center of the row.
taBottom	Aligns the text to the bottom of the row.



The default value of this property is **taCenterY**.

See Also

TCustomdxPageObject.CenterTextAlignY; TCustomdxPageObject.LeftTextAlignY

RightTitle property

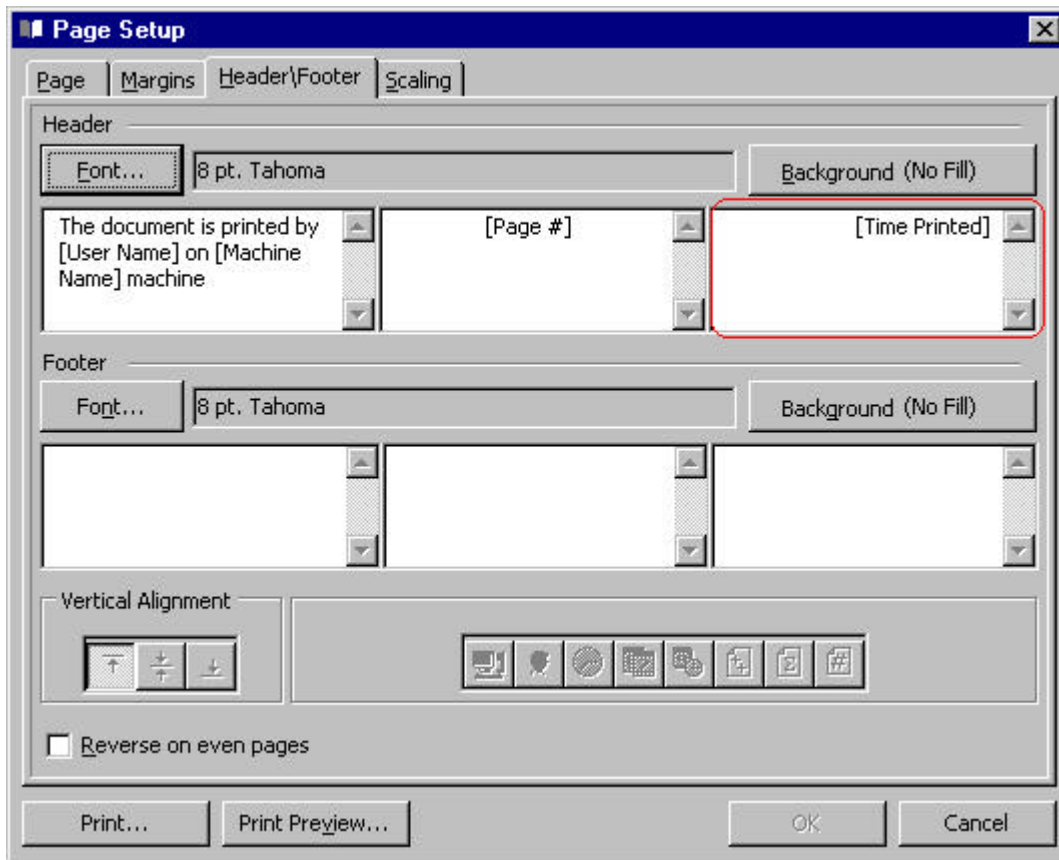
[read / write](#)

Specifies the text located in the right corner of the title.

```
property RightTitle: TStrings;
```

Description

Use the **RightTitle** property to set text for the right title.



See Also

TCustomdxPageObject.CenterTitle; TCustomdxPageObject.LeftTitle

TextAlignY property

[read / write](#)

Specifies the text vertical alignment.

type

```
TdxPageTitlePart = (tpLeft, tpCenter, tpRight);
```

```
TdxTextAlignY = (taTop, taCenterY, taBottom);
```

property TextAlignY[Index: TdxPageTitlePart]: TdxTextAlignY;

Description

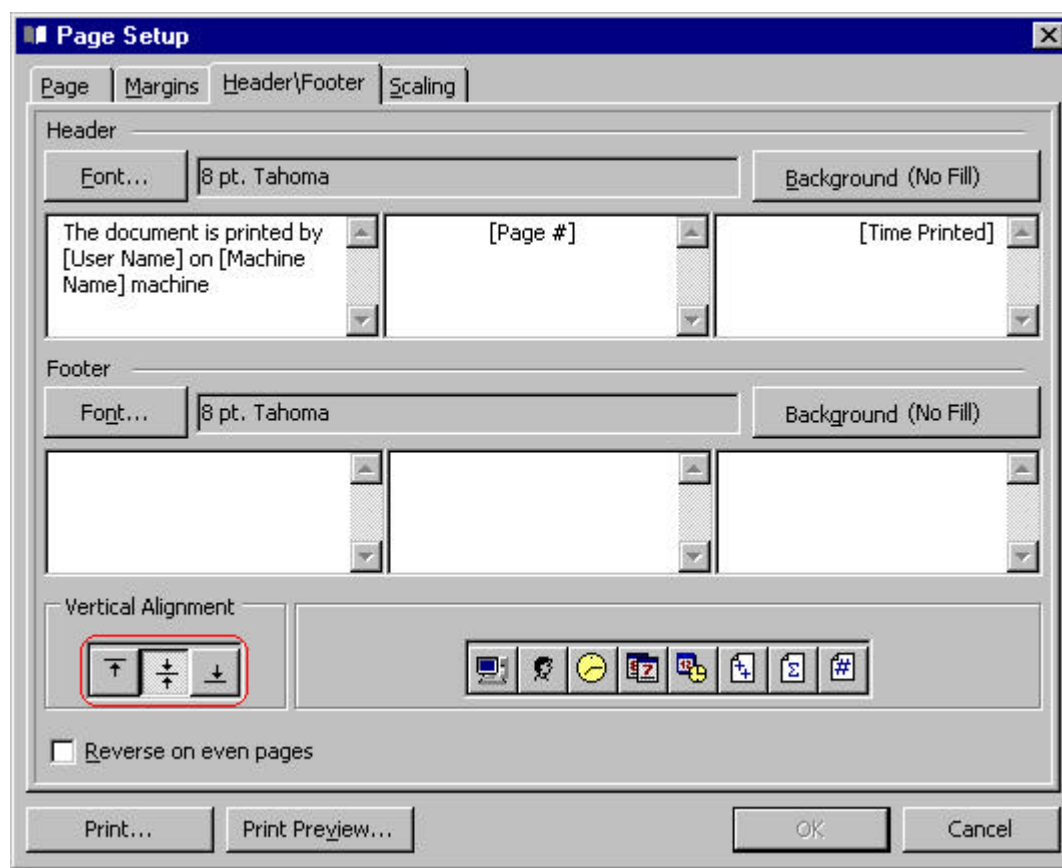
Use **TextAlignY** property to specify a kind of text vertical alignment.

Index parameter specifies the part of the title to which the **TextAlignY** property is applied.

Value	Description
taTop	Aligns the text to the top of the row.
taCenterY	Aligns the text to the center of the row.

taBottom

Aligns the text to the bottom of the row.

**See Also**

TCustomdxPageObject.CenterTextAlignY; *TCustomdxPageObject.LeftTextAlignY*;
TCustomdxPageObject.RightTextAlignY

Titles property[read / write](#)

Specifies titles for printing page.

type

```
TdxPageTitlePart = (tpLeft, tpCenter, tpRight);
```

property Titles[Index: TdxPageTitlePart]: TStrings;

Description

Use **Titles** property to refer to a particular part of the page title, specified by the *Index* parameter.

See Also

TCustomdxPageObject.CenterTitle; *TCustomdxPageObject.LeftTitle*;
TCustomdxPageObject.RightTitle

TCustomdxPageObject Methods

Derived from TdxBaseObject

BeginUpdate	public	LoadFromFile	public
CancelUpdate	public	LoadFromStream	public
Clone	public	SaveToFile	public
EndUpdate	public	SaveToStream	public
IsEqual	public		

TdxPageHeader

Provides access to header methods and properties.

Unit

dxPrnPg

```
TdxPageHeader = class(TCustomdxPageObject)
```

Description

Use the **TdxPageHeader** object to access report page header methods and properties.

Hierarchy

```
Object
  TPersistent
    TdxBaseObject
      TdxPrinterPageObject
        TCustomdxPageObject
```

TdxPageHeader Properties

Derived from TCustomdxPageObject

CenterTextAlignY	published / read / write	RightTextAlignY	published / read / write
CenterTitle	published / read / write	RightTitle	published / read / write
Font	published / read / write	TextAlignY	public / read / write
LeftTextAlignY	published / read / write	Titles	public / read / write
LeftTitle	published / read / write		

Derived from TdxPrinterPageObject

Background	published / read / write
Page	public / read only

TdxPageHeader Methods

Derived from TdxBaseObject

BeginUpdate	public	LoadFromFile	public
CancelUpdate	public	LoadFromStream	public
Clone	public	SaveToFile	public
EndUpdate	public	SaveToStream	public
IsEqual	public		

TdxPageFooter

Provides access to the footer property.

Unit

dxPrnPg

```
TdxPageFooter = class(TCustomdxPageObject)
```

Description

Use the **TdxPageFooter** object to access the report page footer methods and properties.

Hierarchy

```
Object
  TPersistent
    TdxBaseObject
      TdxPrinterPageObject
        TCustomdxPageObject
```

TdxPageFooter Properties

Derived from TCustomdxPageObject

CenterTextAlignY	published / read / write	RightTextAlignY	published / read / write
CenterTitle	published / read / write	RightTitle	published / read / write
Font	published / read / write	TextAlignY	public / read / write
LeftTextAlignY	published / read / write	Titles	public / read / write
LeftTitle	published / read / write		

Derived from TdxPrinterPageObject

Background	published / read / write
Page	public / read only

TdxPageFooter Methods

Derived from TdxBaseObject

BeginUpdate	public	LoadFromFile	public
CancelUpdate	public	LoadFromStream	public
Clone	public	SaveToFile	public
EndUpdate	public	SaveToStream	public
IsEqual	public		

TdxPreviewOptions Object

Unit

dxPSCore

```
TdxPreviewOptions = class(TdxBaseObject);
```

Description

This class is used to specify preview window attributes at design-time. Use properties of the **TdxPreviewOptions** class to set caption, size, and icon of a preview window. It also provides specific help context for a preview window (if necessary) and allows users to customize a preview form's toolbar. Use the PreviewWindow property of the TCustomdxComponentPrinter class to work directly with a preview window.

Hierarchy

```
    TObject
      TPersistent
        TdxBaseObject
```

TdxPreviewOptions Properties

TdxPreviewOptions

Caption	published / read / write	RegistryPath	published / read / write
EnableOptions	published / read / write	SavePosition	published / read / write
Height	published / read / write	SaveZoomPosition	published / read / write
HelpContext	published / read / write	Top	published / read / write
HelpFile	published / read / write	VisibleOptions	published / read / write
Icon	published / read / write	Width	published / read / write
Left	published / read / write	WindowState	published / read / write
Rect	public / read / write		

Caption property

read / write

```
property Caption : string;
```

Description

Specifies a preview window's caption. Use this caption to provide a user with more information about a printed component. The following code is the same as when setting the caption of a preview window within the TdxComponentPrinter OnBeforePreview event handler:

```
// using Caption
. . .
dxComponentPrinter1.PreviewOptions.Caption := 'New Preview Caption';
dxComponentPrinter1.Preview(. . .);
. . .

// within an OnBeforePreview event handler
procedure TForm1.dxCComponentPrinter1BeforePreview(Sender: TObject);
begin
    . . .
    dxComponentPrinter1.PreviewWindow.Caption := 'New Preview Caption';
    . . .
end;
```

See Also

TCustomdxComponentPrinter.PreviewOptions; TCustomdxComponentPrinter.PreviewWindow

EnableOptions property

read / write

Determines which buttons and menu items are available to the end user within a preview window.

type

```
TdxPreviewEnableOption = (peoCanChangeMargins, peoHelp,
peoPageBackground, peoPageSetup, peoPreferences, peoPrint,
peoReportDesign);
TdxPreviewEnableOptions = set of TdxPreviewEnableOption;
```

```
property EnableOptions : TdxPreviewEnableOptions;
```

Description

Use this property to specify which controls will be displayed on the toolbar of the preview window and which menu items are enabled for the end user. By default, the following options are active:

Value	Meaning
peoPreferences	If active, the end user can customize the preview window.
peoPrint	If active, the end user can call the Print dialog from a preview window.
peoPageSetup	If active, the end user can call the PageSetup dialog from a preview window.
peoReportDesign	Activates the report link designer.
peoPageBackground	Allows the user to customize a report's background (background color and gradient fill).
peoCanChangeMargins	Allows end users to change margins within a preview window.
peoHelp	Activate this option to display help for a preview window.

Note: The *EnableOptions* property enables specific toolbar buttons and menu items to a user. The *VisibleOptions* property determines which buttons and menu items will be visible to a user. Use the *EnableOptions* property to disable specific functions in your programs.

See Also

TdxPreviewOptions.VisibleOptions

Height property

[read / write](#)

Specifies the vertical size of the preview form in pixels.

```
property Height : Integer;
```

Description

Use the **Height** property to read or change the height of a preview window.

See Also

TCustomdxComponentPrinter.PreviewWindow;TdxPreviewOptions.Left;TdxPreviewOptions.Right;TdxPreviewOptions.Top;TdxPreviewOptions.Width

HelpContext property

[read / write](#)

Provides the context number for use when calling context-sensitive online Help.

type

```
THelpContext = -MaxLongInt..MaxLongInt;  
property HelpContext : THelpContext;
```

Description

Assign a value to the **HelpContext** property to determine which help screen appears when a user presses F1 to request help. Each screen in the Help system should have a unique context number. When a preview window is active, pressing F1 displays the Help screen. The Help topic displayed depends on the **HelpContext** property value.

See Also

TdxPreviewOptions.HelpFile

HelpFile property

[read / write](#)

Specifies the name of the help file used to display Help for a preview window.

```
property HelpFile : string;
```

Description

Use the **HelpFile** property to specify a help file used to display help for a preview window. Windows selects Help topics from this file. The default help file is not used for a preview form at runtime.

See Also

TdxPreviewOptions.HelpContext

Icon property

[read / write](#)

Specifies the icon of a preview form.

```
property Icon : TIcon;
```

Description

Set the **Icon** property to provide an icon for the preview window. If the icon is not set, then the default icon is used.

See Also

TCustomdxComponentPrinter.PreviewWindow;TdxPreviewOptions.RestoreOriginalIcon

Left property

[read / write](#)

Specifies the horizontal coordinates of the preview window's left edge relative to the parent control of the preview window in pixels.

```
property Left : Integer;
```

Description

Use the **Left** property to determine where the left side of the preview window begins. If the preview window is stored in another control, (in TTabSheet, for instance), the **Left** and **Top** properties are relative to the TTabSheet control.

See Also

TCustomdxComponentPrinter.PreviewWindow;TdxPreviewOptions.Height;TdxPreviewOptions.Rect;TdxPreviewOptions.Top;TdxPreviewOptions.Width

Rect property

read only

Specifies the rectangle occupied by the preview window.

property Rect : TRect;

Description

Use the **Rect** property to set the four coordinates of the preview window.

See Also

TdxPreviewOptions.Height;TdxPreviewOptions.Left;TdxPreviewOptions.Top;TdxPreviewOptions.Width

RegistryPath property

read / write

Specifies the registry path for preview window settings.

property RegistryPath : string;

Description

The **RegistryPath** property is used to specify the path to save design-time and run-time settings of the preview window.

The ExpressPrinting System 2 allows you to entirely specify the registry path by setting the **RegistryPath** property of the **TdxPSEngineController** component.

See Also

TCustomdxComponentPrinter.PreviewWindow

SavePosition property

read / write

Determines whether to save the position of the preview window.

property SavePosition : Boolean;

Description

You should set the **SaveZoomPosition** property to **True** if you want the preview window to be displayed at the same position when you call the preview window again. The default value for this property is **True**.

The ExpressPrintingSystem 2.0 allows you to entirely specify whether to save the window position by setting the **SaveFormsPosition** property of the **TdxPSEngineController**

SaveZoomPosition property

read / write

Determines whether to save zoom settings when exiting the preview window.

property SaveZoomPosition : **Boolean**;

Description

While working with the preview window you may tune up the zoom level to display report pages. You should set the **SaveZoomPosition** property to **True** if you want to have the same report view when you call the preview window again. The default value for this property is **True**.

Top property

read / write

Specifies the vertical position of the preview window's top edge in pixels, relative to its parent or container control.

property Top : **Integer**;

Description

Use the **Top** property to locate the top of a control or position it to another vertical coordinate. The **Top** property, like the **Left** property, is the position of the preview window relative to its container. Thus, if the window is within another window, the **Left** and **Top** properties are relative to this parent window.

See Also

TCustomdxComponentPrinter.PreviewWindow;TdxPreviewOptions.Height;TdxPreviewOptions.Left;TdxPreviewOptions.Rect;TdxPreviewOptions.Width

VisibleOptions property

read / write

Determines which toolbar buttons and menu items of the preview window are visible.

type

`TdxPreviewVisibleOption = [pvoPreferences, pvoPrint, pvoPageSetup, pvoReportDesign, pvoPageBackground, pvoHelp];`

`TdxPreviewVisibleOptions = set of TdxPreviewVisibleOption;`

property VisibleOptions : TdxPreviewVisibleOptions;

Description

Use this property to customize the look of the preview window toolbar and menu. Available options are as follows:

Value	Meaning
pvoHelp	Activate this option to display the Help button and Help menu items for the preview window.

pvoPageBackground	If active, both a button and menu item for the background customization are visible.
pvoPageSetup	If active, the PageSetup button and menu item are visible in the preview window.
pvoPreferences	If active, the Preferences menu item is visible.
pvoPrint	If active, the Print button and menu item are visible.
pvoReportDesign	If active, the menu item that calls the report link designer is visible.

Note: The *VisibleOptions* property determines which buttons and menu items will be visible. The *EnableOptions* property enables/disables specific toolbar buttons and menu items to a user.

See Also

TdxPreviewOptions.EnableOptions

Width property

[read / write](#)

Specifies the horizontal size of the preview window in pixels.

```
property Width : Integer;
```

Description

Use the **Width** property to read or set the width of the preview window.

See Also

TdxPreviewOptions.Height;TdxPreviewOptions.Left;TdxPreviewOptions.Rect;TdxPreviewOptions.Top

WindowState property

[read / write](#)

Determines the manner in which the preview window is displayed.

type

```
TWindowState = (wsNormal, wsMinimized, wsMaximized);
```

```
property WindowState : TWindowState;
```

Description

Use the **WindowMenu** property to determine whether a preview form appears minimized, maximized or as standard on screen. The **WindowState** property can have one of the following values:

Value	Meaning
wsNormal	Preview form appears in a standard state (it is neither minimized nor maximized). The standard state is defined by the Left, Top, Height and Width coordinates.
wsMinimized	Preview form appears minimized.
wsMaximized	Preview form appears maximized.

By default, the **WindowState** property is equal to **wsNormal**.

See Also

TCustomdxComponentPrinter.PreviewWindow

TdxPreviewOptions Methods

TdxPreviewOptions

RestoreOriginalIcon **public**

RestoreOriginalIcon method

Restores the default icon.

```
procedure RestoreOriginalIcon;
```

Description

Since a user can change the icon of the preview form, the **RestoreOriginalIcon** method allows you to return to the predefined icon.

See Also

TdxPreviewOptions.Icon

TdxReportCells Object

Unit

dxPSCore

```
TdxReportCells = class(TdxBaseObject);
```

Description

The result report consists of cells that contain data from VCL components. The **TdxReportCells** object is a collection of such cells. It is passed as a parameter to the ConstructReport method. This object is for internal use.

Hierarchy

```
Object
  TPersistent
    TdxBaseObject
```

TBasedxPreviewWindow Object

Unit

dxPSCore

```
TBasedxPreviewWindow = class(TForm);
```

Description

The **TBasedxPreviewWindow** object is the common ancestor for all preview windows. It is used for internal purposes.

Hierarchy

TForm

TAbstractdxPreviewWindowDesigner Object

Unit

dxPSCore

```
TAbstractdxPreviewWindowDesigner = class(TObject);
```

Description

The **TAbstractdxPreviewWindowDesigner** object controls the life-time of the preview window and is used at design-time only.

Hierarchy

TObject

TdxReportTitle Object

Unit

dxPSCore

```
TdxReportTitle = class(TPersistent);
```

Description

The **TdxReportTitle** object controls the appearance of the report title.

Hierarchy

```
Object
  TPersistent
    TComponent
```

TdxReportTitle Properties

TdxPreviewOptions

Color	published / read / write	TextAlignX	published / read / write
Font	published / read / write	TextAlignY	published / read / write
ReportLink	public / read only		

Color property

read / write

Specifies the color of a report's title text.

```
property Color : TColor;
```

Description

Use the **Color** property to specify the background color when drawing a report's title.

Font property

read / write

Specifies the font for a report's title text.

```
property Font : TFont;
```

ReportLink property

read only

Determines the report link for which to assign a title.

```
property ReportLink: TBasedxReportLink ;
```

TextAlignX property

read / write

Specifies the horizontal alignment of an item's text.

type

```
TdxTextAlignY = (taLeft, taCenterX, taRight);
```

```
property TextAlignX : TdxTextAlignX;
```

Description

Use the **TextAlignX** property to specify horizontal report title text alignment. The following values are available:

Values	Meaning
taLeft	Report title text is left justified.
taCenterX	Report title text is centered within its drawing rectangle.
taRight	Report title text is left justified.

The default value for the **TextAlignX** property is **taCenterX**.

See Also

TdxReportTitle.TextAlignY

TextAlignY property

[read / write](#)

Specifies the horizontal alignment of an item's text.

type

```
TdxTextAlignY = ( taTop, taCenterY, taBottom );
```

property TextAlignY : TdxTextAlignY;

Description

Use the **TextAlignX** property to specify horizontal report title text alignment. The following values are available:

Values	Meaning
taLeft	Report title text is left justified.
taCenterX	Report title text is centered within its drawing rectangle.
taRight	Report title text is left justified.

The default value for the **TextAlignY** property is **taCenterY**.

See Also

TdxReportTitle.TextAlignX

TdxReportTitle.Methods

TdxPreviewOptions

DoChange

protected

DoChange method

Repaints the current report.

```
procedure DoChange ; dynamic;
```

Description

This method is called when changing properties of the TdxReportTitle object.

Global Routines and Variables

GetDefaultMeasurementUnits
 DefaultPrinterPage
 RereadDefaultPrinterPage
 RegisterPreviewWindow

UnRegisterPreviewWindow
 dxPageSetup
 FPageSetupDialogProc

GetDefaultMeasurementUnits

routine

Unit

dxPrnPg

type

```
TdxMeasurementUnits = (muDefault, muInches, muMillimeters);  

function GetDefaultMeasurementUnits: TdxMeasurementUnits;
```

Description

The **GetDefaultMeasurementUnits** function determines the default measurement units used in a system. This function returns **muInches** if inch is the base measurement unit in a system; Otherwise, it returns **muMillimeters**.

DefaultPrinterPage

routine

Unit

dxPrnPg

```
function DefaultPrinterPage;
```

Description

The **DefaultPrinterPage** returns an instance of the **TdxPrinterPage** object with default parameters for a page, such as margins, minimum margins, page size, etc.

RereadDefaultPrinterPage

routine

Unit

dxPrnPg

```
procedure RereadDefaultPrinterPage;
```

Description

The **RereadDefaultPrinterPage** procedure re-reads current printer parameters and assigns these values to the default printer page.

RegisterPreviewWindow**routine**

Unit

dxPSCore

type`TdxPreviewWindowClass = class of TBasedxPreviewWindow;``procedure RegisterPreviewWindow (APreviewClass :TdxPreviewWindowClass);`**Description**

This procedure registers the preview window specified by the *APreviewClass* parameter (Places it in stack of all preview windows). The last registered preview window is displayed when a user calls the **Preview**.

See Also*UnRegisterPreviewWindow*

UnRegisterPreviewWindow**routine**

Unit

dxPSCore

`procedure UnRegisterPreviewWindow;`**Description**

This procedure unregisters the preview window specified as last in the stack of all preview windows. If all preview windows are unregistered, then a preview is not displayed when a user calls the **Preview**.

See Also*RegisterPreviewWindow*

dxPageSetup**routine**

Activates the **PageSetup** dialog box.

`function PageSetup: Boolean;`**Description**

This method activates the **PageSetup** dialog which in turn enables a user to specify the following attributes of a printed page:

Size and source

Orientation and width of page margins.

This method is called from the PageSetup method of the report link.

FPageSetupDialogProc

variable

Determines which type of **PageSetup** dialog to use.

type

```
TdxPageSetupDialogProc = function(AReportLink : TBasedxReportLink) :  
Boolean;  
FPageSetupDialogProc : TdxPageSetupDialogProc = nil;
```

Description

By default, the **FPageSetupDialogProc** global constant value is nil and the standard window dialog is used. You can specify you own PageSetup dialog via this constant.

Index

A

Activate.....	172
ActivePageIndex.....	142
AddStyle.....	134
AddStyleEx.....	134
AllowChangeHFText.....	115
AllowChangeMargins.....	115
AllowChangeOrientation.....	115
AllowChangePaper.....	116
AllowChangeScale.....	116
AllowCustomPaperSizes.....	116
ApplyToPrintDevice.....	44
AssignReportLinks.....	78
AssignStyles.....	134
AutoSave.....	129
AutoUpdateDateTime.....	68

B

Background.....	178
BeepAfterLongOperations.....	68
BeginClone.....	136
BeginUpdate.....	137, 174
Bottom.....	59
BottomRight.....	59
BuiltIn.....	116
ButtonsEnabled.....	142, 151
ButtonsVisible.....	145, 152

C

CancelUpdate.....	174
Caption.....	194
CenterOnPageH.....	34
CenterOnPageV.....	34
CenterTextAlignY.....	181
CenterTitle.....	182
Clear.....	135
Clone.....	54, 174
CloneStyleCaptionPrefix.....	129
Collate.....	153
Color.....	206
Copies.....	154
Count.....	129
CurrentLinkIndex.....	69
CurrentStyle.....	130
CurrentStyleIndex.....	130

D

Data.....	116
DefaultPrinterPage function.....	209
DefinePrintStylesDlg.....	138
Delete.....	135
DeleteAllLinks.....	78
DeleteLink.....	78
DeleteNonBuiltIns.....	135
Description.....	117
Designer.....	130
DesignerExists.....	79
DesignReport.....	79
DesignTimeRegistryPath.....	166
DialogData.....	154
DMPaper.....	35
DoApply.....	26
DoChange.....	26, 208
DoPaintEx.....	27
DoPrintDeviceBusy.....	84
DrawPageFooter.....	86
DrawPageHeader.....	86

dxPageSetup.....	210
------------------	-----

E

Empty.....	55, 64
EnableOptions.....	195
EndClone.....	137
EndUpdate.....	137, 175
EnumPagesAsImages.....	109
Execute.....	147, 162

F

FileList.....	155
FindLinkByComponent.....	86
FitToPagesByTall.....	35
FitToPagesByWide.....	35
FixMarginsOutside.....	44
Font.....	182, 206
Footer.....	36
FooterLoMetric.....	44
FooterRect.....	45
FooterRectLoMetric.....	45
FPageSetupDialogProc.....	211

G

GetDefaultMeasurementUnits function.....	209
GetFilteredPapers.....	120
GetInnerMeasurementUnits.....	45
GetRealMinMargins.....	46
GetRealRegistryPath.....	88
Getting Started.....	6
GrayShading.....	36

H

Header.....	36
HeaderLoMetric.....	46
HeaderRect.....	46
HeaderRectLoMetric.....	47
Height.....	59, 195
HelpContext.....	130, 144, 156, 196
HelpFile.....	166, 170
HFMode.....	144

I

Icon.....	196
ImageIndex.....	117
Images.....	131
Index.....	117
IndexOfLinkByName.....	89
IndexOfStyle.....	135
IsCurrentStyle.....	117
IsEqual.....	175
IsRebuildBeforePrint.....	90

L

Left.....	60, 196, 197
LeftTextAlignY.....	183
LeftTitle.....	184
LinkCount.....	70
LoadFromFile.....	138, 175
LoadFromStream.....	138, 175
LongOperationTime.....	70
LookAndFeel.....	166, 170

M

Margins.....	36
--------------	----

MarginsLoMetric	47
MaxRange	156
MeasurementUnits	37
MinMarginsLoMetric	47
MinPrintableArea	47
MinPrintableAreaLoMetric	48
MinRange	157

N

NonBuiltInsExists	136
-------------------------	-----

O

OnAfterGenerating	126
OnAfterPreview	97
OnAfterPrinting	126
OnBeforeGenerating	126
OnBeforePrinting	127
OnChange	30, 51
OnChangeComponent	98
OnChangeCurrentLink	98, 99
OnChangeCurrentStyle	140
OnChanged	57, 65
OnChanging	57, 65
OnClose	148, 163
OnCustomDrawPage	99
OnCustomDrawPageFooter	99
OnCustomDrawPageHeader	100
OnCustomDrawPreview	148
OnCustomDrawReportTitle	100
OnDesignReport	102
OnDestroy	123
OnExportGetPageFileName	112
OnExportPrepareGraphic	112
OnExportProgress	113
OnFilterPaper	123
OnGenerateReportProgress	103
OnPageSetup	163
OnPaint	31
OnPaintEx	31
OnShow	148, 163
OnStartGenerateReport	106
OnStartPrint	106
OnStyleListChanged	140
OptionsEnabled	143, 157
OptionsVisible	145, 158
Orientation	38
OverWriteExistingFiles	108

P

Page	178
PageCount	158
PageFooter	38
PageHeader	38
PageNumberFormat	71
PageNums	159
PageOrder	39
PageRanges	159
Pages	160
PageSetup	120
PageSetupDialog	131
PageSetupEx	91, 121
PageSize	40
PageSizeLoMetric	48
PageSizePixels	48
Paint	27
PaintEx	28
PaintRectLoMetric	48
PaintRectPixels	49
Point	53
Preview	92
PreviewBtnClicked	160
PreviewOptions	71
PreviewWindowDesigner	72

Print	93
PrintBtnClicked	160
PrinterPage	117
PrintEx	93
PrintFileList	72
PrintPage	94
PrintPages	94
PrintPagesEx	95
PrintStyle	144
PrintToFile	160

R

RealPageSize	41
RealPageSizeLoMetric	49
RealPageSizePixels	49
RealRegistryPath	167
RealTitle	147
Rect	60, 197
RegisterPreviewWindow	210
RegistryPath	167, 170, 197
RepaintNeeded	28
ReportLink	73
ReportLinkDesigner	73
RereadDefaultPrinterPage procedure	209
RestoreDefaults	122, 138
RestoreOriginalIcon	201
ReverseTitlesOnEvenPages	41
Right	60
RightTextAlignY	184
RightTitle	185

S

SaveFormsPosition	167, 171
SavePagesAsImagesToDisk	110
SavePosition	197
SaveToFile	139, 176
SaveToStream	139, 176
SaveZoomPosition	198
ScaleFactor	42
ScaleMode	42
SetupOptions	122
ShowPageSetupDlg	118
State	74, 118
StdProcessBusy	96
StorageName	131
StyleByCaption	136
StyleByName	136
StyleCaption	118
StyleGlyph	119
StyleManager	119, 161
Styles	131

T

TAbstractdxPreviewWindowDesigner	204
TBasedxPreviewWindow	203
TBasedxPrintStyle	
AllowChangeHFText	115
AllowChangeMargins	115
AllowChangeOrientation	115
AllowChangePaper	116
AllowChangeScale	116
AllowCustomPaperSizes	116
BuiltIn	116
Data	116
Description	117
GetFilteredPapers	120
ImageIndex	117
Index	117
IsCurrentStyle	117
OnDestroy	123
OnFilterPaper	123
PageSetup	120
PageSetupEx	121

PrinterPage.....	117	BkColor.....	21
RestoreDefaults.....	122	Brush.....	21
SetupOptions.....	122	DoApply.....	26
ShowPageSetupDlg.....	118	DoChange.....	26
State.....	118	DoPaintEx.....	27
StyleCaption.....	118	Effects.....	21
StyleGlyph.....	119	Mode.....	22
StyleManager.....	119	NeedRepaint.....	23
TCustomdxComponentPrinter		OnApply.....	30
BeepAfterLongOperations.....	68	OnChange.....	30
CurrentLinkIndex.....	69	OnPaint.....	31
DeleteLink.....	78	OnPaintEx.....	31
DrawPageFooter.....	86	Paint.....	27
DrawPageHeader.....	86	PaintEx.....	28
EnumPagesAsImages.....	109	Picture.....	23
FindLinkByComponent.....	86	PictureMode.....	23
GetRealRegistryPath.....	88	RepaintNeeded.....	28
LongOperationTime.....	70	TdxBaseObject.....	173
OnChangeCurrentLink.....	98	BeginUpdate.....	174
OnCustomDrawPage.....	99	CancelUpdate.....	174
OnCustomDrawPageFooter.....	99	Clone.....	174
OnCustomDrawPageHeader.....	100	EndUpdate.....	175
OnExportGetPageFileName.....	112	IsEqual.....	175
OnExportPrepareGraphic.....	112	LoadFromFile.....	175
OnExportProgress.....	113	LoadFromStream.....	175
OverWriteExistingFiles.....	108	SaveToFile.....	176
PageNumberFormat.....	71	SaveToStream.....	176
PageSetupEx.....	91	TdxComponentPrinter.....	107
SavePagesAsImagesToDisk.....	110	TdxPageSetupDialog	
TimeFormat.....	74	ActivePageIndex.....	142
TCustomdxComponentPrinter.....	67	ButtonsEnabled.....	142
AbortPrinting.....	68	ButtonsVisible.....	145
AutoUpdateDateTime.....	68	Execute.....	147
CurrentLink.....	69	HelpContext.....	144
Destroy.....	79	HFMode.....	144
DestroyReport.....	79	OnClose.....	148
DoPrintDeviceError.....	85	OnCustomDrawPreview.....	148
IsGenerateReportProgressEvent.....	89	OnShow.....	148
IsRebuildBeforePreview.....	90	OptionsEnabled.....	143
IsShowHourGlass.....	90	OptionsVisible.....	145
LinkByName.....	91	PrintStyle.....	144
OnAddReportLink.....	97	RealTitle.....	147
OnBeforePreview.....	98	Title.....	144
OnDeleteReportLink.....	101	TdxPointWrapper.....	52
OnEndGenerateReport.....	102	TdxPreviewOptions.....	193
OnEndPrint.....	102	HelpFile.....	196
OnGetPrintTitle.....	103	SavePosition.....	197
OnNewPage.....	104	Top.....	198
OnPageSetUp.....	104	VisibleOptions.....	198
OnPrintDeviceBusy.....	105	Width.....	199
OnPrintDeviceError.....	105	WindowState.....	199
Options.....	70, 71	TdxPrintDialog	
PrepareReport.....	92	ButtonsEnabled.....	151
PreviewWindow.....	72	ButtonsVisible.....	152
Print.....	93	Collate.....	153
PrintEx.....	93	Copies.....	154
PrintFileList.....	72	DialogData.....	154
PrintPage.....	94	Execute.....	162
PrintPages.....	94	FileList.....	155
PrintPagesEx.....	95	FileName.....	156
PrintTitle.....	73	HelpContext.....	156
RebuildReport.....	96	MaxRange.....	156
ReportLink.....	73	MinRange.....	157
ReportLinkDesigner.....	73	OnClose.....	163
State.....	74	OnPageSetup.....	163
TCustomdxPageObject		OnShow.....	163
CenterTextAlignY.....	181	OptionsEnabled.....	157
CenterTitle.....	182	OptionsVisible.....	158
Font.....	182	PageCount.....	158
LeftTextAlignY.....	183	PageNums.....	159
LeftTitle.....	184	PageRanges.....	159
RightTextAlignY.....	184	Pages.....	160
RightTitle.....	185	PreviewBtnClicked.....	160
TextAlignY.....	186	PrintBtnClicked.....	160
Titles.....	187	PrintToFile.....	160
TdxBackground.....	20	StyleManager.....	161

Title.....	161
UseFileList.....	161
TdxPrinterPage.....	33
ApplyToPrintDevice.....	44
CenterOnPageH.....	34
CenterOnPageV.....	34
FitToPagesByTall.....	35
FitToPagesByWide.....	35
FixMarginsOutside.....	44
Footer.....	36
FooterLoMetric.....	44
FooterRect.....	45
FooterRectLoMetric.....	45
GetInnerMeasurementUnits.....	45
GetRealMinMargins.....	46
GrayShading.....	36
Header.....	36
HeaderLoMetric.....	46
HeaderRect.....	46
HeaderRectLoMetric.....	47
MarginsLoMetric.....	47
MinMargins.....	37
MinMarginsLoMetric.....	47
MinPrintableArea.....	47
MinPrintableAreaLoMetric.....	48
OnChange.....	51
PageFooter.....	38
PageHeader.....	38
PageOrder.....	39
PageSizeLoMetric.....	48
PageSizePixels.....	48
PaintRectLoMetric.....	48
PaintRectPixels.....	49
RealMeasurementUnits.....	41
RealPageSize.....	41
RealPageSizeLoMetric.....	49
RealPageSizePixels.....	49
ReverseTitlesOnEvenPages.....	41
ScaleFactor.....	42
ScaleMode.....	42
ValidateMargins.....	50
TdxPrinterPageObject.....	
Background.....	178
Page.....	178
TdxPrintStyleManager.....	
AddStyle.....	134
AddStyleEx.....	134
AssignStyles.....	134
AutoSave.....	129
BeginClone.....	136
BeginUpdate.....	137
Clear.....	135
CloneStyleCaptionPrefix.....	129
Count.....	129
CurrentStyle.....	130
CurrentStyleIndex.....	130
DefinePrintStylesDlg.....	138
Delete.....	135
DeleteNonBuiltIns.....	135
Designer.....	130
EndClone.....	137
EndUpdate.....	137
HelpContext.....	130
Images.....	131
IndexOfStyle.....	135
LoadFromFile.....	138

LoadFromStream.....	138
NonBuiltInsExists.....	136
OnChangeCurrentStyle.....	140
OnStyleListChanged.....	140
PageSetupDialog.....	131
RestoreDefaults.....	138
SaveToFile.....	139
SaveToStream.....	139
StorageName.....	131
StyleByCaption.....	136
StyleByName.....	136
Styles.....	131
Title.....	132
UpdateCount.....	132
Version.....	132
TdxPSEngine.....	
DesignTimeRegistryPath.....	166
HelpFile.....	166
LookAndFeel.....	166
RealRegistryPath.....	167
RegistryPath.....	167
SaveFormsPosition.....	167
TdxPSEngineController.....	
Activate.....	172
HelpFile.....	170
LookAndFeel.....	170
RegistryPath.....	170
SaveFormsPosition.....	171
TdxPSPrintStyle.....	
OnAfterGenerating.....	126
OnAfterPrinting.....	126
OnBeforeGenerating.....	126
OnBeforePrinting.....	127
TdxRectWrapper.....	58
TextAlignX.....	206
TextAlignY.....	186, 207
TimeFormat.....	74
Title.....	132, 144, 161
Titles.....	187
Top.....	61
TopLeft.....	61

U

UnRegisterPreviewWindow.....	210
UpdateCount.....	132
UseFileList.....	161
Using the Express Printing System.....	6

V

ValidateMargins.....	50
Version.....	132

W

What is new in version 2.....	19
Width.....	61, 62

X

X.....	53
--------	----

Y

Y.....	53
--------	----